

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU TRIỂN KHAI AI AGENT ĐỂ
TỰ ĐỘNG HÓA QUY TRÌNH XÂY DỰNG
VIDEO BÀI GIẢNG**

Giảng viên hướng dẫn : **TS. NGUYỄN BẢO ÂN**

Sinh viên thực hiện: **NGUYỄN VĂN TỔNG**

Mã số sinh viên: **110122188**

Lớp : **DA22TTC**

Khoá : **2022**

Vĩnh Long, tháng 6 năm 2026

**ĐẠI HỌC TRÀ VINH
TRƯỜNG KỸ THUẬT VÀ CÔNG NGHỆ**



ĐỒ ÁN TỐT NGHIỆP

**NGHIÊN CỨU TRIỂN KHAI AI AGENT ĐỂ
TỰ ĐỘNG HÓA QUY TRÌNH XÂY DỰNG
VIDEO BÀI GIẢNG**

Giảng viên hướng dẫn : **TS. NGUYỄN BẢO ÂN**
Sinh viên thực hiện: **NGUYỄN VĂN TỔNG**
Mã số sinh viên: **110122188**
Lớp : **DA22TTC**
Khoá : **2022**

Vĩnh Long, tháng 6 năm 2026

LỜI CAM ĐOAN

Tôi xin cam đoan rằng khóa luận tốt nghiệp với đề tài "Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng" là công trình nghiên cứu do chính tôi thực hiện dưới sự hướng dẫn của thầy Nguyễn Bảo Ân. Tất cả các số liệu, kết quả và nội dung trình bày trong khóa luận này là trung thực và là sản phẩm của quá trình lao động học thuật nghiêm túc của cá nhân tôi. Mọi tài liệu tham khảo đều đã được trích dẫn rõ ràng và đầy đủ theo quy định. Tôi xin chịu hoàn toàn trách nhiệm về tính xác thực và nguyên bản của công trình nghiên cứu này.

Vĩnh Long, ngày...tháng...năm 2026

Sinh viên thực hiện

Nguyễn Văn Tổng

LỜI MỞ ĐẦU

Trí tuệ nhân tạo sinh hiện đã được sử dụng rộng rãi trong nhiều công đoạn sản xuất nội dung số, từ hỗ trợ viết kịch bản đến tổng hợp tri thức và tạo nội dung. Các mô hình ngôn ngữ lớn hiện nay có khả năng hỗ trợ xây dựng kịch bản, tổng hợp tri thức và tạo nội dung với tốc độ cao. Tuy nhiên, quá trình chuyển đổi từ tài liệu thô thành các video bài giảng hoàn chỉnh vẫn đòi hỏi nhiều thao tác thủ công như xây dựng kịch bản, tạo slide, ghi âm lời thuyết minh, đồng bộ hóa nội dung và biên tập video. Điều này làm gia tăng đáng kể thời gian và chi phí sản xuất nội dung giáo dục.

Xuất phát từ nhu cầu đó, đề tài “Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng” được thực hiện với mục tiêu xây dựng một nền tảng có khả năng tự động hóa phần lớn quy trình sản xuất video bài giảng. Trong phạm vi báo cáo, hệ thống nguyên mẫu được đặt tên là WebReel để thuận tiện cho việc trình bày kiến trúc, các phân hệ triển khai và kết quả thực nghiệm. Hệ thống cho phép tiếp nhận tài liệu đầu vào, sử dụng mô hình trí tuệ nhân tạo để tạo kịch bản, điều phối các tác vụ tự động hóa và kết xuất thành video hoàn chỉnh. Để kiểm chứng tính khả thi của hướng tiếp cận này, hệ thống được triển khai thử nghiệm trên môi trường máy chủ đám mây, sử dụng Docker để đóng gói Worker, Redis để điều phối tác vụ, MongoDB để lưu metadata và Cloudflare R2 để lưu video thành phẩm.

Bên cạnh việc tích hợp các công nghệ trí tuệ nhân tạo hiện có, trọng tâm của đề tài tập trung vào việc giải quyết các bài toán kỹ thuật phát sinh trong quá trình vận hành thực tế, bao gồm điều phối tác vụ bất đồng bộ, quản lý tài nguyên hệ thống, tái sử dụng trạng thái xác thực, đồng bộ hóa nội dung âm thanh – hình ảnh và nâng cao khả năng mở rộng trên hạ tầng điện toán đám mây có tài nguyên giới hạn.

Đề tài được triển khai theo hướng nghiên cứu ứng dụng, kết hợp giữa thiết kế kiến trúc hệ thống và thực nghiệm đánh giá hiệu năng. Các kết quả thu được chứng minh tính khả thi của hệ thống và cung cấp cơ sở thực nghiệm cho các quyết định thiết kế kiến trúc và khả năng vận hành trong môi trường thực tế.

LỜI CẢM ƠN

Trong suốt quá trình học tập và thực hiện đồ án tốt nghiệp, tôi đã nhận được nhiều sự hỗ trợ, hướng dẫn và động viên quý báu từ các cá nhân và tập thể.

Trước hết, tôi xin bày tỏ lòng biết ơn sâu sắc tới quý thầy cô của Khoa Công nghệ thông tin đã truyền đạt những kiến thức nền tảng và chuyên môn trong suốt quá trình học tập. Những kiến thức này là cơ sở quan trọng giúp tôi hoàn thành đề tài nghiên cứu và phát triển hệ thống.

Đặc biệt, tôi xin gửi lời cảm ơn chân thành tới giảng viên hướng dẫn đã dành thời gian góp ý, định hướng và hỗ trợ tôi trong suốt quá trình thực hiện đồ án. Những nhận xét và hướng dẫn của thầy/cô đã giúp tôi hoàn thiện cả về mặt học thuật lẫn kỹ thuật của đề tài.

Tôi cũng xin cảm ơn gia đình và bạn bè đã luôn động viên, hỗ trợ và tạo điều kiện thuận lợi để tôi có thể tập trung hoàn thành chương trình học cũng như đồ án tốt nghiệp này.

Mặc dù đã nỗ lực hoàn thiện đề tài trong phạm vi thời gian và nguồn lực cho phép, báo cáo khó tránh khỏi những thiếu sót nhất định. Tôi rất mong nhận được những ý kiến đóng góp từ quý thầy cô và Hội đồng để tiếp tục hoàn thiện kiến thức và kỹ năng của bản thân.

Xin chân thành cảm ơn.

Sinh viên thực hiện

Nguyễn Văn Tổng

NHẬN XÉT
(Của cơ quan thực tập, nếu có)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

NHẬN XÉT

(Của giảng viên hướng dẫn trong đề án, khoá luận của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giảng viên hướng dẫn
(ký và ghi rõ họ tên)

Nguyễn Bảo Ân

UBND TỈNH VĨNH LONG
ĐẠI HỌC TRÀ VINH

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập – Tự do – Hạnh Phúc

BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP
(Của giảng viên hướng dẫn)

Họ và tên sinh viên: Nguyễn Văn Tổng

MSSV: 110122188

Ngành: Công nghệ thông tin

Khóa: 2022

Tên đề tài: Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng

Họ và tên Giáo viên hướng dẫn:.....

Chức danh:..... Học vị:.....

NHẬN XÉT

1. Nội dung đề tài:

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

2. Ưu điểm:

.....

.....

.....

3. Khuyết điểm:

.....

.....

.....

.....

4. Điểm mới đề tài:

.....

.....

.....

.....

.....

5. Giá trị thực trên đề tài:

.....

.....

.....

.....

.....

.....

.....

7. Đề nghị sửa chữa bổ sung:

.....

.....

.....

.....

.....

.....

.....

8. Đánh giá:

.....

.....

.....

.....

Vĩnh Long, ngày tháng năm 20...
Giảng viên hướng dẫn
(Ký & ghi rõ họ tên)

NHẬN XÉT

(Của giảng viên chấm trong đồ án, khoá luận của sinh viên)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

Giảng viên chấm
(ký và ghi rõ họ tên)

UBND TỈNH VĨNH LONG
ĐẠI HỌC TRÀ VINH

CỘNG HOÀ XÃ HỘI CHỦ NGHĨA VIỆT NAM
Độc lập - Tự do - Hạnh phúc

BẢN NHẬN XÉT ĐỒ ÁN, KHÓA LUẬN TỐT NGHIỆP
(Của cán bộ chấm đồ án, khóa luận)

Họ và tên người nhận xét:
Chức danh: Học vị:
Chuyên ngành:
Cơ quan công tác:
Tên sinh viên:
Tên đề tài đồ án, khóa luận tốt nghiệp:
.....
.....

I. Ý KIẾN NHẬN XÉT

1. Nội dung:
-
.....
.....
.....
.....
.....
.....
.....
.....
2. Điểm mới các kết quả của đồ án, khóa luận:
-
.....
.....
3. Ứng dụng thực tế:
-
.....
.....
.....
.....

II. CÁC VẤN ĐỀ CẦN LÀM RÕ

(Các câu hỏi của giáo viên phản biện)

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

.....

III. KẾT LUẬN

(Ghi rõ đồng ý hay không đồng ý cho bảo vệ đề án khóa luận tốt nghiệp)

.....

.....

.....

.....

.....

....., ngày tháng năm 20...

Người nhận xét

(Ký & ghi rõ họ tên)

MỤC LỤC

LỜI CAM ĐOAN	iii
LỜI MỞ ĐẦU	iv
LỜI CẢM ƠN	v
NHẬN XÉT	vi
NHẬN XÉT	vii
MỤC LỤC	xiii
DANH MỤC BẢNG BIỂU	xvii
DANH MỤC HÌNH ẢNH	xviii
KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT	xix
CHƯƠNG 1 GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Bối cảnh và tính cấp thiết của đề tài	1
1.2 Tổng quan các nghiên cứu và giải pháp liên quan	1
1.2.1 Nhóm nền tảng tạo video AI thương mại	2
1.2.2 Nhóm nghiên cứu về Tác nhân AI và tự động hóa trình duyệt	2
1.2.3 Nhóm nghiên cứu tự động hóa sản xuất học liệu	2
1.2.4 Đánh giá đối sánh và khoảng trống nghiên cứu.....	3
1.3 Mục tiêu đề tài.....	3
1.4 Đối tượng và phạm vi nghiên cứu.....	4
1.5 Phương pháp nghiên cứu.....	4
1.6 Cấu trúc báo cáo.....	5
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT.....	6
2.1 Tác nhân trí tuệ nhân tạo và kiến trúc ReAct.....	6
2.2 Mô hình ngôn ngữ lớn và kỹ thuật thiết kế câu lệnh	7
2.3 Tự động hóa quy trình nhận thức	8
2.4 Kiến trúc hệ thống phân tán và Microservices.....	10
2.5 Nguyên lý xử lý và đồng bộ hóa tín hiệu đa phương tiện số	11
2.6 Các công nghệ và công cụ hỗ trợ	12
2.6.1 Công nghệ Container hóa Docker.....	12
2.6.2 Framework phát triển API bất đồng bộ FastAPI	13

2.6.3 Hệ quản trị cơ sở dữ liệu MongoDB và Redis.....	13
2.6.4 Mô hình trí tuệ nhân tạo đa phương thức Google Gemini.....	14
2.6.5 Tự động hóa trình duyệt Playwright và Browser-Use	14
2.6.6 Khung xử lý đa phương tiện FFmpeg và Edge-TTS	15
2.6.7 Tự động hóa hệ điều hành với Pywinauto	15
2.6.8 Hạ tầng lưu trữ đối tượng đám mây Cloudflare R2.....	16
CHƯƠNG 3 HIỆN THỰC HÓA NGHIÊN CỨU	17
3.1 Phân tích yêu cầu hệ thống.....	17
3.1.1 Yêu cầu chức năng.....	17
3.1.2 Yêu cầu phi chức năng.....	18
3.1.3 Yêu cầu bảo mật và phân quyền	19
3.1.4 Mô hình hóa ca sử dụng.....	19
3.2 Thiết kế kiến trúc tổng thể	23
3.2.1 Phân rã kiến trúc vi dịch vụ và các lớp thành phần	24
3.2.2 Cơ chế Điều phối thông điệp hướng sự kiện	26
3.2.3 Sơ đồ luồng dữ liệu tổng quát.....	28
3.2.4 Điểm dừng kiểm duyệt và Tiền đề mở rộng	31
3.3 Thiết kế cơ sở dữ liệu và hạ tầng lưu trữ	31
3.3.1 Thiết kế lược đồ dữ liệu tài liệu.....	31
3.3.2 Chiến lược thiết lập chỉ mục	36
3.3.3 Kiến trúc lưu trữ đối tượng đám mây	37
3.4 Thiết kế chi tiết các phân hệ thực thi	38
3.4.1 Phân hệ Web Worker quy trình 6 pha.....	39
3.4.2 Phân hệ Presentation Workers	42
3.4.3 Phân hệ OS Worker	44
3.4.4 Phân tích kiến trúc Worker đa môi trường	46
3.5 Thiết kế giao diện người dùng và luồng tương tác	48
3.5.1 Kiến trúc bảng điều khiển tập trung.....	48
3.5.2 Luồng giao tiếp đồng bộ tiến độ qua WebSocket/SSE.....	50
3.5.3 Thiết kế điểm dừng kiểm duyệt phân tán	52
3.6 Thiết kế giải pháp bảo mật và an toàn hệ thống.....	53
3.6.1 Bảo mật tầng nhận thức AI	53

3.6.2 Bảo mật hệ thống tệp tin	54
3.6.3 Bảo mật kiến trúc mạng phân tán và API	55
3.6.4 Phòng thủ trạng thái trình duyệt và cắt mạch khẩn cấp	56
3.7 Các quyết định kiến trúc và đánh đổi kỹ thuật	58
3.7.1 Phân rã vi dịch vụ thay vì nguyên khối	59
3.7.2 Lựa chọn Redis làm trung tâm điều phối thông điệp	59
3.7.3 Cơ sở thiết lập các tham số vật lý của hệ thống	60
3.7.4 Lựa chọn kiến trúc điểm dừng kiểm duyệt	62
3.8 Môi trường triển khai và công cụ phát triển	63
3.8.1 Môi trường phần cứng và nền tảng Đám mây	63
3.8.2 Ngôn ngữ lập trình, công cụ và thư viện lõi	63
3.9 Triển khai các phân hệ cốt lõi	64
3.9.1 Triển khai Giao diện Bảng điều khiển	64
3.9.2 Triển khai luồng điều phối ngầm và quản lý phiên	66
3.9.3 Triển khai trình kết xuất đa phương tiện	67
CHƯƠNG 4 KẾT QUẢ NGHIÊN CỨU	69
4.1 Kết quả nghiệm thu bảo mật	69
4.1.1 Nghiệm thu vành đai an toàn hạ tầng	69
4.1.2 Kiểm thử chống tiêm nhiễm lời nhắc	69
4.1.3 Kiểm thử giới hạn lưu lượng và làm sạch tệp tin	69
4.2 Phân tích sự cố mất đồng bộ đa phương tiện và giải pháp giải quyết	70
4.3 Thực nghiệm và đánh giá định lượng	72
4.3.1 Đánh giá hiệu năng cơ chế Session Snapshot	72
4.3.2 Đánh giá chiến lược dòng thời gian âm thanh chủ đạo	73
4.3.3 Đánh giá sức chịu tải của hệ thống phân tán	74
4.3.4 Đánh giá pipeline xử lý bài giảng từ Google Slides	76
CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN	79
5.1 Các đóng góp kiến trúc cốt lõi của đề tài	79
5.1.1 Kiến trúc phân tán đa môi trường thực thi	79
5.1.2 Cơ chế điểm dừng kiểm duyệt trong luồng bất đồng bộ	79
5.1.3 Cơ chế Browser Session Snapshot	79

5.1.4 Cơ chế điều phối dòng thời gian âm thanh chủ đạo	80
5.2 Kết quả đạt được tổng thể	80
5.3 Hạn chế của hệ thống	80
5.4 Hướng phát triển	80
DANH MỤC TÀI LIỆU THAM KHẢO	82

DANH MỤC BẢNG BIỂU

Bảng 1.1 So sánh các tính năng lõi giữa các nền tảng và nghiên cứu liên quan.....	3
Bảng 3.1 Đặc tả Ca sử dụng Đăng nhập/ Đăng ký.....	21
Bảng 3.2 Đặc tả Ca sử dụng Gửi yêu cầu tạo video.....	21
Bảng 3.3 Đặc tả ca sử dụng Duyệt và chỉnh sửa kịch bản	22
Bảng 3.4 Đặc tả Ca sử dụng Xem/ Tải về video kết quả.....	22
Bảng 3.5 Đặc tả Ca sử dụng Quản lý người dùng.....	23
Bảng 3.6 Đặc tả Ca sử dụng Quản lý phiên trình duyệt gốc	23
Bảng 3.7 Đặc tả các trường dữ liệu Collection users	32
Bảng 3.8 Đặc tả các trường dữ liệu cốt lõi Collection jobs.....	35
Bảng 3.9 So sánh các worker	47
Bảng 4.1 Hiệu năng khởi tạo phiên làm việc	73
Bảng 4.2 So sánh hiệu năng theo tốc độ khung hình	73
Bảng 4.3 Kết quả thử nghiệm tải.....	74
Bảng 4.4 Hiệu năng pipeline Presentation_gg theo cấu hình vận hành	77

DANH MỤC HÌNH ẢNH

Hình 2.1 Kiến trúc ReAct.....	7
Hình 2.2 Tự động hóa bằng robot	9
Hình 3.1 Sơ đồ usecase tổng quát.....	20
Hình 3.2 Kiến trúc hệ thống và các lớp thành phần	24
Hình 3.3 Sơ đồ tuần tự quá trình điều phối bất đồng bộ	27
Hình 3.4 Sơ đồ luồng dữ liệu tổng quát	29
Hình 3.5 Sơ đồ quy tắc vòng đời đối tượng	38
Hình 3.6 Quy trình điều hướng trình duyệt.....	40
Hình 3.7 Quy trình biên dịch dữ liệu.....	40
Hình 3.8 Quy trình tổng hợp và đo lường vi giây	41
Hình 3.9 Quy trình tự động hóa ở cấp độ hệ điều hành	44
Hình 3.10 Phác thảo giao diện khởi tạo tác vụ.....	48
Hình 3.11 Chi tiết job	49
Hình 3.12 Giao diện Bảng điều khiển tập trung.....	65
Hình 3.13 Biểu mẫu kiểm duyệt chờ tương tác người dùng.	65
Hình 3.14 Nhật ký hệ thống luồng phân phối tác vụ bất đồng bộ qua Redis.....	66
Hình 3.15 Trạm quản lý phiên làm việc đồ họa từ xa.	66
Hình 3.16 Nhật ký tiến trình FFmpeg thực thi thuật toán Stream Copy.	67
Hình 3.17 Chi tiết thuộc tính của tệp video thành phẩm.	67
Hình 4.1 Cơ chế tự động đóng băng hệ thống khi phát hiện mật khẩu yếu.	69
Hình 4.2 Trạng thái HTTP 429 minh chứng hệ thống Rate Limiter.	70
Hình 4.3 Throughput theo số lượng Worker	75
Hình 4.4 Mức sử dụng CPU theo số lượng Worker	75

KÍ HIỆU CÁC CỤM TỪ VIẾT TẮT

STT	Từ viết tắt	Cụm từ đầy đủ
1	AI	Artificial Intelligence
2	LLM	Large Language Model
3	RPA	Robotic Process Automation
4	API	Application Programming Interface
5	JSON	JavaScript Object Notation
6	HTML	HyperText Markup Language
7	DOM	Document Object Model
8	CDP	Chrome DevTools Protocol
9	CPU	Central Processing Unit
10	RAM	Random Access Memory
11	VPS	Virtual Private Server
12	TTS	Text-To-Speech
13	SSE	Server-Sent Events
14	HTTP	HyperText Transfer Protocol
15	HTTPS	HyperText Transfer Protocol Secure
16	URL	Uniform Resource Locator
17	OTP	One-Time Password
18	PDF	Portable Document Format
19	FPS	Frames Per Second
20	I/O	Input/Output
21	Pub/Sub	Publish/Subscribe
22	OS	Operating System

23	GAN	Generative Adversarial Network
24	BSON	Binary JSON
25	RBAC	Role-Based Access Control
26	UUID	Universally Unique Identifier
27	CDN	Content Delivery Network
28	UIA	UI Automation
29	DDoS	Distributed Denial of Service
30	SSH	Secure Shell
31	MP3	MPEG Audio Layer III
32	MP4	MPEG-4 Part 14
33	PPTX	PowerPoint Open XML Presentation
34	Xvfb	X Virtual Framebuffer
35	noVNC	HTML5 Virtual Network Computing Client
36	ASGI	Asynchronous Server Gateway Interface
37	ODM	Object Document Mapper
38	GPU	Graphics Processing Unit
39	JWT	JSON Web Token

CHƯƠNG 1 GIỚI THIỆU ĐỀ TÀI

1.1 Bối cảnh và tính cấp thiết của đề tài

Trong giáo dục trực tuyến, video bài giảng là dạng học liệu được sử dụng phổ biến vì giúp người học quan sát trực tiếp nội dung và thao tác thực hành. Tuy nhiên, thực trạng sản xuất video giáo dục hiện nay đang bộc lộ một điểm nghẽn về mặt hiệu suất và chi phí. Quá trình sản xuất một video bài giảng tiêu chuẩn đòi hỏi quy trình thủ công và tiêu tốn nhiều nguồn lực: từ khâu soạn thảo kịch bản chi tiết, ghi hình màn hình thao tác, thu âm lời bình, cho đến công đoạn hậu kỳ cắt ghép và đồng bộ hóa âm thanh với hình ảnh theo thời gian thực. Đối với các cơ sở giáo dục, chuyên gia đào tạo hay thậm chí là các doanh nghiệp công nghệ cần sản xuất tài liệu hướng dẫn nội bộ, quy trình này tạo ra gánh nặng tài chính và làm chậm trễ khả năng cập nhật và mở rộng kho học liệu số.

Cùng với sự phát triển vượt bậc của Trí tuệ nhân tạo, đặc biệt là các Mô hình ngôn ngữ lớn và hệ thống Tác nhân tự trị, cơ hội tự động hóa các tác vụ số trên máy tính đang trở nên rõ ràng hơn. Mặc dù trên thị trường đã xuất hiện nhiều công cụ ứng dụng AI để tạo video, phần lớn vẫn dừng lại ở việc sinh tạo video từ văn bản tĩnh hoặc yêu cầu con người can thiệp sâu vào khâu thiết lập. Vẫn tồn tại một khoảng trống lớn về một hệ thống có khả năng tự động hóa toàn trình: tự đọc hiểu tài liệu, tự lập kế hoạch, trực tiếp điều khiển trình duyệt để ghi hình thao tác thực tế và tự động đồng bộ hóa đa phương tiện.

Xuất phát từ thực tiễn trên, việc nghiên cứu và xây dựng một nền tảng ứng dụng Tác nhân AI để tự động hóa quy trình sản xuất video hướng dẫn và bài giảng là một bài toán kỹ thuật có tính ứng dụng và là một nhu cầu cấp thiết. Giải pháp này hướng tới giảm khối lượng thao tác thủ công, rút ngắn thời gian sản xuất học liệu và hỗ trợ mở rộng quy trình tạo video bài giảng.

1.2 Tổng quan các nghiên cứu và giải pháp liên quan

Để định vị chính xác khoảng trống công nghệ mà hệ thống WebReel cần giải quyết, các giải pháp và nghiên cứu hiện có được phân loại thành ba nhóm cốt lõi dựa trên mục tiêu và nền tảng kỹ thuật.

1.2.1 Nhóm nền tảng tạo video AI thương mại

Các hệ thống tiêu biểu trên thị trường hiện nay như HeyGen, Synthesia hay D-ID đang được sử dụng phổ biến trong nhóm công cụ sinh tạo video. Mặc dù các nền tảng này sở hữu khả năng kết xuất hình ảnh ổn định - điển hình như HeyGen đạt độ trễ đồng bộ khẩu hình chỉ ở mức 0.02 giây, hay Synthesia cung cấp bộ công cụ tối ưu cho doanh nghiệp - nhưng bản chất kiến trúc của chúng vẫn là các công cụ kết xuất một chiều. Người dùng buộc phải chuẩn bị sẵn kịch bản và thiết lập môi trường bằng tay. Hạn chế của nhóm này là sự vắng bóng của một Tác nhân AI đóng vai trò điều phối. Hệ thống không có khả năng tự đọc hiểu tài liệu đầu vào, không tự lập kế hoạch và các nhân vật ảo chỉ đứng yên thuyết trình mà không thể thao tác trực tiếp trên các phần mềm hay ứng dụng.

1.2.2 Nhóm nghiên cứu về Tác nhân AI và tự động hóa trình duyệt

Ở góc độ học thuật về tự động hóa, bài toán điều khiển trình duyệt bằng Tác nhân AI đang được quan tâm trong các nghiên cứu gần đây. Nghiên cứu về WebAgent (Gur và cộng sự, 2023) đã áp dụng kiến trúc tích hợp hai mô hình ngôn ngữ lớn để lập kế hoạch và tổng hợp mã HTML, giúp tăng tỷ lệ thành công lên 50% trên các môi trường web thực tế. Tương tự, các kiến trúc đa phương thức như SeeAct (Zheng và cộng sự, 2024) hay WebVoyager (He và cộng sự, 2024) đã đạt được tỷ lệ hoàn thành tác vụ lên tới 59.1% nhờ kết hợp khả năng phân tích hình ảnh và cấu trúc DOM thông qua framework Playwright. Ngay cả thư viện mã nguồn mở browser-use - lớp thực thi lỗi mà đề án này kế thừa - cũng cho thấy năng lực tự chủ cao. Tuy nhiên, khoảng trống lớn của nhóm nghiên cứu này là chúng chỉ tập trung giải quyết việc “hoàn thành một tác vụ web” (ví dụ: đặt vé, điền form) chứ không được thiết kế kiến trúc đường ống để ghi hình, phối trộn đa phương tiện hay xuất bản học liệu.

1.2.3 Nhóm nghiên cứu tự động hóa sản xuất học liệu

Đi sâu vào bài toán giáo dục, một số nghiên cứu gần đây đã nỗ lực tự động hóa quy trình tạo bài giảng. Hệ thống AutoLV (Wang và cộng sự, 2022) ứng dụng mạng GAN để tạo nhân vật thuyết trình từ các slide đã được con người gắn nhãn thủ công. Gần đây, nghiên cứu AutoLectures (2025) cung cấp một giải pháp tự động sinh lời bình từ slide tĩnh và dùng siêu dữ liệu thời gian của mô hình TTS để đồng bộ, đạt điểm F1 trên 92% ở khâu khớp nhịp. Dù tiến rất gần đến khái niệm tự động hóa, nhược điểm

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
 chung của các nghiên cứu này - bao gồm cả mô hình bán tự động của Zhang và cộng sự (2025) - là chúng chỉ xử lý được các tài liệu tĩnh (slide đã render thành ảnh hoặc PDF). Các hệ thống này chưa phù hợp để ghi lại quy trình làm việc thực tế trên các phần mềm đang vận hành (như các hiệu ứng animation gốc của Google Slides hay các thao tác nhấp chuột trên một nền tảng quản trị).

1.2.4 Đánh giá đối sánh và khoảng trống nghiên cứu

Từ những phân tích trên, đề án tiến hành tổng hợp mức độ đáp ứng của các nhóm giải pháp thông qua Bảng 1.1.

Bảng 1.1 So sánh các tính năng lõi giữa các nền tảng và nghiên cứu liên quan

Tiêu chí	HeyGen / Synthesia	WebAgent / SeeAct	AutoLectures	WebReel (Đề xuất)
Tự động trích xuất nội dung	Không	Có	Một phần	Có
AI Agent điều phối toàn trình	Không	Có	Không	Có
Ghi hình thao tác phần mềm thực	Không	Không	Không	Có
Đồng bộ âm - hình theo thời gian	Có	Không	Có	Có
Cơ chế kiểm duyệt	Không	Không	Không	Có

Khoảng trống nghiên cứu: Thông qua việc phân tích điểm nghẽn của các hệ thống hiện hữu, có thể khẳng định khoảng trống mà WebReel lấp đầy là việc xây dựng một hệ thống kết hợp thành công cả bốn yếu tố kỹ thuật: Tác nhân AI tự lập kế hoạch từ ngữ cảnh, khả năng thao tác trực tiếp trên phần mềm đang vận hành, thuật toán đồng bộ âm thanh và hình ảnh dựa trên thời gian vật lý, cùng với cơ chế điểm dừng kiểm duyệt nằm gọn trong một đường ống tự động hóa toàn trình.

1.3 Mục tiêu đề tài

Mục tiêu của đề án là xây dựng một hệ thống nguyên mẫu có khả năng tự động hóa phần lớn quy trình tạo video bài giảng, từ tiếp nhận tài liệu, phân tích nội dung, sinh kịch bản, điều khiển trình duyệt đến kết xuất video thành phẩm. Thay vì yêu cầu

người dùng phải tự lắp ráp kịch bản và căn chỉnh thời gian thủ công, hệ thống phải hướng tới tự động hóa phần lớn quy trình sản xuất video bài giảng: từ lúc AI tự đọc hiểu cấu trúc tệp Google Slides hoặc giao diện trang web, cho đến khâu điều khiển trình duyệt và xuất ra thành phẩm cuối cùng. Một mục tiêu khác của đề án là phải có cơ chế đồng bộ thời gian thực giữa hình ảnh và âm thanh. Cụ thể, kiến trúc xử lý luồng phải đảm bảo độ khớp giữa thao tác thị giác và lời thoại thuyết minh và giảm sai lệch đồng bộ xuống mức mili giây trong điều kiện thử nghiệm, ngay cả khi toàn bộ quá trình kết xuất bị ép chạy trong các môi trường ảo hóa vùng chứa có giới hạn về sức mạnh tài nguyên.

1.4 Đối tượng và phạm vi nghiên cứu

Về mặt đối tượng, nghiên cứu xoáy sâu vào cơ chế vận hành của các kiến trúc Tác nhân, kỹ thuật thao tác trình duyệt và các thuật toán xử lý dữ liệu truyền thông đa phương tiện lỗi thông qua FFmpeg. Tuy nhiên, để đảm bảo tính khả thi và tập trung khắc phục các điểm nghẽn kỹ thuật, phạm vi ứng dụng của hệ thống được giới hạn một cách có chủ đích. Phiên bản WebReel hiện tại chỉ tập trung giải quyết bài toán sản xuất video dựa trên hai nguồn đầu vào cốt lõi: nền tảng trình chiếu Google Slides và các ứng dụng web tuân thủ chuẩn HTML và các phần mềm máy tính cục bộ hoặc hệ điều hành. Toàn bộ các tương tác nằm ngoài ranh giới của trình duyệt web và máy tính cục bộ sẽ nằm ngoài phạm vi hỗ trợ của đề tài này.

1.5 Phương pháp nghiên cứu

Quá trình nghiên cứu được thực hiện theo hướng xây dựng nguyên mẫu và kiểm thử lặp lại. Sau mỗi lần triển khai, hệ thống được chạy thử với các tác vụ tạo video thực tế để ghi nhận lỗi, đo thời gian xử lý, mức sử dụng CPU/RAM và độ ổn định của Worker. Bắt đầu từ việc nghiên cứu lý thuyết các bộ thư viện mã nguồn mở và giao thức Chrome DevTools (CDP), hệ thống được mô hình hóa theo tư duy phân tán để quy hoạch rõ ràng các luồng giao tiếp giữa API Gateway và các Worker. Trọng tâm của phương pháp nằm ở giai đoạn triển khai thực tiễn: mã nguồn nguyên mẫu liên tục được đẩy lên môi trường đám mây ảo hóa để chạy tải thật. Thông qua việc phân tích nhật ký hệ thống, đo lường độ trễ mạng và xác định các điểm thắt cổ chai I/O, đề án liên tục tinh chỉnh các thông số luồng và tung ra các bản vá cho đến khi hệ thống đạt được độ ổn định và chính xác mục tiêu.

1.6 Cấu trúc báo cáo

Toàn bộ hành trình phân tích và xây dựng hệ thống được trình bày qua 5 chương, bám sát luồng tư duy từ lý thuyết đến thực tiễn vận hành.

Chương 1: Giới thiệu đề tài: Trình bày bối cảnh, phân tích các nghiên cứu liên quan để làm nổi bật tính cấp thiết và xác định mục tiêu của đề án.

Chương 2: Cơ sở lý thuyết: Cung cấp nền tảng kiến thức về kiến trúc Tác nhân AI, công nghệ vi dịch vụ và các bộ thư viện lõi được sử dụng trong hệ thống.

Chương 3: Hiện thực hóa nghiên cứu: Đặc tả và triển khai về quy hoạch vi dịch vụ, luồng hoạt động tự động hóa 6 pha và các cơ chế chịu lỗi, bảo mật của WebReel.

Chương 4: Kết quả nghiên cứu: Trình bày các chỉ số thực nghiệm khi hệ thống vận hành trên môi trường đám mây, tập trung vào phân tích đối sánh và các giải pháp khắc phục sự cố rớt khung hình.

Chương 5: Kết luận và Hướng phát triển: Tổng kết các giá trị đề án mang lại, nhìn nhận thẳng thắn các giới hạn về tài nguyên và đề xuất lộ trình mở rộng nền tảng trong tương lai.

Để hiện thực hóa mục tiêu tự động hóa toàn trình đã đặt ra, giải pháp đòi hỏi sự hội tụ và phối hợp chặt chẽ của nhiều nền tảng công nghệ lõi. Do đó, việc thấu hiểu bản chất vận hành của Tác nhân AI, kiến trúc vi dịch vụ và kỹ thuật xử lý đa phương tiện sẽ được làm rõ ngay ở chương sau.

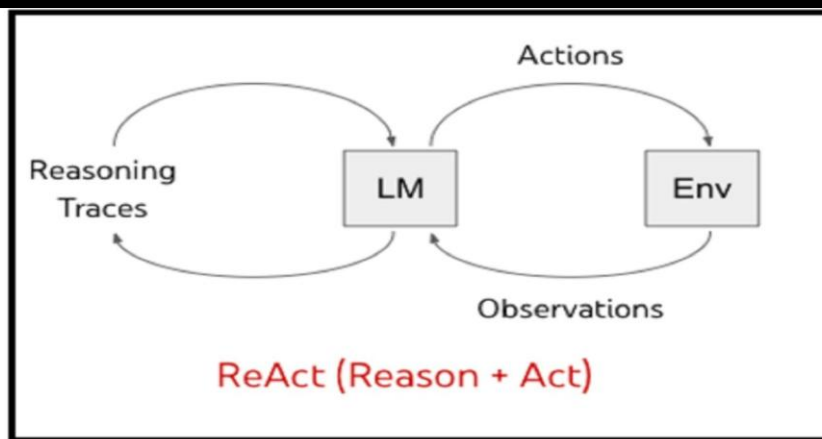
CHƯƠNG 2 CƠ SỞ LÝ THUYẾT

2.1 Tác nhân trí tuệ nhân tạo và kiến trúc ReAct

Trong khoa học máy tính, Tác nhân trí tuệ nhân tạo (AI Agent) được định nghĩa là một thực thể tính toán có khả năng tự chủ, nhận thức được trạng thái của môi trường xung quanh và đưa ra các quyết định hành động để đạt được một hoặc nhiều mục tiêu được yêu cầu. Các Mô hình ngôn ngữ lớn thuần túy hoạt động theo cơ chế phản hồi bằng cách tiếp nhận văn bản và xuất văn bản dựa trên xác suất thống kê, AI Agent mở rộng giới hạn này bằng cách sử dụng LLM như một bộ não để điều phối các công cụ ngoại vi. Sự phát triển cấu trúc LLM cơ bản sang hệ thống AI Agent đánh dấu bước tiến từ việc sinh văn bản sang thực thi tác vụ, cho phép phần mềm chủ động can thiệp và thay đổi trạng thái của môi trường bên ngoài thông qua các giao diện lập trình ứng dụng hoặc kịch bản tự động hóa. Trong WebReel, AI Agent được thể hiện rõ nhất ở các Worker, nơi LLM phân tích ngữ cảnh đầu vào và sinh ra kịch bản thao tác cho các bước tự động hóa phía sau.

Để giải quyết bài toán suy luận và ra quyết định trong các môi trường phức tạp, hệ thống áp dụng kiến trúc ReAct, một phương pháp luận được đề xuất bởi Yao và cộng sự (2022). Điểm đặc biệt của kiến trúc ReAct nằm ở khả năng đan xen đồng thời các luồng suy luận logic và các hành động tương tác với môi trường. Thay vì yêu cầu mô hình giải quyết toàn bộ bài toán trong một lần tính toán – rất dễ dẫn đến sự suy giảm tính logic và phát sinh ảo giác thông tin – ReAct chia nhỏ tiến trình thành một vòng lặp tuần tự và khép kín [1].

Vòng lặp cơ sở của ReAct vận hành dựa trên ba trạng thái tương hỗ: Suy luận, Hành động và Quan sát. Bắt đầu chu trình, ở trạng thái Suy luận, tác nhân phân tích ngữ cảnh hiện tại, phân rã mục tiêu lớn thành các tác vụ nhỏ và xác định công cụ phù hợp để thực thi. Tiếp theo, trạng thái Hành động được kích hoạt để thực thi công cụ đã chọn, chẳng hạn như truy vấn cơ sở dữ liệu, điều khiển trình duyệt ảo hoặc gọi một hàm trích xuất dữ liệu. Kết quả trả về từ môi trường thực tế được hệ thống ghi nhận ở trạng thái Quan sát. Việc trạng thái Quan sát cung cấp các dữ liệu xác định từ thế giới thực, được dùng làm cơ sở thực chứng để mô hình hiệu chỉnh lại chuỗi suy luận tiếp theo, thay vì dựa vào các dự đoán xác suất.



Hình 2.1 Kiến trúc ReAct

Trong ngữ cảnh tự động hóa quy trình xây dựng video bài giảng, kiến trúc ReAct cung cấp nền tảng lý thuyết để thiết kế luồng thực thi. Khi hệ thống tiếp nhận một tệp tài liệu, Tác nhân không tự phỏng đoán nội dung mà sẽ liên tục gọi các công cụ bóc tách dữ liệu, quan sát kết quả trả về, sau đó mới suy luận để sinh lời thoại hoặc điều hướng sang các phân đoạn kế tiếp. Cơ chế đan xen giữa suy luận và hành động này giúp duy trì tính chính xác của luồng nghiệp vụ, cấp cho hệ thống khả năng tự phục hồi khi gặp lỗi ngoại lệ, đáp ứng các tiêu chuẩn của một phần mềm tự động hóa theo hướng nhận thức.

2.2 Mô hình ngôn ngữ lớn và kỹ thuật thiết kế câu lệnh

Trong cấu trúc của một hệ thống tác nhân thông minh, Mô hình ngôn ngữ lớn là bộ xử lý chính, chịu trách nhiệm diễn giải ngôn ngữ tự nhiên và thực hiện các thao tác suy luận logic. Khác với các mô hình máy học truyền thống dựa trên các tập dữ liệu có nhãn, LLM hiện đại được xây dựng dựa trên kiến trúc Transformer với quy mô hàng tỷ tham số, cho phép mô hình nắm bắt được các quy luật ngôn ngữ phức tạp và sở hữu khả năng “học trong ngữ cảnh”. Đặc biệt, với sự xuất hiện của các dòng mô hình đa phương thức, LLM không còn giới hạn ở việc xử lý văn bản thuần mà có khả năng phân tích hình ảnh, cấu trúc phân cấp trong tài liệu trình chiếu và logic bố cục của các giao diện web, tạo tiền đề cho việc bóc tách nội dung bài giảng một cách tự động.

Tuy nhiên, năng lực của LLM chỉ có thể được khai thác tối đa thông qua kỹ thuật thiết kế câu lệnh - một phương pháp tiếp cận có hệ thống nhằm định hướng hành vi của mô hình mà không cần đào tạo lại. Trong kiến trúc của hệ thống, Prompt Engineering được dùng để soạn thảo yêu cầu đầu vào, quy định định dạng đầu ra, giới hạn phạm vi phản hồi và định hướng hành vi của mô hình. Thành phần tương đối quan

trọng trong quy trình này là việc xây dựng câu lệnh hệ thống, nơi định nghĩa vai trò chuyên gia cho tác nhân, đồng thời quy định các tiêu chuẩn về tông giọng sư phạm, cách ngắt nghỉ trong lời thoại và các quy tắc chuyển cảnh. Một cấu trúc System Prompt chặt chẽ được sử dụng như một bộ lọc tiền xử lý, đảm bảo nội dung sinh ra luôn bám sát mục tiêu giáo dục và giảm thiểu tối đa các phản hồi ngoài phạm vi nghiệp vụ.

Một thách thức kỹ thuật khi sử dụng LLM trong các hệ thống tự động hóa là không xác định được kết quả đầu ra. Để đảm bảo dữ liệu kịch bản từ LLM có thể được chuyển tiếp sang các phân hệ xử lý âm thanh và video, hệ thống áp dụng các kỹ thuật định dạng dữ liệu có cấu trúc. Việc kết hợp giữa câu lệnh định hướng đầu ra và các thư viện kiểm chứng như Pydantic cho phép hệ thống ép kiểu các dữ liệu đầu ra của AI về định dạng JSON chuẩn hóa. Quá trình này đảm bảo rằng mọi thành phần trong kịch bản, từ nội dung lời thoại đến các tham số về thời gian và chỉ thị thao tác, đều tuân thủ một lược đồ dữ liệu nhất định, giúp loại bỏ các lỗi cú pháp có thể gây gián đoạn dây chuyền xử lý đa phương tiện phía sau.

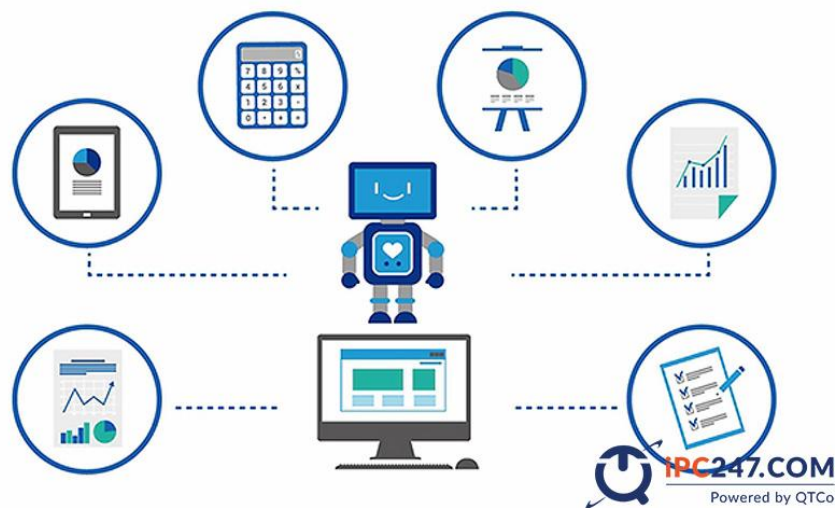
Bên cạnh việc kiểm soát định dạng, kỹ thuật thiết kế câu lệnh còn tập trung vào việc nâng cao tính chính xác và logic của kịch bản thông qua phương pháp Chuỗi suy nghĩ. Bằng cách yêu cầu mô hình thực hiện các bước phân tích trung gian trước khi đưa ra kết quả cuối cùng, hệ thống có thể theo dõi được lộ trình suy luận của AI, từ đó nhận diện và ngăn chặn sớm các hiện tượng ảo giác thông tin. Sự kết hợp giữa khả năng xử lý ngôn ngữ mạnh mẽ của LLM và các kỹ thuật điều phối câu lệnh có hệ thống tạo nên một thành phần xử lý có khả năng sáng tạo nội dung và đảm bảo được tính kỷ luật của dữ liệu, phục vụ cho các bài toán đồng bộ hóa đa phương tiện.

2.3 Tự động hóa quy trình nhận thức

Tự động hóa quy trình bằng robot là công nghệ sử dụng các thực thể phần mềm để mô phỏng và thực hiện các thao tác có tính lặp lại của con người trên giao diện đồ họa [2]. Trong các hệ thống truyền thống, RPA vận hành dựa trên tập hợp các quy tắc cố định, đòi hỏi cấu trúc dữ liệu đầu vào và các thành phần giao diện phải ở trạng thái tĩnh. Tuy nhiên, đối với các nền tảng web hiện đại như Office Online hoặc các trình soạn thảo tài liệu trực tuyến, việc áp dụng RPA thuần túy gặp nhiều trở ngại do cấu

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
trúc mã nguồn HTML phức tạp, các cơ chế chống truy cập tự động và sự thay đổi linh hoạt của các thành phần giao diện theo thời gian thực.

Để giải quyết các hạn chế này, Tự động hóa quy trình nhận thức được triển khai như một sự kết hợp giữa khả năng thực thi của RPA và năng lực phân tích của Trí tuệ nhân tạo. Thay vì dựa vào tọa độ cố định hoặc các bộ chọn phần tử tĩnh vốn dễ bị phá vỡ, Cognitive RPA sử dụng các thuật toán xử lý ngôn ngữ tự nhiên và thị giác máy tính để phân tích được ngữ cảnh của giao diện. Trong hệ thống đề xuất, phân hệ này thực hiện các lệnh nhấp chuột hay cuộn trang và có khả năng hiểu được cấu trúc phân cấp của một trang trình chiếu, nhận diện các vùng chứa nội dung quan trọng và xác định thời điểm thích hợp để thực hiện lệnh trích xuất dữ liệu.



Hình 2.2 Tự động hóa bằng robot

Sự tích hợp giữa AI Agent và Cognitive RPA tạo ra một cơ chế vận hành tương hỗ, trong đó AI Agent là cơ quan điều phối trung tâm, còn các công cụ RPA được xem như là thực thể thực thi vật lý. Quá trình tự động hóa được thực hiện qua chu trình ba bước: tiếp nhận trạng thái giao diện, phân tích nhận thức và thực thi thao tác. Khả năng nhận thức cho phép hệ thống tự động điều chỉnh hành vi khi giao diện thay đổi hoặc khi gặp các thông báo ngoại lệ từ nền tảng web, từ đó duy trì tính ổn định của luồng bóc tách nội dung bài giảng mà không cần sự can thiệp thủ công.

Ứng dụng Cognitive RPA còn cho phép hệ thống xử lý các loại dữ liệu phi cấu trúc và các tương tác phức tạp trên trình duyệt một cách hiệu quả. Bằng cách kết hợp khả năng bóc tách dữ liệu thông qua giao thức API và mô phỏng thao tác người dùng trực tiếp trên trình duyệt, hệ thống đảm bảo thu thập đầy đủ các thành phần từ nội

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
dung văn bản, hình ảnh cho đến các hiệu ứng chuyển cảnh của Slide. Đây là nền tảng kỹ thuật để xây dựng một quy trình tự động hóa toàn diện, giúp chuyển đổi các tài liệu thô thành kịch bản có cấu trúc mà vẫn đảm bảo tính vẹn toàn của dữ liệu ban đầu.

2.4 Kiến trúc hệ thống phân tán và Microservices

Trong kỷ nghệ phần mềm hiện đại, việc chuyển dịch từ kiến trúc đơn khối sang kiến trúc vi dịch vụ là một giải pháp để giải quyết các bài toán về tính linh hoạt và khả năng mở rộng của hệ thống [3]. Kiến trúc Microservices phân tách một ứng dụng phức tạp thành một tập hợp các dịch vụ nhỏ, độc lập, trong đó mỗi dịch vụ đảm nhận một chức năng nghiệp vụ chuyên biệt và giao tiếp với nhau thông qua các giao thức mạng nhẹ. Đối với hệ thống tự động hóa video bài giảng, mô hình này cho phép tách biệt các tác vụ có đặc tính kỹ thuật khác nhau như: xử lý yêu cầu người dùng, bóc tách dữ liệu trình chiếu và mô phỏng thao tác trình duyệt.

Tính độc lập giữa các phân hệ giúp cô lập lỗi, đảm bảo rằng sự cố tại một worker không gây gián đoạn cho toàn bộ tiến trình, cho phép tối ưu hóa tài nguyên tính toán theo từng loại tác vụ cụ thể. Việc triển khai các dịch vụ này dựa trên nền tảng container hóa, cụ thể là Docker, tạo ra một môi trường thực thi nhất quán từ giai đoạn phát triển đến khi vận hành trên máy chủ VPS. Các container được xem như các đơn vị triển khai biệt lập, chứa đầy đủ mã nguồn và các thư viện phụ thuộc, giúp loại bỏ các xung đột về môi trường hệ điều hành và đơn giản hóa quy trình quản trị hạ tầng.

Để duy trì sự phối hợp đồng bộ giữa các thành phần trong một mạng lưới phân tán, hệ thống áp dụng mô hình hướng sự kiện kết hợp với cơ chế hàng đợi dựa trên Redis. Các dịch vụ tương tác thông qua việc đẩy và nhận thông điệp bất đồng bộ. Đặc biệt, việc ứng dụng cơ chế Redis Pub/Sub cho phép hệ thống thực hiện truyền tin theo thời gian thực giữa API Gateway và các Worker nằm trong các container riêng biệt. Cơ chế này phục vụ việc cập nhật tiến độ xử lý video lên giao diện người dùng và là nền tảng để triển khai bộ điều phối tự động, giúp hệ thống tự động điều chỉnh số lượng worker dựa trên mật độ công việc hiện có trong hàng đợi.

Bên cạnh đó, việc quản lý trạng thái của các tác vụ phân tán được đảm bảo thông qua sự kết hợp giữa bộ lưu trữ in-memory (Redis) và cơ sở dữ liệu bền vững. Điều này cho phép hệ thống duy trì tính toàn vẹn của dữ liệu ngay cả khi sự cố ngắt quãng về kết nối hoặc khởi động lại dịch vụ. Sự hội tụ của kiến trúc Microservices, kỹ

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
thuật container hóa và các giao thức giao tiếp bất đồng bộ tạo nên một hạ tầng vững chắc, đáp ứng các tiêu chuẩn về tính sẵn sàng cao và khả năng mở rộng linh hoạt trong môi trường điện toán đám mây.

2.5 Nguyên lý xử lý và đồng bộ hóa tín hiệu đa phương tiện số

Xử lý đa phương tiện số là quá trình thao tác trên các dòng dữ liệu không đồng nhất bao gồm âm thanh, hình ảnh và dữ liệu văn bản để tạo ra một thực thể thông tin thống nhất. Trong lý thuyết xử lý tín hiệu, thách thức của các hệ thống tự động hóa là bài toán đồng bộ hóa giữa các luồng dữ liệu có đặc tính thời gian khác nhau. Nguyên lý này dựa trên việc thiết lập một dòng thời gian chủ, thường là luồng âm thanh, để làm mốc tham chiếu cho các sự kiện hình ảnh. Do tín hiệu giọng nói được tổng hợp từ các mô hình xác suất thường có thời lượng không cố định và phụ thuộc vào các biến số như ngữ điệu hay tốc độ phát âm, việc đồng bộ hóa đòi hỏi một cơ chế xác định thời điểm thực thi dựa trên các sự kiện thực tế thay vì các tham số tĩnh. Lý thuyết về đồng bộ hóa sự kiện thời gian quy định rằng mỗi hành động trực quan phải được gắn kết với một điểm neo trong luồng âm thanh để triệt tiêu hiện tượng sai lệch tích lũy, đảm bảo tính nhất quán của nội dung truyền tải trong suốt chiều dài của dữ liệu.

Song song với quá trình đồng bộ, nguyên lý về quản lý luồng dữ liệu đa phương tiện tập trung vào việc tối ưu hóa hiệu suất truyền dẫn và lưu trữ. Có hai phương pháp chính trong lý thuyết kết xuất video là tái mã hóa và ghép nối luồng. Về mặt lý thuyết, quá trình tái mã hóa toàn bộ khung hình thường tiêu tốn tài nguyên tính toán cực lớn và có thể gây suy giảm chất lượng dữ liệu gốc. Ngược lại, kỹ thuật ghép nối luồng dữ liệu dựa trên nguyên tắc trộn các dòng bit đã được mã hóa sẵn vào trong một tệp tin chứa. Phương pháp này giúp duy trì tính vẹn toàn của dữ liệu và giảm thiểu chi phí xử lý tại bộ vi xử lý trung tâm bằng cách chỉ can thiệp vào cấu trúc đóng gói thay vì thay đổi bản chất của các điểm ảnh. Việc ứng dụng lý thuyết ghép nối luồng là cơ sở khoa học quan trọng để xây dựng các hệ thống kết xuất hiệu năng cao, cho phép xử lý song song nhiều tác vụ trong môi trường có nguồn lực tài nguyên hạn chế.

Ngoài ra, việc xử lý âm thanh kỹ thuật số trong các hệ thống tự động cũng đòi hỏi sự hiểu biết về các đặc tính vật lý của tín hiệu như tần số lấy mẫu và thời lượng biên độ. Lý thuyết về nội suy thời gian trong đa phương tiện cho phép tính toán các khoảng trễ cần thiết để bù đắp sự chênh lệch giữa tốc độ sinh dữ liệu của AI và tốc độ

hiển thị thực tế của giao diện. Sự hội tụ của các nguyên lý về đồng bộ hóa sự kiện, quản lý luồng dữ liệu và nội suy thời gian tạo thành một khung lý thuyết hoàn chỉnh, cho phép thiết kế các kiến trúc phần mềm có khả năng tự động hóa việc sản xuất nội dung video với độ chính xác và tính ổn định cao.

Các định lý về xử lý tín hiệu đa phương tiện này không chỉ mang tính học thuật mà chính là nền tảng cốt lõi để thiết kế kiến trúc phân hệ kết xuất của WebReel. Trong các chương tiếp theo, khái niệm Dòng thời gian chủ sẽ được ứng dụng trực tiếp bằng việc lấy các tệp âm thanh lời bình làm trục thời gian chính. Ngay cả khi luồng ghi hình màn hình bị nghẽn hoặc rớt khung hình do giới hạn I/O của môi trường ảo hóa Docker, hệ thống vẫn duy trì đồng bộ các sự kiện thị giác theo thời lượng vật lý của âm thanh. Đồng thời, khái niệm Nội suy thời gian chính là cơ sở lý thuyết vững chắc để đề xuất thuật toán chèn khoảng đệm 1000ms. Thuật toán này được dùng như một bộ giảm xóc, hấp thụ toàn bộ độ trễ của các lệnh giao tiếp trên hệ thống phân tán, đảm bảo video thành phẩm giữ được nhịp điệu tự nhiên.

2.6 Các công nghệ và công cụ hỗ trợ

2.6.1 Công nghệ Container hóa Docker

Công nghệ container hóa bắt đầu thu hút sự chú ý mạnh mẽ vào năm 2013 khi công ty dotCloud (sau này đổi tên thành Docker, Inc.) giới thiệu Docker ra cộng đồng mã nguồn mở dưới sự dẫn dắt của Solomon Hykes. Trước thời điểm này, việc cô lập môi trường ứng dụng chủ yếu dựa vào Máy ảo, vốn cồng kềnh và tiêu tốn nhiều tài nguyên do phải chạy toàn bộ một hệ điều hành khách. Sự ra đời của Docker góp phần đơn giản hóa quá trình triển khai và vận hành ứng dụng trong môi trường phân tán và kỹ thuật phần mềm bằng cách đóng gói mã nguồn, thư viện và các tệp cấu hình vào những đơn vị phần mềm nhỏ gọn gọi là “container”, giúp ứng dụng có thể chạy nhất quán trên mọi môi trường hạ tầng từ máy cá nhân đến máy chủ đám mây.

Về mặt lý thuyết, Docker không mô phỏng phần cứng như máy ảo mà hoạt động dựa trên việc chia sẻ nhân của hệ điều hành máy chủ. Nó sử dụng các tính năng cốt lõi của Linux kernel như cgroups để giới hạn, hạch toán tài nguyên và namespaces để cô lập không gian làm việc (tiến trình, mạng, tệp hệ thống). Trong kiến trúc của hệ thống WebReel, bộ công cụ Docker Compose được ứng dụng để định nghĩa và vận hành hạ tầng vi dịch vụ phân tán [4]. Thông qua một tệp cấu hình, hệ thống có thể điều

phối hàng loạt các container hoạt động song song như cơ sở dữ liệu, message broker và các AI Worker, đảm bảo khả năng cô lập tài nguyên cho các tác vụ xử lý đồ họa ngầm mà không gây xung đột ở cấp độ hệ điều hành.

2.6.2 Framework phát triển API bất đồng bộ FastAPI

FastAPI là một web framework hiện đại dành cho ngôn ngữ Python [5], được phát triển bởi Sebastián Ramírez và ra mắt lần đầu vào năm 2018. Trong bối cảnh các framework truyền thống như Django hay Flask gặp giới hạn về hiệu suất xử lý đồng thời, FastAPI được sử dụng rộng rãi nhờ hỗ trợ lập trình bất đồng bộ và khả năng sinh tài liệu API tự động mới của Python. Chỉ trong một thời gian ngắn, framework này đã vươn lên trở thành một trong những công cụ phát triển Backend được ưa chuộng nhờ tốc độ xử lý nhanh, đồng thời tự động hóa việc tạo tài liệu API theo chuẩn OpenAPI.

Cơ sở lý thuyết của FastAPI được xây dựng trên sự kết hợp giữa hai thư viện mạnh mẽ: Starlette (chịu trách nhiệm định tuyến web và cơ chế bất đồng bộ) và Pydantic (chịu trách nhiệm kiểm chứng và xác định kiểu dữ liệu). Đặc tính bất đồng bộ cho phép máy chủ không bị phong tỏa khi chờ đợi các tác vụ ngoại vi, chẳng hạn như truy vấn cơ sở dữ liệu hoặc chờ phản hồi từ API của các Mô hình ngôn ngữ lớn. Điều này đặc biệt quan trọng đối với kiến trúc của WebReel, nơi RESTful API phải liên tục tiếp nhận, xác thực và quản lý trạng thái của hàng loạt công việc tiêu tốn thời gian mà vẫn đảm bảo độ trễ tương đối thấp cho phía giao diện người dùng.

2.6.3 Hệ quản trị cơ sở dữ liệu MongoDB và Redis

Sự chuyển dịch từ cơ sở dữ liệu quan hệ sang cơ sở dữ liệu NoSQL diễn ra mạnh mẽ vào cuối thập niên 2000. MongoDB, ra mắt năm 2009 bởi 10gen, đi tiên phong trong mô hình lưu trữ hướng tài liệu, loại bỏ cấu trúc bảng và hàng cứng nhắc [6]. Cùng năm đó, Salvatore Sanfilippo phát triển Redis ban đầu chỉ để theo dõi lượng truy cập web theo thời gian thực, nhưng sau đó đã phát triển thành một hệ thống lưu trữ cấu trúc dữ liệu trên bộ nhớ mã nguồn mở có hiệu năng cao.

MongoDB lưu trữ dữ liệu dưới dạng các cấu trúc BSON, cung cấp sự linh hoạt. Tính năng này cho phép hệ thống dễ dàng lưu trữ các siêu dữ liệu phức tạp, cấu trúc cấu hình RBAC hay các trạng thái Job động mà không cần định nghĩa lại lược đồ liên tục. Trong khi đó, Redis hoạt động trên bộ nhớ RAM, khiến nó trở thành công cụ lý tưởng cho vai trò Message Broker. Bằng cách sử dụng cơ chế Pub/Sub và cấu trúc dữ

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
liệu hàng đợi, Redis điều phối luồng thông điệp giữa Backend API và các Worker phân tán một cách liên tục và theo thời gian thực, đảm bảo tính nhất quán của dữ liệu truyền tải mà không gặp hiện tượng thất cổ chai ở ổ cứng [7].

2.6.4 Mô hình trí tuệ nhân tạo đa phương thức Google Gemini

Google Gemini là mô hình ngôn ngữ đa phương thức do Google DeepMind nghiên cứu và công bố vào cuối năm 2023. Đây là kết quả của sự hội tụ kỹ thuật học sâu sau nhiều năm Google phát triển các mô hình ngôn ngữ lớn như LaMDA và PaLM [8]. Điểm nổi bật của Gemini là nó được xây dựng từ đầu với kiến trúc đa phương thức. Thay vì ghép nối nhiều mô hình riêng lẻ (một cho văn bản, một cho hình ảnh), Gemini được huấn luyện song song trên đa dạng các loại dữ liệu, cho phép nó “hiểu” và lập luận chéo giữa văn bản, mã nguồn, hình ảnh và video một cách liền mạch.

Cơ sở lý thuyết cốt lõi của Gemini dựa trên kiến trúc Transformer nâng cao, lấy cơ chế chú ý làm nền tảng. Khác với các mạng nơ-ron tuần tự truyền thống bị giới hạn bởi bộ nhớ ngắn hạn, cơ chế Attention cho phép mô hình đánh trọng số tầm quan trọng của từng vùng thông tin trên toàn bộ dữ liệu đầu vào. Trong bối cảnh tự động hóa, khi hệ thống đẩy một cây cấu trúc DOM tương đối lớn chứa hàng ngàn dòng mã HTML rác, thuật toán Attention giúp Tác nhân AI bỏ qua các đoạn mã định dạng dư thừa để tập trung phân tích chính xác ngữ nghĩa của một nút bấm hay một trường nhập liệu, từ đó ra quyết định điều hướng chính xác mà không bị nhiễu thông tin.

2.6.5 Tự động hóa trình duyệt Playwright và Browser-Use

Trong lĩnh vực tự động hóa kiểm thử web, Selenium từng được sử dụng rộng rãi trong hơn một thập kỷ. Tuy nhiên, những hạn chế về tốc độ và sự thiếu đồng bộ đã thúc đẩy Microsoft phát triển và ra mắt Playwright vào năm 2020. Playwright mang đến một sự mới mẻ khi hỗ trợ đa trình duyệt với hiệu suất vượt trội. Gần đây, sự bùng nổ của AI Agent đã khai sinh ra thư viện Browser-Use, một lớp trừu tượng thông minh được xây dựng trên nền tảng của Playwright, mở rộng khả năng tự động hóa bằng cách tích hợp mô hình ngôn ngữ lớn vào quá trình ra quyết định.

Về cơ chế hoạt động, Playwright vượt qua các hạn chế của Web Driver truyền thống bằng cách giao tiếp trực tiếp với tiến trình trình duyệt thông qua giao thức nội bộ như Chrome DevTools Protocol trên kết nối WebSockets. Điều này cho phép bắt các yêu cầu mạng và quản lý vòng đời của DOM theo thời gian thực. Khi kết hợp với

thư viện Browser-use, kiến trúc này cho phép LLM tương tác với các phần tử web và thực thi các chiến lược phức tạp như vượt qua iframe, quản lý các thẻ và tự động nhận thức ngữ cảnh để thao tác trên các ứng dụng văn phòng trực tuyến ngay bên trong môi trường Linux không giao diện đồ họa [9].

2.6.6 Khung xử lý đa phương tiện FFmpeg và Edge-TTS

FFmpeg là một trong những dự án mã nguồn mở, được khởi xướng bởi Fabrice Bellard vào năm 2000. Ban đầu chỉ là một công cụ dòng lệnh xử lý video và audio trên Linux, FFmpeg ngày nay là lõi đa phương tiện của hầu hết các nền tảng công nghệ toàn cầu (bao gồm cả YouTube và VLC). Song song đó, công nghệ Tổng hợp giọng nói cũng có lịch sử phát triển dài, nhưng chỉ thực sự đạt độ chân thực như con người khi các hệ thống mạng nơ-ron sâu ra đời, tiêu biểu là dịch vụ Edge-TTS sử dụng các mô hình ngôn ngữ giọng nói đám mây của Microsoft.

Cơ sở lý thuyết của FFmpeg xoay quanh quy trình xử lý tín hiệu số với bốn giai đoạn cốt lõi: Demuxing, Decoding, Filtering/Processing và Encoding/Muxing [10]. Trong dự án, FFmpeg thực hiện nghiệp vụ hậu kỳ tự động bằng cách kết xuất, trộn trực tiếp luồng hình ảnh quay được từ khung hiển thị ảo với luồng âm thanh. Âm thanh này được cấu thành từ dịch vụ Edge-TTS, nơi văn bản được chuyển đổi thành phổ âm thông qua các thuật toán học máy dự đoán ngữ điệu, và được biên dịch thành sóng âm thanh vật lý định dạng MP3. Sự kết hợp này cho phép tạo ra các video đầu ra có chất lượng, đồng bộ về mặt thời gian giữa hành động và lời thoại.

2.6.7 Tự động hóa hệ điều hành với Pywinauto

Bên cạnh tự động hóa Web, việc điều khiển các ứng dụng cục bộ trên hệ điều hành Windows luôn là một bài toán kỹ thuật phức tạp. Pywinauto là thư viện Python được phát triển từ năm 2006 để giải quyết thách thức này. Nó được xây dựng để cung cấp một giao diện lập trình cấp cao, bao bọc lại các thư viện lõi của Microsoft như Windows API và sau này là Microsoft UI Automation - những nền tảng được thiết kế ban đầu để hỗ trợ người khuyết tật sử dụng máy tính.

Nguyên lý hoạt động của tự động hóa OS khác biệt so với Web. Thay vì thao tác với cây DOM, Pywinauto và các thư viện như UIAutomation giao tiếp trực tiếp với nhân hệ điều hành để truy xuất “Cây phần tử UI” của các ứng dụng đang chạy. Mỗi đối tượng đồ họa (nút bấm, hộp thoại) được hệ điều hành định danh bằng một thẻ quản

lý hoặc thuộc tính UIA. Bằng cách gửi các tín hiệu ngắt và sự kiện phản ứng giả lập, các OS Worker có thể chiếm quyền điều khiển chuột, bàn phím để tương tác với các ứng dụng Native mà không cần can thiệp vào mã nguồn của phần mềm đó. Kiến trúc mạng thông qua đường hầm bảo mật cho phép điều khiển các tác vụ cục bộ này từ xa thông qua hạ tầng container hóa.

2.6.8 Hạ tầng lưu trữ đối tượng đám mây Cloudflare R2

Khái niệm Lưu trữ đối tượng bắt đầu định hình kiến trúc điện toán đám mây hiện đại từ năm 2006 khi Amazon Web Services ra mắt dịch vụ S3. Khác với hệ thống tệp tin truyền thống lưu trữ theo cấp bậc thư mục hoặc lưu trữ khối chia nhỏ dữ liệu trên đĩa cứng, Object Storage gộp dữ liệu gốc, siêu dữ liệu phân tích phong phú và một định danh duy nhất vào một “đối tượng” được đặt trong một không gian phẳng. Đến năm 2021, Cloudflare đã tạo ra một bước ngoặt lớn trên thị trường hạ tầng đám mây khi ra mắt dịch vụ Cloudflare R2. Sự mạnh mẽ của R2 nằm ở kiến trúc phân tán toàn cầu và ở mô hình kinh tế giảm thiểu phí băng thông truyền tải dữ liệu ra bên ngoài, khắc phục rào cản chi phí đối với các ứng dụng phân phối nội dung đa phương tiện.

Về mặt nguyên lý hoạt động, Cloudflare R2 tương thích với tập lệnh API chuẩn của Amazon S3, cho phép các ứng dụng giao tiếp thông qua giao thức HTTP/HTTPS an toàn. Trong kiến trúc của hệ thống, R2 là kho lưu trữ cho các video bài giảng đầu ra. Quá trình giao tiếp giữa Backend FastAPI và R2 được thực hiện thông qua các trình điều khiển I/O bất đồng bộ, đảm bảo tiến trình truyền tải các tệp video có dung lượng lớn không gây tắc nghẽn luồng sự kiện của máy chủ. Đồng thời, hệ thống tận dụng sức mạnh của Mạng phân phối nội dung từ Cloudflare kết hợp với các chỉ thị bộ nhớ đệm để phân phối video đến người dùng cuối.

CHƯƠNG 3 HIỆN THỰC HÓA NGHIÊN CỨU

3.1 Phân tích yêu cầu hệ thống

Phân tích yêu cầu là một quá trình quan trọng trong việc định hình kiến trúc và xác định các ràng buộc kỹ thuật của hệ thống WebReel AI Agent. Dựa trên mục tiêu tự động hóa quy trình sản xuất nội dung bài giảng đa phương tiện, các yêu cầu của hệ thống được phân rã thành ba nhóm chính: yêu cầu chức năng, yêu cầu phi chức năng và yêu cầu bảo mật.

3.1.1 Yêu cầu chức năng

Yêu cầu chức năng đặc tả các hành vi và tác vụ cụ thể mà hệ thống phải thực hiện để đáp ứng nhu cầu của người dùng. Các chức năng này được chia thành nhóm quản trị lõi và nhóm thực thi tự động hóa:

Nhóm chức năng Quản trị lõi và Tương tác người dùng:

Quản lý tài khoản và định danh: Hệ thống phải cung cấp các chức năng đăng ký, đăng nhập và quản lý người dùng.

Quản lý hàng đợi công việc: Cho phép người dùng khởi tạo các yêu cầu sản xuất video, theo dõi trạng thái tiến độ theo thời gian thực và xem lại lịch sử các tác vụ đã thực hiện.

Điểm dừng kiểm duyệt: Hệ thống phải cung cấp cơ chế tạm dừng luồng tự động hóa để hiển thị kịch bản thuyết minh do AI sinh ra. Người dùng có quyền can thiệp, chỉnh sửa văn bản, thay đổi giọng đọc Text-to-Speech trước khi xác nhận cho hệ thống tiến hành quay và kết xuất video.

Lưu trữ và phân phối tài sản số: Hệ thống tự động tải các video thành phẩm lên hạ tầng lưu trữ đám mây và trả về đường dẫn để người dùng tải xuống phục vụ mục đích giảng dạy.

Nhóm chức năng Thực thi Tự động hóa:

Phân hệ Web Worker: Có nhiệm vụ tự động hóa các thao tác trên nền tảng trình duyệt web. Worker này phải thực hiện được chuỗi hành động bao gồm: quét cây DOM của trang web mục tiêu, gửi dữ liệu cho AI sinh kịch bản, tổng hợp giọng nói, thêm độ trễ để đồng bộ hóa, và cuối cùng là quay lại màn toàn bộ quá trình tương tác.

Phân hệ Presentation Worker: Tập trung giải quyết bài toán tự động hóa tài liệu thuyết trình (Google Slides/Microsoft PowerPoint). Hệ thống phải có khả năng tải tài liệu đầu vào lên nền tảng đám mây, thao túng cấu trúc URL để kích hoạt chế độ trình chiếu tối ưu. Nhiệm vụ cốt lõi của phân hệ này là thực hiện lệnh chuyển trang đồng bộ với thời lượng của tệp âm thanh thuyết minh tương ứng.

Phân hệ OS Worker: Đảm nhận việc điều khiển các phần mềm ứng dụng cục bộ trên hệ điều hành Windows. Worker này phải có khả năng sử dụng các hàm API của hệ điều hành để định vị các phần tử giao diện đồ họa, mô phỏng thao tác chuột/bàn phím vật lý, quay màn hình hệ thống và truyền tải tệp tin thành phẩm về máy chủ trung tâm.

3.1.2 Yêu cầu phi chức năng

Yêu cầu phi chức năng xác định các tiêu chí chất lượng nhằm đảm bảo hệ thống vận hành ổn định, liên tục và duy trì trải nghiệm sử dụng nhất quán trong môi trường thực tế:

Hiệu năng xử lý: Các API giao tiếp phải có thời gian phản hồi trung bình được duy trì dưới 1 giây đối với các truy vấn thông thường. Đối với các tác vụ xử lý tệp đa phương tiện dung lượng lớn, hệ thống bắt buộc phải áp dụng kiến trúc I/O bất đồng bộ để tránh hiện tượng thất cổ chai và tắc nghẽn luồng xử lý chính.

Khả năng mở rộng: Hạ tầng phải được thiết kế theo hướng dịch vụ phân tán. Trình quản lý hệ thống cần có khả năng tự động tăng hoặc giảm số lượng các container thực thi dựa trên số lượng công việc đang tồn đọng trong hàng đợi Redis, tối ưu hóa việc sử dụng tài nguyên CPU và RAM.

Tính sẵn sàng và Chịu lỗi: Hệ thống cần duy trì tính khả dụng cao. Khi một Worker gặp sự cố ngắt kết nối hoặc tràn bộ nhớ trong quá trình quay video, hệ thống phải tự động dọn dẹp các tiến trình lỗi và đưa Job đó vào cơ chế thử lại mà không làm gián đoạn toàn bộ đường ống.

Tính tương thích sản phẩm đầu ra: Tệp video thành phẩm phải được kết xuất dưới định dạng tiêu chuẩn MP4, sử dụng bộ giải mã H.264 với tốc độ khung hình duy trì ổn định, đảm bảo phát ổn định trên mọi thiết bị và nền tảng giáo dục trực tuyến.

3.1.3 Yêu cầu bảo mật và phân quyền

Do đặc thù xử lý dữ liệu tài liệu cá nhân và vận hành các tác vụ điều khiển máy tính từ xa, hệ thống đặt ra các tiêu chuẩn bảo mật nghiêm ngặt:

Kiểm soát truy cập dựa trên vai trò: Phân định ranh giới rõ ràng giữa người dùng tiêu chuẩn và quản trị viên hệ thống. Các API yêu cầu xác thực bằng JSON Web Token và kiểm tra quyền hạn trước khi thực thi lệnh.

Cô lập dữ liệu: Môi trường thực thi của mỗi tác vụ phải được đóng gói độc lập. Các tệp tin cấu hình, tệp tạm thời và hình ảnh giao diện của một phiên làm việc không được phép truy cập chéo bởi các tiến trình khác.

Bảo mật kênh truyền tải: Luồng dữ liệu giao tiếp giữa OS Worker (nằm ngoài hạ tầng đám mây) và máy chủ Redis bắt buộc phải được mã hóa thông qua đường hầm bảo mật để ngăn chặn rủi ro nghe lén mạng.

Quản trị vòng đời tài sản: Để đáp ứng giới hạn vật lý của máy chủ lưu trữ cục bộ và quy định về bảo vệ quyền riêng tư, hệ thống cần tự động thanh lọc các tệp tin tạm và tài liệu gốc. Song song đó, hạ tầng Object Storage cần được cấu hình quy tắc vòng đời để tự động xóa video thành phẩm sau một chu kỳ lưu trữ nhất định.

3.1.4 Mô hình hóa ca sử dụng

Để làm rõ sự tương tác giữa các thực thể và hệ thống WebReel, cũng như xác định rõ ranh giới phần mềm, quá trình mô hình hóa ca sử dụng được thiết lập. Mô hình này định nghĩa các tác nhân có quyền can thiệp vào hệ thống và luồng sự kiện cốt lõi của từng nghiệp vụ.

Xác định các tác nhân

Hệ thống WebReel tương tác với hai nhóm tác nhân chính, được phân tách rõ ràng bên trong và bên ngoài ranh giới hệ thống:

Tác nhân con người:

- Người dùng: Là thực thể thực hiện dịch vụ, có quyền khởi tạo các yêu cầu sản xuất video, tải lên tài liệu, hiệu đính kịch bản và tải về sản phẩm đầu ra.

– Quản trị viên: Là thực thể vận hành hệ thống. Tuân thủ nguyên tắc Phân tách trách nhiệm, Admin không trực tiếp tạo video mà đảm nhận việc quản lý tài khoản người dùng, phân bổ hạn mức và quản trị trạng thái phiên làm việc gốc.

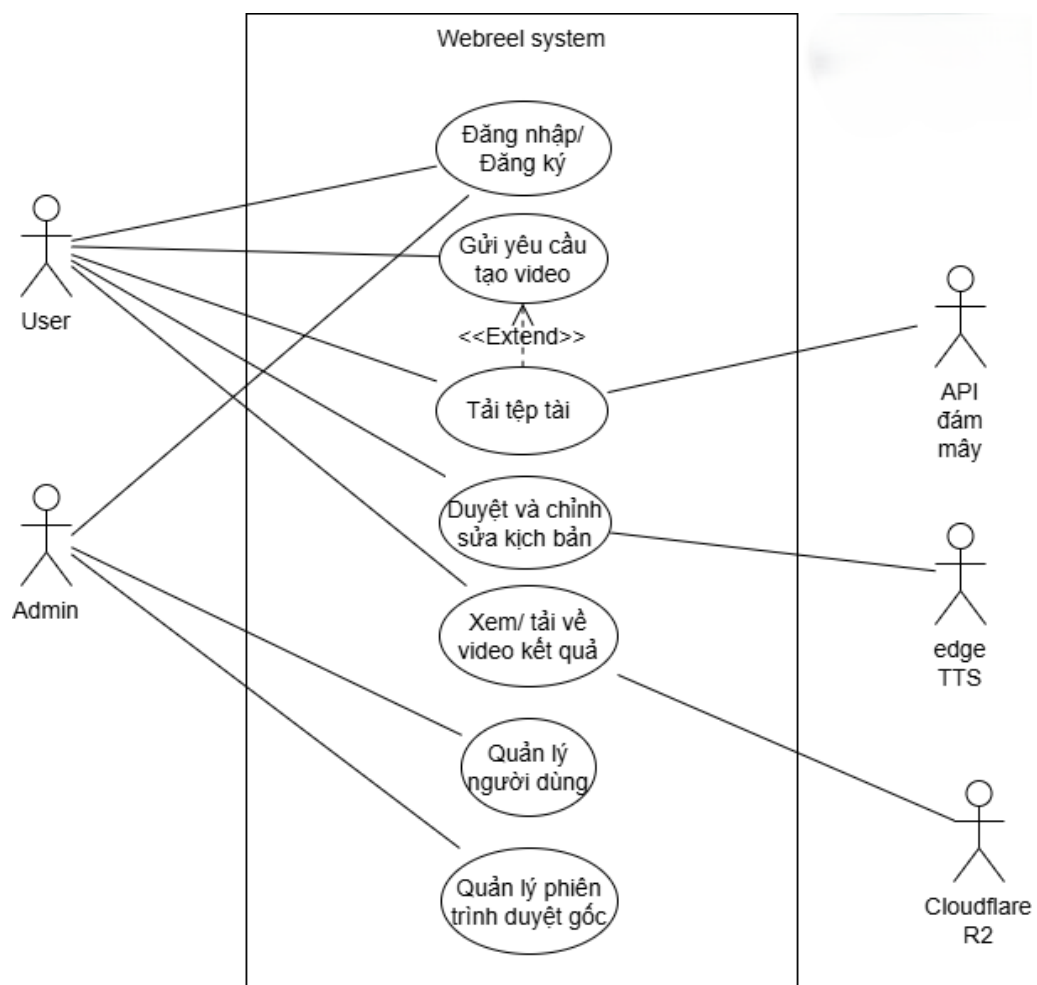
Tác nhân hệ thống ngoại vi:

– API Đám mây: Giao diện lập trình của các đối tác (Google Drive, Microsoft Graph) tiếp nhận tài liệu và trả về liên kết môi trường trình chiếu.

– Dịch vụ Edge TTS: Máy chủ tổng hợp giọng nói mạng nơ-ron của Microsoft, tiếp nhận văn bản và trả về luồng dữ liệu âm thanh.

– Cloudflare R2: Hạ tầng lưu trữ đối tượng và mạng phân phối nội dung quản lý các tệp video thành phẩm.

Sơ đồ Use Case tổng quát



Hình 3.1 Sơ đồ usecase tổng quát

Đặc tả chi tiết các ca sử dụng

Dựa trên sơ đồ tổng quát, các luồng nghiệp vụ của hệ thống được đặc tả chi tiết làm cơ sở cho quá trình lập trình và kiểm thử.

Bảng 3.1 Đặc tả Ca sử dụng Đăng nhập/ Đăng ký

Thuộc tính	Đặc tả
Tác Nhân	User, Admin
Tóm Tắt Nghiệp Vụ	Xác thực danh tính và cấp quyền truy cập hệ thống dựa trên vai trò.
Luồng Sự Kiện Chính	1. Tác nhân nhập thông tin định danh (Email/Mật khẩu). 2. Hệ thống mã hóa thông tin và đối chiếu với cơ sở dữ liệu MongoDB. 3. Nếu hợp lệ, hệ thống khởi tạo và trả về JSON Web Token chứa thông tin vai trò tương ứng. 4. Tác nhân được điều hướng đến Bảng điều khiển riêng biệt tùy theo phân quyền.

Bảng 3.2 Đặc tả Ca sử dụng Gửi yêu cầu tạo video

Thuộc tính	Đặc tả
Tác Nhân	User, API Đám mây
Ca sử dụng mở rộng	<<Extend>> Tải tệp tài liệu
Tóm Tắt Nghiệp Vụ	Người dùng cấu hình thông số và khởi tạo một tác vụ sản xuất video đa phương tiện.
Luồng Sự Kiện Chính	1. User chọn phương thức tạo video (Web, OS, hoặc Trình chiếu đám mây). 2. [Luồng Mở rộng - Tải tệp tài liệu]: Nếu chọn phương thức thao tác tĩnh hoặc trình chiếu, User tải lên tệp.pptx hoặc.pdf. Hệ thống

Thuộc tính	Đặc tả
	làm sạch tên tệp và gọi API Đám mây để lưu trữ/chuyển đổi định dạng. 3. User thiết lập cấu hình: Giọng đọc AI, thời gian đệm và chế độ điểm dừng kiểm duyệt. 4. Nhấp xác nhận. API Backend ghi nhận siêu dữ liệu vào cơ sở dữ liệu và đẩy Job vào hàng đợi Redis tương ứng.

Bảng 3.3 Đặc tả ca sử dụng Duyệt và chỉnh sửa kịch bản

Thuộc tính	Đặc tả
Tác Nhân	User, Edge TTS
Tóm Tắt Nghiệp Vụ	Cung cấp điểm dừng kiểm duyệt để con người hiệu đính dữ liệu do AI sinh ra trước khi kết xuất.
Luồng Sự Kiện Chính	1. Khi Job đạt trạng thái pending_review, hệ thống thông báo cho User. 2. User truy cập giao diện, kiểm tra các câu thoại thuyết minh tương ứng với từng thao tác/slide. 3. User thực hiện chỉnh sửa thuật ngữ hoặc bố cục câu chữ, sau đó nhấn “Phê duyệt”. 4. Hệ thống cập nhật kịch bản, gọi dịch vụ Edge TTS để tổng hợp giọng nói, và đánh thức Worker tiếp tục luồng ghi hình.

Bảng 3.4 Đặc tả Ca sử dụng Xem/ Tải về video kết quả

Thuộc tính	Đặc tả
Tác Nhân	User, Cloudflare R2
Tóm Tắt Nghiệp Vụ	Cho phép người dùng truy cập và tải xuống các tài sản số đã được hệ thống kết xuất thành công.
Luồng Sự Kiện Chính	1. User truy cập danh mục dự án hoàn thành. 2. Hệ thống Backend tương tác với Cloudflare R2 để sinh ra một liên kết tải xuống đã được xác thực an toàn. 3. Trình duyệt của User sử dụng URL này để phát video trực tiếp hoặc tải tệp MP4/Tài liệu về thiết bị cục bộ.

Bảng 3.5 Đặc tả Ca sử dụng Quản lý người dùng

Thuộc tính	Đặc tả
Tác Nhân	Admin
Tóm Tắt Nghiệp Vụ	Quản trị viên kiểm soát vòng đời tài khoản và phân bổ tài nguyên hệ thống.
Luồng Sự Kiện Chính	- Admin truy cập Bảng điều khiển quản trị. - Admin thực hiện các thao tác: Cấp phát hạn mức tạo video cho từng User, tạm ngưng các tài khoản vi phạm, hoặc xuất báo cáo tài nguyên. - Hệ thống cập nhật các ràng buộc vào cơ sở dữ liệu, áp dụng cho các phiên API tiếp theo của User.

Bảng 3.6 Đặc tả Ca sử dụng Quản lý phiên trình duyệt gốc

Thuộc tính	Đặc tả
Tác Nhân	Admin
Tóm Tắt Nghiệp Vụ	Quản lý và duy trì trạng thái đăng nhập để hệ thống AI Worker vượt qua các cơ chế kiểm tra bảo mật.
Luồng Sự Kiện Chính	- Admin truy cập cổng giao tiếp noVNC được nhúng trên Bảng điều khiển. - Admin điều khiển trình duyệt vật lý của máy chủ để đăng nhập thủ công vào các nền tảng (Microsoft, Google) và giải quyết các thử thách Captcha/OTP. - Đóng giao diện noVNC. Hệ thống tự động lưu trữ trạng thái phiên làm việc thành một bản chụp tĩnh để các Worker sao chép và tái sử dụng.

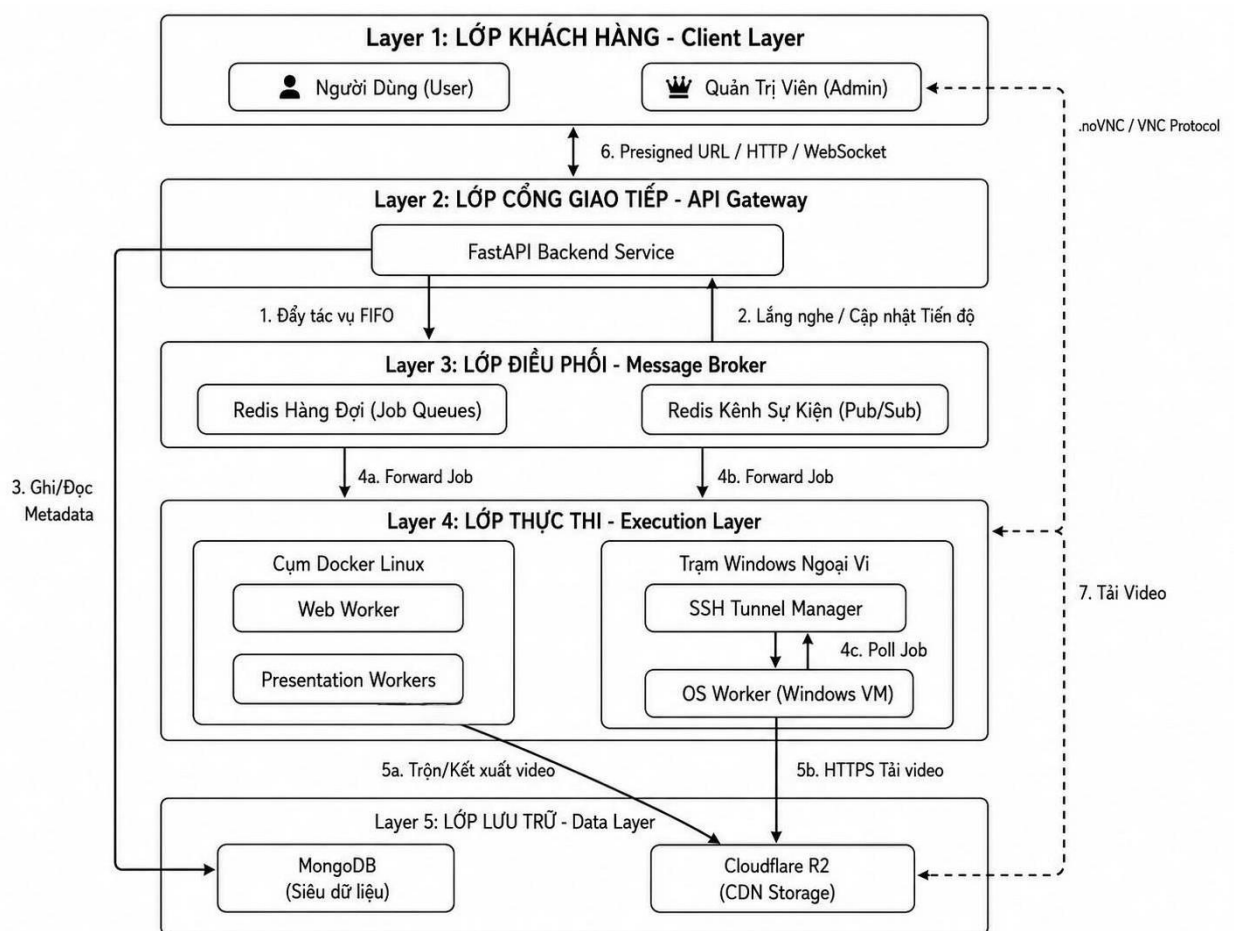
3.2 Thiết kế kiến trúc tổng thể

Để đáp ứng các yêu cầu khắt khe về tính xử lý bất đồng bộ, khả năng cô lập dữ liệu nhiều người dùng và đảm bảo an toàn thông tin trên môi trường mạng diện rộng, hệ thống WebReel được thiết kế từ đầu theo mô hình kiến trúc vi dịch vụ phân tán, kết hợp chặt chẽ với cơ chế giao tiếp hướng sự kiện. Sự kết hợp này giúp hạn chế các giới

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng hạn của các mô hình nguyên khối truyền thống, khắc phục nút thắt cổ chai khi xử lý các tác vụ render video vốn tiêu tốn rất nhiều tài nguyên phần cứng và thời gian.

3.2.1 Phân rã kiến trúc vi dịch vụ và các lớp thành phần

Kiến trúc của hệ thống WebReel được bóc tách thành các phân lớp hoạt động độc lập, giao tiếp với nhau thông qua các tiêu chuẩn mạng được định nghĩa sẵn. Thiết kế này tuân thủ nguyên tắc “tách rời”, cho phép một thành phần có thể được nâng cấp, mở rộng hoặc khởi động lại mà không làm gián đoạn toàn bộ hệ thống. Để đáp ứng khối lượng công việc lớn và tính chất phức tạp của việc tự động hóa đa nền tảng, hệ thống được quy hoạch thành 5 phân lớp chức năng rõ rệt:



Hình 3.2 Kiến trúc hệ thống và các lớp thành phần

Lớp Khách hàng: Đây là điểm tiếp xúc vật lý giữa con người và hệ thống, bao gồm hai phân hệ độc lập:

- Web App Frontend: Phục vụ người dùng cuối trong việc gửi yêu cầu dự án, theo dõi tiến trình theo thời gian thực, duyệt kịch bản và tải kết quả.

– Admin Portal: Cổng quản trị đặc quyền dành riêng cho quản trị viên, cung cấp luồng truyền phát đồ họa để can thiệp từ xa vào trình duyệt gốc dùng để quản lý trạng thái phiên làm việc và xử lý các thử thách bảo mật.

Lớp Cổng giao tiếp: Được xây dựng dựa trên framework FastAPI, lớp này là điểm chạm tiếp nhận mọi yêu cầu từ phía ngoài. Nhiệm vụ cốt lõi của API Gateway không phải là xử lý video, mà là thực hiện các nghiệp vụ đồng bộ đòi hỏi tốc độ phản hồi tính bằng mili-giây, bao gồm: xác thực định danh, kiểm tra quyền hạn truy cập, kiểm chứng tính hợp lệ của tệp tin đầu vào, ghi nhận siêu dữ liệu và định tuyến tác vụ xuống lớp điều phối.

Lớp Điều phối trung gian: Là bộ xử lý của hệ thống, lớp này sử dụng cơ sở dữ liệu trên bộ nhớ Redis để giải quyết bài toán xử lý bất đồng bộ. Lớp điều phối được chia thành hai luồng:

– Redis Queues: Lưu trữ tạm thời các Job theo cơ chế First-In-First-Out và phân mảnh thành các hàng đợi chuyên dụng để tối ưu hóa việc phân bổ tài nguyên cho từng loại Worker.

– Redis Pub/Sub: Đảm nhận việc truyền phát thông điệp hai chiều, giúp API Gateway nhận các bản tin về tiến độ xử lý và các tín hiệu chờ phê duyệt kịch bản từ lớp thực thi ngầm.

Lớp Thực thi Tự động hóa: Đây là phân lớp tiêu hao tài nguyên cốt lõi, nơi trực tiếp diễn ra các tác vụ giả lập phần cứng, điều khiển trình duyệt và kết xuất đa phương tiện. Lớp này được chia thành hai cụm nút riêng biệt:

– Cụm Docker Workers: Bao gồm Web Worker, Office Worker và Presentation Workers hoạt động trên môi trường Linux. Các Worker này được đóng gói bên trong hạ tầng Docker, tích hợp sẵn môi trường giả lập hiển thị ảo để chạy trình duyệt Headless và các công cụ biên dịch như LibreOffice, FFmpeg.

– Trạm Windows Ngoại vi: Do đặc thù yêu cầu tương tác trực tiếp với API hệ điều hành, OS Worker được triển khai độc lập trên nền tảng Windows. Để đảm bảo an toàn thông tin, luồng giao tiếp giữa trạm ngoại vi này và Redis trung tâm được mã hóa thông qua Trình quản lý Đường hầm bảo mật.

Lớp Lưu trữ Đa mô hình: Hệ thống áp dụng chiến lược đa cơ sở dữ liệu để tối ưu hóa việc đọc/ghi dựa trên đặc thù của từng loại dữ liệu:

- MongoDB: Cơ sở dữ liệu phi quan hệ lưu trữ các thông tin có cấu trúc linh hoạt như hồ sơ người dùng, phân quyền, cấu hình hệ thống và lịch sử vòng đời của các Job.
- Cloudflare R2: Hạ tầng lưu trữ đối tượng tương thích chuẩn S3, đảm nhận việc lưu trữ vật lý các tệp tin video thành phẩm. R2 kết hợp với Mạng phân phối nội dung để tối ưu hóa băng thông truyền tải tài sản số đến trình duyệt của người dùng với độ trễ tương đối thấp.

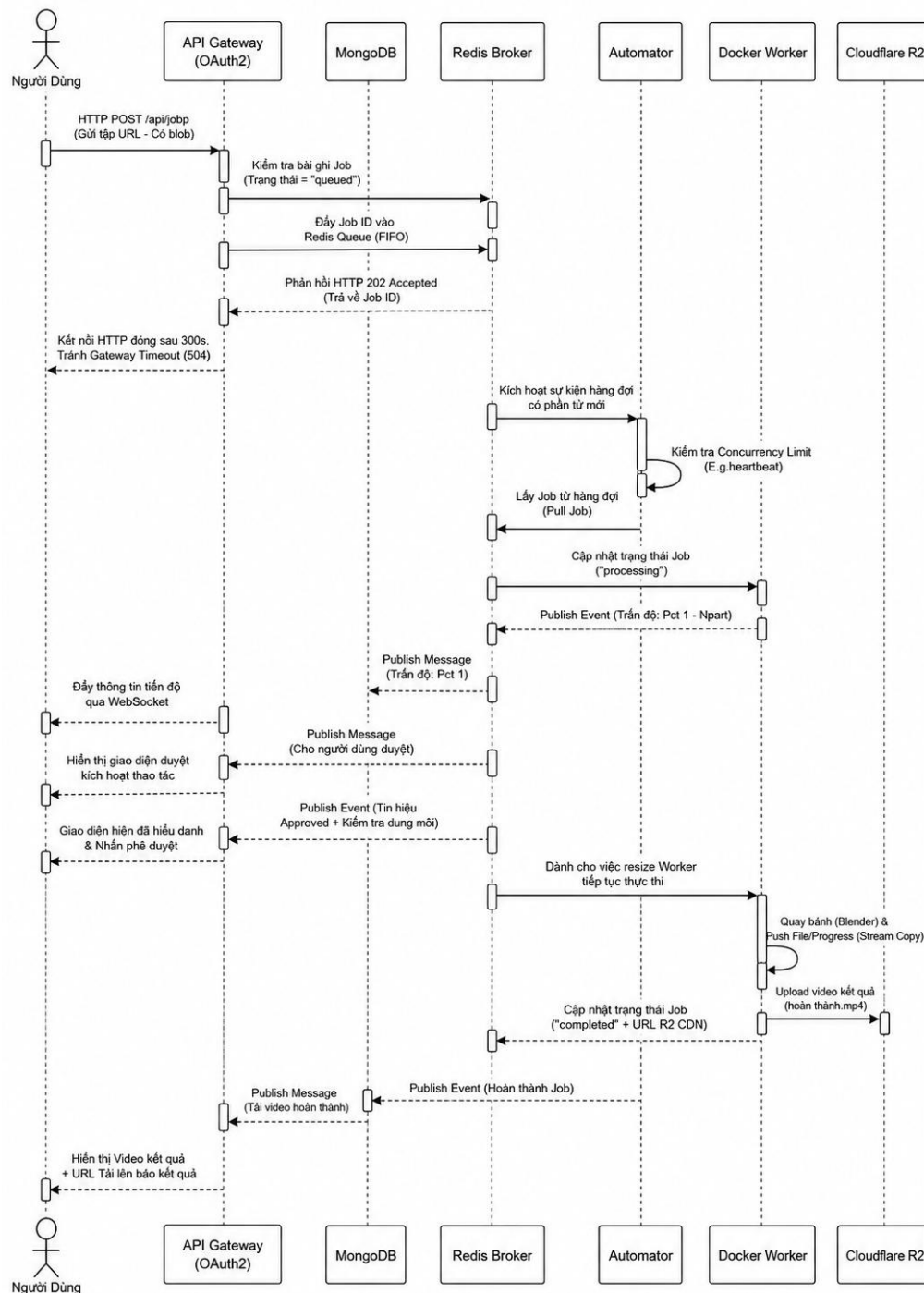
3.2.2 Cơ chế Điều phối thông điệp hướng sự kiện

Một trong những thách thức kỹ thuật lớn đối với hệ thống WebReel là thời gian thực thi tác vụ. Quá trình tự động hóa trình duyệt, phân tích ngữ cảnh, tổng hợp giọng nói AI và phối trộn đa phương tiện thông qua FFmpeg đòi hỏi thời gian xử lý thực tế kéo dài từ 3 đến 15 phút cho mỗi video. Nếu hệ thống áp dụng mô hình giao tiếp đồng bộ truyền thống, máy chủ API sẽ buộc phải duy trì kết nối mở liên tục với trình duyệt của người dùng. Điều này dẫn đến nguy cơ cạn kiệt luồng xử lý trên máy chủ và chắc chắn sẽ kích hoạt lỗi vượt quá thời gian chờ từ các cổng trung chuyển như Nginx sau 60 giây, làm đứt gãy luồng nghiệp vụ.

Để khắc phục bài toán thất cổ chai này, hệ thống áp dụng cơ chế điều phối hướng sự kiện bất đồng bộ lấy Redis làm trung tâm lưu chuyển thông điệp. Cơ chế này được thiết kế dựa trên hai nguyên lý cốt lõi:

Cơ chế bất đồng bộ: Thay vì bất luồng chính phải chờ đợi tiến trình kết xuất video hoàn tất, API Gateway áp dụng chiến lược phản hồi tức thì. Khi người dùng gửi yêu cầu khởi tạo video, API chỉ thực hiện các nghiệp vụ xác thực cơ bản, khởi tạo bản ghi trong MongoDB với trạng thái `queued` (đang xếp hàng), và đẩy mã định danh tác vụ vào hàng đợi Redis. Ngay sau đó, API trả về mã trạng thái HTTP 202 Accepted kèm theo Job ID. Toàn bộ quá trình tiếp nhận này diễn ra trong vòng chưa tới 100 mili-giây, giúp giải phóng kết nối HTTP và bảo vệ API Gateway khỏi tình trạng quá tải.

Cơ chế Giao tiếp Xuất bản - Đăng ký: Để duy trì tính liên tục của thông tin mà không cần kết nối đồng bộ, hệ thống sử dụng kênh Redis Pub/Sub làm cầu nối giao tiếp giữa các Worker ngầm và API Gateway. Xuyên suốt vòng đời thực thi, các Worker không bao giờ giao tiếp trực tiếp với giao diện người dùng. Thay vào đó, chúng xuất bản các tín hiệu sự kiện lên kênh Redis. API Gateway là một Subscriber, lắng nghe các sự kiện này và truyền tải ngược lại Frontend theo thời gian thực thông qua giao thức WebSocket hoặc Server-Sent Events.



Hình 3.3 Sơ đồ tuần tự quá trình điều phối bất đồng bộ

Đặc tả luồng sự kiện theo Sơ đồ Tuần tự: Quá trình phối hợp bất đồng bộ giữa các thành phần kiến trúc được chia thành 5 giai đoạn thực thi khép kín:

Giai đoạn 1 - Gửi yêu cầu bất đồng bộ: Người dùng gửi HTTP POST chứa tệp tin hoặc URL. API tạo Job trong MongoDB, đẩy Job ID vào hàng đợi web-queue hoặc os-queue trên Redis và ngắt kết nối an toàn với HTTP 202.

Giai đoạn 2 - Tự động khởi chạy Worker: Tiến trình Autoscaler liên tục giám sát Redis. Khi phát hiện biến động trong hàng đợi, Autoscaler tự động cấp phát và khởi tạo một Container Worker chuyên dụng để xử lý.

Giai đoạn 3 - Thực thi và Cập nhật tiến trình: Worker lấy Job khỏi hàng đợi, cập nhật trạng thái thành `processing` trên MongoDB. Trong lúc chạy Pha 1 (Trình sát), Worker liên tục phát bản tin tiến độ qua Redis Pub/Sub để API Gateway đẩy lên màn hình người dùng.

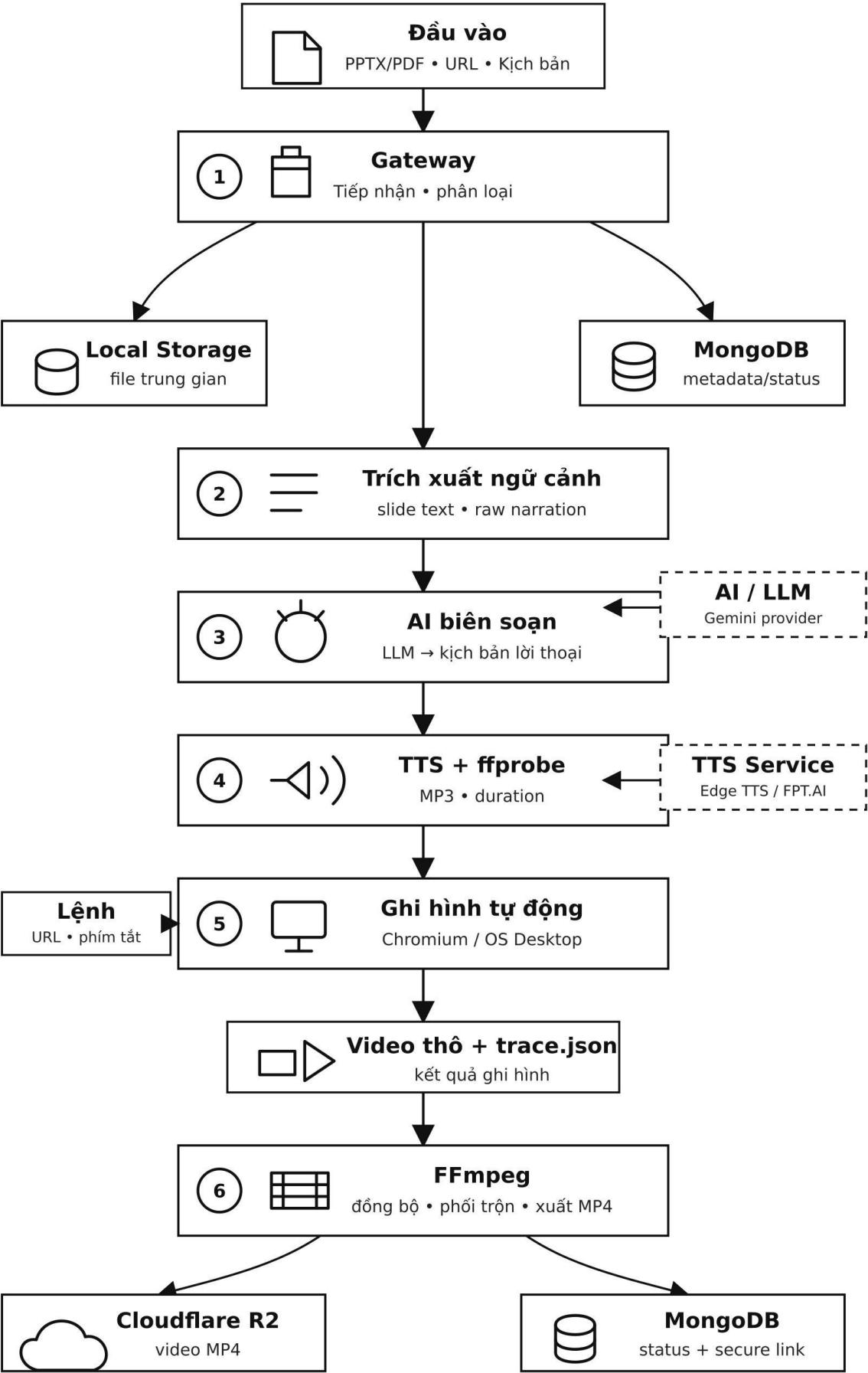
Giai đoạn 4 - Điểm dừng phê duyệt kịch bản (Pha 2.5): Khi hoàn tất việc sinh kịch bản, Worker lưu văn bản thô vào cơ sở dữ liệu, chuyển trạng thái thành `pending_review` và phát sự kiện chờ duyệt lên Redis. API Gateway nhận tín hiệu, hiển thị biểu mẫu trên Frontend. Sau khi người dùng hiệu đính và nhấn phê duyệt, API phát tín hiệu đánh thức qua Redis, cho phép Worker khôi phục tiến trình thực thi.

Giai đoạn 5 - Kết xuất và Phân phối: Worker hoàn tất quá trình ghi hình và biên tập đa phương tiện thông qua FFmpeg. Thành phẩm được tải trực tiếp lên Cloudflare R2. Cuối cùng, Worker cập nhật trạng thái `completed` kèm đường dẫn phân phối vào MongoDB, phát sự kiện hoàn thành qua Redis để thông báo đến người dùng, sau đó tiến trình Worker tự động tự hủy để thu hồi tài nguyên.

Có thể hiểu Redis Queue là hàng chờ giao việc cho Worker, còn Redis Pub/Sub là kênh phát thông báo tiến độ từ Worker về API Gateway và giao diện người dùng.

3.2.3 Sơ đồ luồng dữ liệu tổng quát

Để làm rõ hành trình biến đổi của dữ liệu từ khi hệ thống tiếp nhận cho đến khi kết xuất thành phẩm, luồng dữ liệu tổng quát của WebReel được thiết kế theo một đường ống khép kín, đi qua các phân hệ vi dịch vụ, các dịch vụ AI và các kho lưu trữ đám mây.



Hình 3.4 Sơ đồ luồng dữ liệu tổng quát

Quá trình luân chuyển dữ liệu này được chia thành 6 tiến trình xử lý cốt lõi:

Tiến trình 1: Tiếp nhận và phân loại yêu cầu: Người dùng gửi yêu cầu khởi tạo dự án chứa tệp tin (PPTX/PDF) hoặc siêu liên kết URL kèm theo kịch bản chữ. Lớp Công giao tiếp tiếp nhận và phân tách luồng dữ liệu: tệp tin vật lý được chuyển vào phân vùng lưu trữ cục bộ, trong khi siêu dữ liệu và trạng thái dự án được ghi nhận vào cơ sở dữ liệu MongoDB.

Tiến trình 2: Trích xuất ngữ cảnh: Hệ thống đọc tệp trình chiếu từ phân vùng cục bộ và sử dụng các công cụ phân tích chuyên dụng để bóc tách thông tin. Dữ liệu đầu ra của tiến trình này bao gồm luồng văn bản thuyết minh thô.

Tiến trình 3: Nhận thức và biên soạn: Luồng văn bản slide và mô tả tác vụ được truyền tải đến mô hình ngôn ngữ lớn. Tác nhân AI phân tích ngữ cảnh và trả về các câu thuyết minh tiếng Việt hoàn chỉnh, được dùng làm kịch bản chi tiết cho tiến trình ghi hình.

Tiến trình 4: Tổng hợp và đo đạc âm thanh: Kịch bản lời thoại sau khi được phê duyệt sẽ được gửi đến các Dịch vụ TTS ngoài (như Edge TTS hoặc FPT.AI) để chuyển đổi thành văn bản phát âm. Dịch vụ ngoài vi trả về các tệp âm thanh định dạng MP3; các tệp này được tải về phân vùng cục bộ, kết hợp với công cụ `ffprobe` để đo đạc chính xác thời lượng phát âm phục vụ khâu đồng bộ thời gian.

Tiến trình 5: Thực thi tự động hóa và ghi hình: Dựa trên liên kết web hoặc các lệnh phím tắt điều khiển đã được chỉ định, môi trường trình duyệt ảo Chromium hoặc hệ điều hành OS Desktop tiến hành thực thi các thao tác cơ học. Quá trình này xuất ra tệp video thô không có âm thanh cùng với tệp nhật ký mốc thời gian, sau đó lưu trữ trực tiếp tại phân vùng cục bộ.

Tiến trình 6: Phối trộn đa phương tiện và phân phối: Công cụ phần mềm FFmpeg tiếp nhận ba luồng dữ liệu đầu vào từ phân vùng lưu trữ cục bộ: tệp âm thanh MP3, video thô và tệp `trace.json`. FFmpeg tiến hành kết hợp hình ảnh và phối trộn âm thanh tại đúng các mốc thời gian thực tế để đồng bộ hóa thành một luồng video bài giảng hoàn chỉnh. Ở bước cuối cùng, tệp video MP4 được tải lên kho lưu trữ đối tượng Cloudflare R2, đồng thời hệ thống cập nhật trạng thái hoàn thành và sinh liên kết tải xuống bảo mật lưu vào MongoDB để người dùng truy xuất.

3.2.4 Điểm dừng kiểm duyệt và Tiền đề mở rộng

Khác với các hệ thống tự động hóa nơi đầu vào và đầu ra được xử lý ngầm, WebReel tích hợp triết lý thiết kế “Human-in-the-loop” tại giai đoạn 2.5 của chu trình. Cụ thể, sau khi phân hệ AI hoàn tất việc phân tích ngữ cảnh giao diện và sinh kịch bản thuyết minh, luồng bất đồng bộ của Worker sẽ tạm thời bị đóng băng. Trạng thái công việc được chuyển sang chờ duyệt.

Tại điểm dừng này, hệ thống giao lại quyền kiểm soát cho người dùng. Họ có thể kiểm tra chéo độ chính xác của kịch bản do LLM tạo ra, điều chỉnh từ vựng học thuật, hoặc thay đổi nhân vật giọng đọc trước khi cấp quyền cho Worker tiến hành quay màn hình và kết xuất video thực tế. Cơ chế này giảm thiểu đáng kể rủi ro sai lệch nội dung bài giảng số, đồng thời tiết kiệm tài nguyên máy chủ khỏi việc render lại các video không đạt yêu cầu.

Bên cạnh đó, kiến trúc hiện tại đã thiết lập sẵn các chỉ số đo lường khối lượng công việc, tạo tiền đề vững chắc cho việc phát triển bộ Quản trị tài nguyên tự động. Trong tương lai gần, hệ thống có khả năng tự động theo dõi biểu đồ tải trọng trên Redis để ra lệnh triển khai thêm các container Web Worker khi lưu lượng người dùng tăng đột biến, hướng tới một nền tảng SaaS hoàn chỉnh.

3.3 Thiết kế cơ sở dữ liệu và hạ tầng lưu trữ

3.3.1 Thiết kế lược đồ dữ liệu tài liệu

Hệ thống WebReel lựa chọn cơ sở dữ liệu phi quan hệ MongoDB làm giải pháp lưu trữ dữ liệu bền vững. Việc sử dụng mô hình dữ liệu dạng tài liệu với cấu trúc BSON mềm dẻo đặc biệt phù hợp với đặc thù của hệ thống, nơi các bản ghi công việc có cấu hình đầu vào và cấu trúc đầu ra biến động rất lớn tùy thuộc vào loại Worker thực thi.

Phần sau trình bày đặc tả chi tiết hai Collection cốt lõi chi phối toàn bộ logic nghiệp vụ của hệ thống:

Lược đồ Collection users

Tập hợp này đảm nhận vai trò lưu trữ thông tin định danh người dùng, kiểm soát quyền truy cập dựa trên vai trò và quản lý hạn mức tài nguyên để ngăn chặn việc lạm dụng năng lực xử lý của máy chủ.

Cấu trúc Collection:

```
{
  "_id": "ObjectId",
  "user_id": "UUID",
  "name": "String",
  "email": "String",
  "password_hash": "String",
  "role": "String",
  "tier": "String",
  "status": "String",
  "email_verified": "Boolean",
  "verification_token": "String",
  "quota": {
    "videos_per_month": "Integer",
    "videos_used_this_month": "Integer",
    "reset_date": "ISODate"
  },
  "created_at": "ISODate",
  "last_login": "ISODate"
}
```

Bảng 3.7 Đặc tả các trường dữ liệu Collection users

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc	Mô Tả Nghiệp Vụ
_id	ObjectId	Khóa chính	Mã định danh vật lý tự động sinh bởi hệ quản trị MongoDB.

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc	Mô Tả Nghiệp Vụ
user_id	UUID	Bắt buộc	Mã định danh logic của người dùng trên toàn hệ thống (được khởi tạo bằng thuật toán uuid4).
email	String	Unique, Bắt buộc	Địa chỉ thư điện tử dùng để xác thực và nhận thông báo.
password_hash	String	Bắt buộc	Chuỗi mật khẩu đã được băm an toàn thông qua thuật toán bcrypt.
role	String	Bắt buộc	Quyền hạn truy cập hệ thống: gồm người dùng, admin.
status	String	Bắt buộc	Trạng thái vòng đời tài khoản: pending_verification, active, suspended.
quota.videos_per_month	Integer	Bắt buộc	Hạn mức tài nguyên: Số lượng video tối đa được phép sản xuất trong một chu kỳ tháng.
quota.videos_used_this_month	Integer	Bắt buộc	Chỉ số đếm ghi nhận số lượng video thực tế đã kết xuất trong tháng hiện tại.
quota.reset_date	ISODate	Bắt buộc	Thời điểm hệ thống tiến hành tái thiết lập chỉ số videos_used_this_month về 0.

Lược đồ Collection jobs

Đây là tập hợp phức tạp của hệ thống, lưu trữ thông tin chi tiết về từng yêu cầu kết xuất bài giảng. Thiết kế ứng dụng kỹ thuật Đối tượng nhúng để nhóm các trường thông tin có tính chất liên quan (như config, progress, result), giúp giảm thiểu các phép kết nối phức tạp khi truy xuất dữ liệu.

Cấu trúc Collection:

<pre>{ "_id": "ObjectId", "job_id": "UUID", "status": "String",</pre>

```
"task": "String",

"environment": "String",

"config": {

    "enable_tts": "Boolean",

    "tts_engine": "String",

    "padding_ms": "Integer",

    "enable_review": "Boolean",

    "uploaded_file_url": "String",

    "browser_url": "String",

    "max_steps": "Integer"

},

"progress": {

    "current_phase": "Integer",

    "phase_name": "String",

    "message": "String"

},

"result": {

    "video_url": "String",

    "document_url": "String",

    "duration_seconds": "Double"

},

"created_at": "ISODate",

"completed_at": "ISODate"

}
```

Bảng 3.8 Đặc tả các trường dữ liệu cốt lõi Collection jobs

Tên Trường	Kiểu Dữ Liệu	Ràng Buộc	Mô Tả Nghiệp Vụ
job_id	UUID	Unique, Indexed	Mã định danh logic của công việc, được sử dụng làm khóa đối chiếu giữa Frontend, Redis và MongoDB.
status	String	Bắt buộc	Kiểm soát máy trạng thái: queued, processing, pending_review, completed, failed, cancelled.
environment	String	Bắt buộc	Định tuyến môi trường thực thi phân tán: web, os, presentation.
config.enable_review	Boolean	Bắt buộc	Cờ kích hoạt điểm dừng phê duyệt kịch bản thoại ở Pha 2.5.
config.padding_ms	Integer	Bắt buộc	Khoảng đệm thời gian âm thanh ở đầu và cuối thao tác phục vụ thuật toán nội suy của FFmpeg.
config.browser_url / uploaded_file_url	String	Tùy chọn	Dữ liệu đầu vào: URL trang web cần thao tác hoặc đường dẫn tải xuống tệp trình chiếu/tài liệu vật lý.
progress.current_phase	Integer	Tùy chọn	Chỉ số pha xử lý hiện tại của Worker (từ 0 đến 6). Dữ liệu này được liên tục cập nhật từ Redis Pub/Sub để phản ánh tiến độ thời gian thực.
result.video_url	String	Tùy chọn	Đường dẫn phân phối thành phẩm trở về Cloudflare R2 CDN sau khi FFmpeg hoàn tất phối trộn.
created_at / completed_at	ISODate	Tự động	Ghi nhận mốc thời gian bắt đầu và kết thúc vòng đời của một tác vụ

3.3.2 Chiến lược thiết lập chỉ mục

Trong kiến trúc xử lý bất đồng bộ của hệ thống WebReel, áp lực truy xuất dữ liệu đè nặng lên các truy vấn đọc tại phân hệ Dashboard. Người dùng có xu hướng liên tục làm mới giao diện để cập nhật tiến độ, lọc các tác vụ đang ở trạng thái kiểm duyệt hoặc truy xuất lịch sử video mới khởi tạo. Nếu cơ sở dữ liệu MongoDB tích lũy hàng chục ngàn bản ghi công việc mà không được định cấu hình chỉ mục hợp lý, hệ quản trị sẽ buộc phải thực hiện cơ chế quét toàn bộ tập hợp. Hành vi này trực tiếp gây ra hiện tượng nghẽn cổ chai, làm tăng đột biến dung lượng CPU máy chủ và kéo dài thời gian phản hồi của FastAPI Backend.

Để tối ưu hóa tốc độ phản hồi và chuyển đổi các câu lệnh truy vấn sang cơ chế quét chỉ mục, hệ thống triển khai chiến lược thiết lập chỉ mục dựa trên nguyên tắc toán học ESR. Chiến lược này không áp dụng việc liệt kê chỉ mục đơn lẻ một cách rời rạc, mà quy hoạch theo từng mô hình luồng dữ liệu thực tế của ứng dụng:

Đối với các tác vụ truy vấn chi tiết hoặc cập nhật tiến trình từ phía các Ephemeral Workers ngằm gửi về API Gateway, hệ thống áp dụng một chỉ mục đơn độc nhất trên trường `{ job_id: 1 }`. Việc băm chỉ mục này đảm bảo thời gian định vị một bản ghi Job cụ thể luôn duy trì ở độ phức tạp thuật toán là $O(1)$, ngay cả khi quy mô phình to của cơ sở dữ liệu.

Mặt khác, để phục vụ luồng hiển thị danh sách mặc định tại Bảng điều khiển của người dùng, hệ thống tích hợp một chỉ mục phức hợp theo cấu trúc `{ user_id: 1, created_at: -1 }`. Thiết kế này giải quyết bài toán sắp xếp dữ liệu trong bộ nhớ RAM. MongoDB sẽ lọc chính xác các tác vụ theo định danh người dùng trước, sau đó trả về kết quả đã được sắp xếp sẵn theo trục thời gian mới nhất nhờ cấu trúc cây chỉ mục, hạn chế độ trễ tính toán của CPU.

Tư duy này tiếp tục được mở rộng cho các bộ lọc nâng cao trên Dashboard thông qua chỉ mục phức hợp đa trường `{ user_id: 1, status: 1, type: 1, created_at: -1 }`. Khi người dùng thực hiện các thao tác lọc chéo như tìm kiếm các video thuộc chế độ Google Slides đã hoàn thành, chỉ mục này được dùng như một màng lọc nhiều tầng, định vị chính xác tập hợp con của dữ liệu và trả về kết quả tức thì cho Client.

Cuối cùng, để hỗ trợ khâu vận hành ngầm của hạ tầng, hệ thống thiết lập riêng một chỉ mục phức hợp chuyên dụng dành cho phân hệ Autoscaler và bộ quản lý hàng đợi với cấu trúc { status: 1, created_at: 1 }. Chỉ mục này giúp tiến trình giám sát liên tục quét nhanh các tác vụ đang có trạng thái xếp hàng theo trình tự thời gian tăng dần. Điều này đảm bảo nguyên tắc của hàng đợi vào trước ra trước, giúp hệ thống nhận diện chính xác tình trạng nghẽn tải để kích hoạt các container thực thi kịp thời mà không gây tổn hao tài nguyên đọc của đĩa cứng.

3.3.3 Kiến trúc lưu trữ đối tượng đám mây

Đối với các tệp tin có dung lượng lớn phi cấu trúc như tài liệu trình chiếu gốc, tệp âm thanh tĩnh và video bài giảng thành phẩm, hệ thống không lưu trữ trực tiếp trên máy chủ VPS hay cơ sở dữ liệu MongoDB tránh hiện tượng quá tải dung lượng đĩa. Thay vào đó, kiến trúc tích hợp dịch vụ lưu trữ đối tượng đám mây Cloudflare R2. Lựa chọn này mang lại ưu thế kỹ thuật vượt trội nhờ chính sách miễn phí băng thông tải xuất, giúp cắt giảm đáng kể chi phí vận hành hạ tầng khi hệ thống phải phân phối luồng video liên tục đến người dùng cuối.

Quy hoạch phân phối tĩnh và Bảo mật đường truyền: Để đảm bảo tốc độ tải trang và bảo vệ tài sản số, hệ thống thiết lập cấu hình phân phối tĩnh kết hợp với Mạng phân phối nội dung:

- Định tuyến Tên miền phụ: Phân vùng lưu trữ được ánh xạ với một tên miền phụ độc lập thông qua bản ghi DNS CNAME, đồng thời hệ thống tự động cấp phát và quản lý chứng chỉ bảo mật SSL/TLS.
- Cơ chế liên kết ký sẵn: Để ngăn chặn hành vi truy cập trái phép hoặc dò quét tệp tin diện rộng, kho lưu trữ được cấu hình đóng kín ở chế độ Riêng tư. Khi Client gửi yêu cầu truy xuất tài liệu hợp lệ, API Gateway sẽ sử dụng khóa bí mật để sinh ra một đường dẫn tải xuống đã được ký mã hóa với thời gian hiệu lực giới hạn. Trình duyệt chỉ có thể sử dụng liên kết tạm thời này để thiết lập luồng phát video an toàn.

Quản trị vòng đời đối tượng: Trong suốt vòng đời thực thi, các Ephemeral Workers liên tục tạo ra hàng trăm Megabytes dữ liệu trung gian như: ảnh tĩnh phân rã từ PDF, tệp âm thanh MP3 đơn lẻ, video thô chưa ghép tiếng và tệp nhật ký vết. Việc lưu trữ vĩnh viễn lượng dữ liệu rác này sẽ gây lãng phí tài nguyên đám mây nghiêm

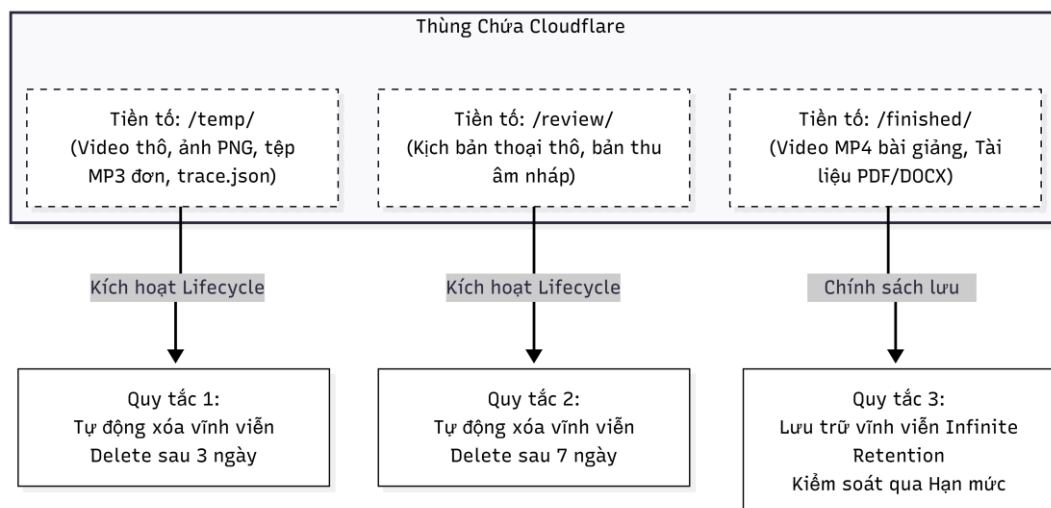
Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
trọng. Do đó, hệ thống thiết lập các Quy tắc vòng đời chạy ngầm trực tiếp trên hạ tầng Cloudflare để tự động thanh lọc dữ liệu dựa trên tiền tố thư mục:

1. Quy tắc thanh lọc thư mục /temp/: Phân vùng này chứa toàn bộ dữ liệu thô phục vụ khâu phối trộn của FFmpeg ở Pha 6. Quy tắc được thiết lập để tự động tiêu hủy toàn bộ các đối tượng mang tiền tố này sau 3 ngày kể từ thời điểm khởi tạo, giải phóng dung lượng mà không cần gọi API thủ công.

2. Quy tắc thanh lọc thư mục /review/: Khu vực lưu trữ các kịch bản thoại thô và tệp âm thanh mẫu phục vụ điểm dừng kiểm duyệt. Nếu người dùng không có hành động phê duyệt, các dữ liệu lưu nháp này sẽ bị hệ thống xóa vĩnh viễn sau 7 ngày.

3. Quy tắc lưu trữ bền vững /finished/: Phân vùng lưu trữ tệp video MP4 và tài liệu DOCX/PDF thành phẩm. Phân vùng này không áp dụng quy tắc tự động xóa theo thời gian. Dữ liệu tại đây chỉ bị loại bỏ khi người dùng chủ động gửi lệnh xóa hoặc khi hệ thống quét thấy tài khoản vượt quá Hạn mức lưu trữ được quy định trong cơ sở dữ liệu MongoDB.

Sơ đồ quy tắc vòng đời đối tượng



Hình 3.5 Sơ đồ quy tắc vòng đời đối tượng

3.4 Thiết kế chi tiết các phân hệ thực thi

Khác với các kiến trúc xử lý truyền thống nơi các Worker hoạt động liên tục để nhận nhiều công việc, hệ thống WebReel áp dụng mô hình Container ngắn hạn. Khi Autoscaler phát hiện một Job mới trong hàng đợi Redis, nó sẽ khởi tạo một Container mới, sở hữu môi trường mạng và hệ thống tệp độc lập. Container này chỉ thực thi đúng

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
một Job, sau đó tự động phát tín hiệu tắt và giải phóng toàn bộ tài nguyên CPU/RAM. Thiết kế này giảm thiểu rủi ro rò rỉ bộ nhớ thường gặp khi Chromium chạy thời gian dài, đồng thời đảm bảo tính cô lập bảo mật giữa dữ liệu của các khách hàng khác nhau.

Các Worker trong WebReel được xây dựng dựa trên cùng một khuôn mẫu xử lý tổng quát gồm sáu giai đoạn: tiếp nhận tác vụ, phân tích bằng AI, xây dựng kịch bản thao tác, thực thi hành động, ghi nhận dữ liệu hình ảnh và kết xuất video. Tuy nhiên, tùy thuộc vào môi trường mục tiêu, từng Worker sẽ điều chỉnh một số bước trong pipeline để phù hợp với đặc tính của nền tảng tương ứng.

Trên cơ sở đó, hệ thống triển khai ba nhóm Worker chuyên biệt:

- Web Worker: xử lý các nền tảng Web thông qua trình duyệt Chromium.
- Presentation Worker: xử lý các bài trình chiếu và nền tảng trình bày nội dung bài giảng.
- OS Worker: xử lý các ứng dụng desktop và giao diện hệ điều hành.

3.4.1 Phân hệ Web Worker quy trình 6 pha

Web Worker là lõi tự động hóa chịu trách nhiệm điều hướng trang web, trích xuất dữ liệu, tổng hợp giọng nói và ghi hình. Về bản chất, Web Worker là nơi AI Agent quan sát giao diện, suy luận bước tiếp theo và chuyển quyết định thành thao tác cụ thể trên trình duyệt. Để đảm bảo video đầu ra ổn định, không có độ trễ tải trang và khớp với giọng đọc, quá trình thực thi được chia cắt thành một đường ống gồm 6 pha hoạt động tuần tự phối hợp chặt chẽ với nhau:

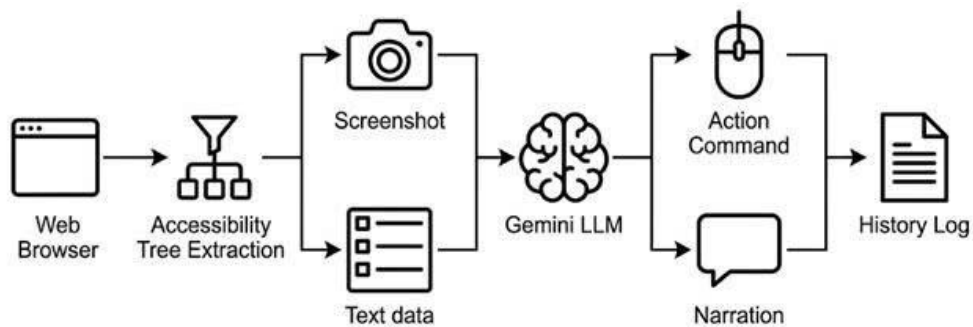
Pha 1: Scout. Pha này như một “bản nháp” để AI đi thử và học luồng thao tác.

Môi trường giả lập: Khởi chạy máy chủ hiển thị ảo Xvfb trên nền tảng Linux, kết hợp trình duyệt Chromium.

Cơ chế Session Snapshot: Trình duyệt ảo không bắt đầu từ con số không, mà được liên kết vào phân vùng chứa Chrome Profile của Session Manager ở chế độ chỉ đọc. Điều này cho phép Worker kế thừa toàn bộ Cookies và Token đăng nhập hợp lệ mà không làm hỏng cấu hình gốc.

Điều khiển CDP và Anti-bot: Tác nhân AI kết nối vào trình duyệt qua cổng Giao thức CDP. Để qua mặt các hệ thống chống Bot, thuộc tính AutomationControlled

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng của WebDriver bị vô hiệu hóa. Các thao tác di chuyển chuột của AI không diễn ra tức thời mà được nội suy toán học theo đường cong Bezier, mô phỏng chính xác gia tốc và độ lệch tự nhiên của bàn tay con người.



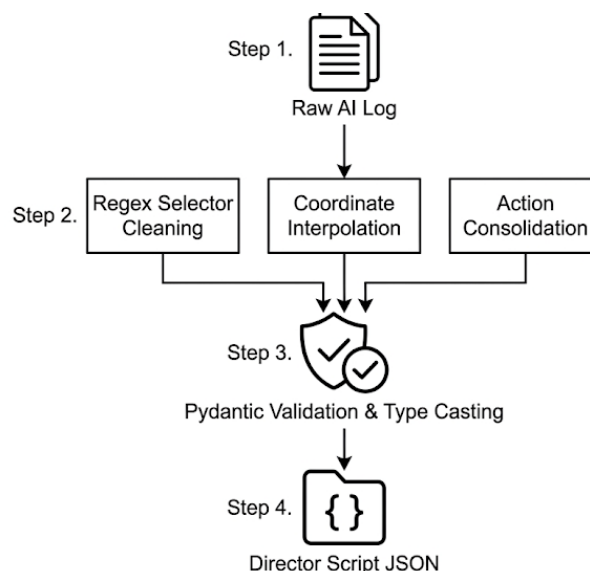
Hình 3.6 Quy trình điều hướng trình duyệt

Pha 2: Parser - Phân tích cú pháp cấu trúc với Pydantic

Dữ liệu lịch sử thao tác từ Pha 1 là một khối JSON lớn và phi cấu trúc. Hệ thống sử dụng thư viện đối tượng hóa Pydantic để kiểm chứng và phân rã dữ liệu thành hai lược đồ độc lập:

Lược đồ Hành động: Chứa các tọa độ X/Y, chuỗi nhập liệu và các thẻ giữ chỗ tạm dừng (ví dụ: [NARRATION:0]).

Lược đồ Thuyết minh: Danh sách tuyến tính các câu thoại tiếng Việt mang tính giáo dục.



Hình 3.7 Quy trình biên dịch dữ liệu

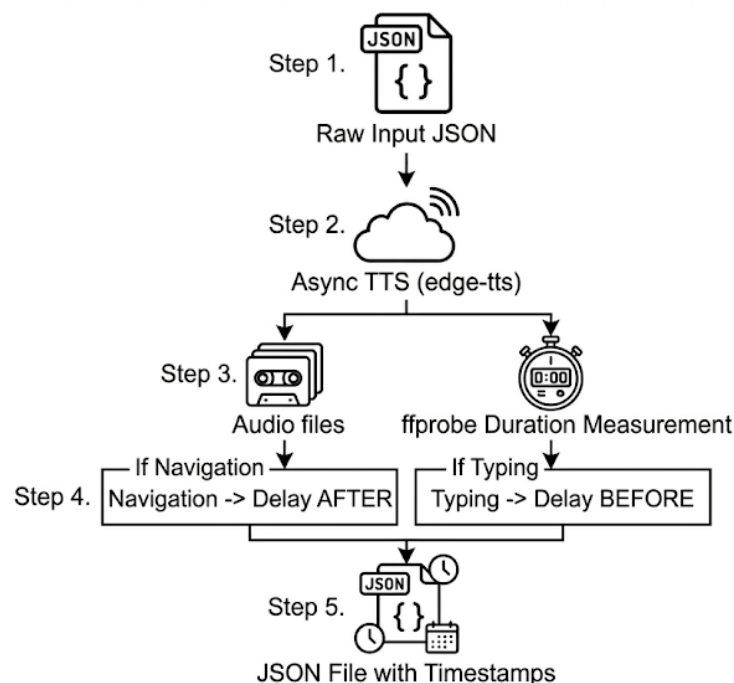
Pha 2.5: Human-in-the-loop - Điểm dừng kiểm duyệt

Thay vì mù quáng kết xuất, Worker tạm dừng tiến trình, đẩy Lược đồ Thuyết minh lên giao diện người dùng và chuyển sang trạng thái chờ (Tối đa 30 phút). Người dùng có quyền hiệu đính các thuật ngữ chuyên ngành. Khi có tín hiệu phê duyệt qua Redis Pub/Sub, tiến trình được đánh thức để tiếp tục.

Pha 3: Ground-Truth TTS - Tổng hợp và Đo đạc vi giây

Gửi Lược đồ Thuyết minh đã duyệt tới dịch vụ TTS ngoài để tổng hợp thành các tệp MP3 chất lượng tốt.

Để đồng bộ hình-âm ở mức khung hình, hệ thống gọi bộ công cụ ffprobe quét trực tiếp vào header của từng tệp MP3, bóc tách và ghi nhận thời lượng phát âm thực tế với độ chính xác đến từng mili-giây.



Hình 3.8 Quy trình tổng hợp và đo lường vi giây

Pha 4: Injector - Tiêm thời lượng và Khoảng đệm

Hệ thống đọc lại Lược đồ Hành động ở pha 2, tìm các thẻ giữ chỗ [NARRATION:X] và thay thế bằng thời lượng mili-giây tương ứng đã đo ở Pha 3.

Thuật toán Padding: Cộng thêm một khoảng đệm thời gian cố định 300ms vào đầu và cuối mỗi mốc thời gian. Khoảng đệm này tạo ra một nhịp nghỉ thị giác trước khi giọng nói cất lên, mô phỏng nhịp điệu giảng dạy của một giảng viên.

Pha 5: Execution - Tái diễn và Ghi hình

Đây là bước ghi hình chính thức, hạn chế độ trễ do mạng lưới hay thời gian suy nghĩ của AI ở Pha đầu tiên.

Sử dụng thư viện WebReel, quá trình bao gồm trình duyệt Chromium tải lại trang ở độ phân giải Full HD. Trình ghi hình gắn trực tiếp vào bộ đệm khung hình của Xvfb, thực hiện kết xuất chuỗi khung hình với tốc độ khung hình duy trì ổn định ở mức 12 FPS.

Worker thực thi kịch bản cơ học. Cứ mỗi thao tác diễn ra, hệ thống ghi nhận chính xác mốc thời gian thực và xuất ra một tệp nhật ký giám sát trace.json.

Pha 6: Composer - Phối trộn Đa phương tiện tốc độ cao

Hệ thống đẩy Video MP4 thô, danh sách tệp âm thanh MP3 và tệp trace.json vào luồng xử lý của FFmpeg.

Tối ưu hóa Stream Copy: Thay vì phải giải mã và mã hóa lại toàn bộ luồng video khổng lồ gây tốn kém CPU, FFmpeg được cấu hình sử dụng tham số -c:v copy. Thuật toán này chỉ thực hiện việc hòa trộn các luồng âm thanh rời rạc vào các mốc thời gian tương ứng của luồng Video tĩnh đã có sẵn, giúp tốc độ xuất bản video bài giảng cuối cùng diễn ra tương đối nhanh mà không suy giảm chất lượng hình ảnh.

Sau khi hoàn thành Pha 6, tệp MP4 cuối cùng được đẩy lên CDN, và Ephemeral Container tự động gửi tín hiệu SIGTERM để tiêu hủy chính nó, giải phóng phần cứng cho các tác vụ tiếp theo.

3.4.2 Phân hệ Presentation Workers

Presentation Worker được xây dựng trên cùng nền tảng pipeline sáu pha của Web Worker. Các thành phần điều phối tác vụ, sinh kịch bản bằng AI, tổng hợp giọng nói, ghi hình và kết xuất video được tái sử dụng gần như nguyên vẹn. Điểm khác biệt nằm ở lớp tương tác nghiệp vụ. Thay vì thao tác trên cây DOM của một ứng dụng Web bất kỳ, Worker này làm việc với tài liệu trình chiếu và sử dụng cơ chế điều hướng theo dòng thời gian của các trang chiếu. Vì vậy, Presentation Worker có thể được xem là một biến thể chuyên biệt của Web Worker dành riêng cho bài toán tự động hóa thuyết trình.

Nếu phân hệ Web Worker tập trung xử lý các tác vụ cần thao tác vào web, thì cụm Presentation Workers được thiết kế để giải quyết một bài toán khác: Ghi hình lại trọn vẹn các hiệu ứng hoạt ảnh và hiệu ứng chuyển trang gốc của tài liệu.

Một trong những thách thức kỹ thuật lớn khi tự động hóa trình duyệt trên các nền tảng đám mây (OneDrive, Google Workspace) là sự can thiệp của Giao diện người dùng. Bất kỳ một chuyển động chuột hay thao tác nhấp nháy nào của AI cũng có thể vô tình kích hoạt các thanh công cụ, menu ngữ cảnh hoặc thanh điều khiển phát lại nổi trên màn hình, dẫn đến việc khung hình video bài giảng bị nhiễu bởi các phần tử rác. Để hạn chế rủi ro này, hệ thống áp dụng hai chiến lược thiết kế lỗi:

Cơ chế Thao túng trạng thái URL: Thay vì để AI tự điều hướng từ trang chủ, tìm kiếm tệp và nhấp vào nút “Trình chiếu” một cách thủ công dễ sinh lỗi, hệ thống can thiệp trực tiếp vào cấu trúc liên kết ngay từ Pha Chuẩn bị:

Tối ưu hóa Google Slides: Sau khi tệp PPTX được tải lên và chuyển đổi định dạng qua Google Drive API, hệ thống sử dụng thuật toán thao túng URL bằng cách nối thêm hậu tố /present vào mã định danh tệp tin. Kỹ thuật ép buộc định tuyến này giúp trình duyệt ảo khi vừa khởi động sẽ đi thẳng vào chế độ trình chiếu toàn màn hình, bỏ qua giao diện soạn thảo phức tạp của Google Slides.

Định tuyến OneDrive: Đối với hệ sinh thái Microsoft, Worker khởi tạo một liên kết chia sẻ trực tiếp kèm mã xác thực nội tại. Liên kết này cho phép trình duyệt ảo danh của Worker vượt qua cổng xác thực và đi thẳng vào giao diện xem tài liệu.

Cơ chế Tối giản hóa hành động chuột Trong tập lệnh chỉ dẫn động gửi cho tác nhân AI điều khiển trình duyệt, hệ thống thiết lập một rào cản hành vi nghiêm ngặt:

Lệnh cấm: Tác nhân AI bị tước bỏ quyền gọi các hàm API liên quan đến con trỏ như di chuyển chuột, nhấp chuột, hoặc cuộn trang. Điều này đảm bảo giao diện trình chiếu duy trì trạng thái tĩnh lặng, không kích hoạt bất kỳ sự kiện onHover hay onMouseMove nào của DOM làm hiện thanh công cụ.

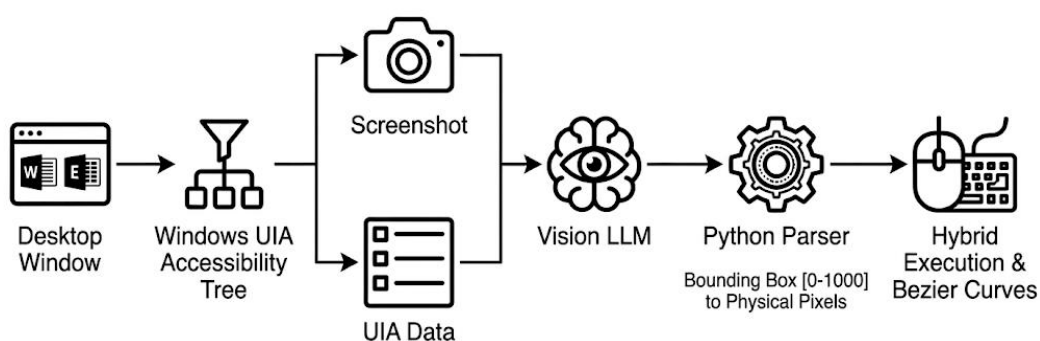
Mô phỏng phím tắt phần cứng: Luồng điều khiển được chuyển sang các sự kiện bàn phím. Với OneDrive: AI được chỉ định gửi tổ hợp phím Ctrl + F5 để kích hoạt trình chiếu, nhấn mũi tên phải để chuyển slide sau khi phát xong tệp âm thanh thuyết minh, và kết thúc bằng phím Escape. Với Google Slides: Do đã thao túng URL

/present, AI chỉ được phép dùng phím mũi tên và bị cấm phím Escape tránh việc trình duyệt bị văng khỏi chế độ toàn màn hình gây hỏng bản ghi hình.

Quy trình Dọn dẹp Đám mây: Vì các Worker này phải tải tài liệu gốc của người dùng lên hạ tầng đám mây cá nhân hoặc của hệ thống để mượn bộ kết xuất của họ, tính riêng tư của dữ liệu được đặt lên hàng đầu. Ngay sau khi Pha ghi hình hoàn tất và xuất ra tệp video thô, một tiến trình dọn dẹp sẽ gọi API đám mây thực thi lệnh DELETE vĩnh viễn tệp tin trình chiếu khỏi máy chủ của bên thứ ba, đảm bảo không để lại bất kỳ dấu vết dữ liệu nhạy cảm nào của khách hàng.

3.4.3 Phân hệ OS Worker

Nếu Web Worker và Presentation Worker hoạt động an toàn bên trong các môi trường giả lập biệt lập trên Linux, thì OS Worker phải đối mặt với một thách thức kiến trúc khác: Tương tác trực tiếp với các phần mềm nguyên bản trên hệ điều hành Windows vật lý hoặc máy ảo.



Hình 3.9 Quy trình tự động hóa ở cấp độ hệ điều hành

Phân hệ này đòi hỏi khả năng kiểm soát phần cứng cấp thấp và xử lý giao diện người dùng phức tạp thông qua 3 cơ chế thiết kế trọng tâm:

Cơ chế giám sát tài nguyên nhân rồi: Vì OS Worker chạy trực tiếp trên môi trường Windows Desktop (nơi có thể có người dùng thật đang làm việc), nguy cơ tranh chấp quyền điều khiển chuột và bàn phím là rất lớn. Hệ thống giải quyết rào cản này thông qua Pha 0 bằng cách tích hợp trực tiếp vào Win32 API của Windows.

Worker liên tục gọi hàm `GetLastInputInfo` để lắng nghe các tín hiệu ngắt từ thiết bị ngoại vi, để đo lường thời gian nhàn rỗi của hệ thống.

Chỉ khi máy tính không có bất kỳ tương tác vật lý nào vượt qua ngưỡng an toàn, Worker mới được phép mở kết nối qua hàng đợi `os-queue` để nhận tác vụ.

Bảo vệ ngắt mạch: Nếu trong quá trình tải tệp hoặc rà quét giao diện, cảm biến phát hiện người dùng dịch chuyển chuột trở lại, Worker sẽ hủy bỏ luồng thực thi, giải phóng phần mềm và đẩy trả Job về lại Redis Queue để đảm bảo không làm gián đoạn công việc của con người.

Giải quyết bài toán Bùng nổ cấu trúc bằng Adapter Pattern Để AI có thể “nhìn thấy” và thao tác trên phần mềm Windows, hệ thống sử dụng thư viện UI Automation để quét toàn bộ màn hình và trích xuất cây cấu trúc phân tử. Tuy nhiên, các phần mềm phức tạp như Excel chứa một cây cấu trúc khổng lồ với hàng vạn node (ô lưới, thanh cuộn, menu ẩn). Việc nạp toàn bộ dữ liệu thô này vào ngữ cảnh của mô hình ngôn ngữ lớn sẽ ngay gây ra hiện tượng Bùng nổ cấu trúc, làm tràn giới hạn từ và gây ảo giác cho AI.

Giải pháp áp dụng: Kiến trúc sử dụng Mẫu thiết kế bộ tiếp hợp. Lớp Adapter đứng ở giữa, thực hiện việc duyệt qua cây UIA gốc của Windows, thanh lọc các phần tử ẩn hoặc không có tính tương tác, và chuyển đổi các tọa độ vật lý thành một lược đồ JSON tối giản. Tác nhân AI đa phương thức chỉ cần đọc lược đồ đã được làm sạch này kết hợp với ảnh chụp màn hình để lập kế hoạch thao tác chính xác.

Khôi phục trạng thái ứng dụng sạch: Việc AI sử dụng UIA để thăm dò và bóc tách cấu trúc phần mềm ở Pha 1 sẽ vô tình làm thay đổi trạng thái nguyên sơ của ứng dụng (ví dụ: các ô dữ liệu bị bôi đen, các menu thả xuống bị kẹt mở). Nếu tiến hành bật trình quay màn hình ngay lúc này, video bài giảng sẽ chứa các khung hình lỗi, làm mất đi tính chuyên nghiệp.

Để giải quyết, WebReel thiết kế một điểm chèn kỹ thuật ở Pha 2.75 gọi là State Reset. Sau khi AI đã hoàn tất lập kế hoạch tọa độ và tổng hợp xong toàn bộ tệp âm thanh TTS, Worker sẽ gửi tín hiệu cưỡng chế đóng ứng dụng mà không lưu lại bất kỳ sự thay đổi nào. Ngay sau đó, Worker gọi lệnh khởi chạy lại chính tệp tài liệu gốc ban đầu. Quá trình này đưa giao diện phần mềm về trạng thái sạch hoàn hảo, sẵn sàng cho OS Recorder bắt đầu ghi hình và thực hiện chuỗi thao tác chuột/phím mô phỏng một cách chính xác.

Mặc dù quy trình thực thi khác biệt đáng kể so với Web Worker và Presentation Worker, OS Worker vẫn sử dụng chung cơ chế điều phối công việc, hàng đợi Redis,

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng quản lý trạng thái Job và vòng đời Ephemeral Worker của toàn hệ thống. Nhờ đó, các Worker có thể vận hành độc lập nhưng vẫn tuân thủ kiến trúc điều phối thống nhất.

3.4.4 Phân tích kiến trúc Worker đa môi trường

Mặc dù cả ba nhóm Worker trong hệ thống đều có chung mục tiêu là chuyển đổi một yêu cầu đầu vào thành video bài giảng hoàn chỉnh, bản chất của các môi trường thực thi mà chúng phải tương tác lại hoàn toàn khác nhau. Sự khác biệt này khiến việc xây dựng một Worker duy nhất để xử lý mọi loại tác vụ trở nên phức tạp, khó bảo trì và không tối ưu tốt về mặt tài nguyên.

Web Worker được thiết kế để làm việc với các ứng dụng Web hiện đại thông qua trình duyệt Chromium. Nhóm Worker này tương tác trực tiếp với cấu trúc DOM, các phần tử HTML và các sự kiện JavaScript được sinh ra trong quá trình duyệt Web. Các thao tác như tìm kiếm nội dung, điều hướng trang, nhập liệu hoặc thu thập dữ liệu đều được thực hiện thông qua lớp tự động hóa Playwright.

Ngược lại, Presentation Worker không thao tác trên cấu trúc DOM mà làm việc với các tài liệu trình chiếu. Quá trình thực thi tập trung vào việc quản lý vòng đời của bài thuyết trình, bao gồm tạo slide, chèn nội dung, đồng bộ hóa lời thoại và điều khiển tiến trình trình chiếu. Đây là nhóm tác vụ có đặc tính thiên về xử lý đa phương tiện và kết xuất nội dung hơn là tương tác Web thông thường.

Trong khi đó, OS Worker hoạt động trên môi trường hệ điều hành cục bộ, nơi các thành phần giao diện không được biểu diễn dưới dạng HTML hay tài liệu trình chiếu. Các thao tác phải được thực hiện thông qua cây giao diện người dùng, cơ chế Accessibility API hoặc các sự kiện bàn phím và chuột ở cấp hệ điều hành. Điều này tạo ra những yêu cầu kỹ thuật hoàn toàn khác so với hai nhóm Worker còn lại.

Từ góc độ kiến trúc hệ thống, việc phân tách thành ba nhóm Worker chuyên biệt mang lại ba lợi ích quan trọng.

Thứ nhất, mỗi nhóm Worker có thể thực hiện độc lập cho môi trường thực thi tương ứng mà không ảnh hưởng đến các nhóm còn lại. Điều này cho phép hệ thống lựa chọn thư viện, cấu hình tài nguyên và chiến lược xử lý phù hợp với từng loại tác vụ.

Thứ hai, kiến trúc này tạo ra khả năng cô lập lỗi hiệu quả. Khi một Worker gặp sự cố do rò rỉ bộ nhớ, treo trình duyệt hoặc lỗi kết xuất đa phương tiện, sự cố chỉ ảnh

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng hưởng đến hàng đợi và vùng thực thi của nhóm Worker đó mà không làm gián đoạn toàn bộ hệ thống.

Thứ ba, mô hình phân rã theo miền thực thi giúp hệ thống dễ dàng mở rộng trong tương lai. Các loại Worker mới có thể được bổ sung để hỗ trợ các nền tảng khác mà không cần thay đổi kiến trúc tổng thể của hệ thống. Ví dụ, một Video Editing Worker hoặc Mobile Automation Worker có thể được triển khai như các thành phần độc lập nhưng vẫn sử dụng chung cơ chế điều phối Redis và API Gateway hiện có.

Vì vậy, việc xây dựng kiến trúc Worker đa môi trường giải quyết các khác biệt về mặt công nghệ và là nền tảng giúp hệ thống đạt được khả năng mở rộng, tính ổn định và khả năng bảo trì trong quá trình vận hành thực tế.

Bảng 3.9 So sánh các worker

Tiêu chí	Web Worker	Presentation Worker	OS Worker
Môi trường thực thi	Chromium	Google Slides / PowerPoint	Windows Desktop
Đầu vào	URL, yêu cầu Web	PPTX, PDF, Script	Ứng dụng cục bộ
Công nghệ lõi	Playwright	Google Slides API, PowerPoint	UI Automation
Cơ chế tương tác	DOM Elements	Slide Timeline	Native UI Tree
Loại tác vụ	Web Automation	Presentation Automation	Desktop Automation
Đặc tính tải	Trung bình	Cao	Trung bình
Khả năng mở rộng	Cao	Trung bình	Phụ thuộc Host
Hàng đợi xử lý	web-queue	presentation-queue	os-queue

Việc phân rã hệ thống thành 3 nhóm Worker chuyên biệt khắc phục các giới hạn kỹ thuật đặc thù trên từng nền tảng (Trình duyệt, Lưu trữ đám mây, Phần mềm hệ điều hành) và đảm bảo tuân thủ nguyên tắc Đơn nhiệm trong kiến trúc Microservices.

Sự kết hợp đồng bộ giữa các thuật toán nội suy quỹ đạo toán học, kỹ thuật xử lý bất đồng bộ đa pha và các rào cản bảo mật ở mức hạ tầng đã cấu thành một đường ống sản xuất học liệu số đạt độ ổn định cao, tối ưu hóa tài nguyên phần cứng và sẵn sàng cho việc mở rộng tải trọng trong môi trường triển khai thực tế.

3.5 Thiết kế giao diện người dùng và luồng tương tác

3.5.1 Kiến trúc bảng điều khiển tập trung

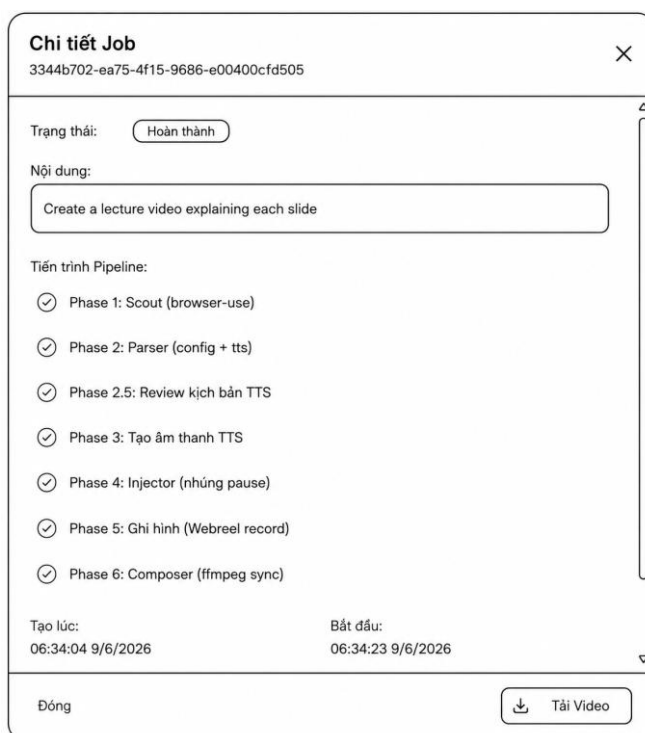
Trong mô hình vi dịch vụ phân tán, sự phức tạp của các tiến trình chạy ngầm cần được trừu tượng hóa để mang lại trải nghiệm liền mạch cho người dùng cuối. Phân hệ Frontend của WebReel được kiến trúc dưới dạng Ứng dụng trang đơn là trung tâm điều khiển kết nối trực tiếp với API Gateway. Kiến trúc giao diện được quy hoạch thành bốn phân khu chức năng, tương ứng với vòng đời của một tác vụ sản xuất video:

Phân khu Khởi tạo Tác vụ: Đây là điểm chạm đầu tiên nơi người dùng định hình sản phẩm. Giao diện khởi tạo được thiết kế linh hoạt dựa trên nguyên lý đa phương thức đầu vào. Người dùng có thể cung cấp siêu liên kết cho các tác vụ Web, tải lên tài liệu vật lý (PPTX/PDF) cho các tác vụ trình chiếu, hoặc nhập lệnh văn bản thuần túy cho môi trường OS. Hệ thống giao diện sẽ tự động đối chiếu các tham số này để đóng gói thành một tệp cấu hình JSON và định tuyến chính xác yêu cầu của người dùng vào các hàng đợi Redis tương ứng ở tầng Backend thông qua cơ chế chọn môi trường thực thi.

The screenshot displays the 'WebReel' application interface for creating a new video. On the left, a sidebar contains the user's profile (test, user: test@example.org) and navigation options like 'Tổng quan' and 'Tạo mới'. The main area, titled 'Tạo Video Mới', guides the user through video creation. It includes a subtitle 'Cung cấp ý tưởng hoặc file trình chiếu, Agent sẽ tự động xử lý.' and a 'Cài đặt Pipeline Video' section. This section allows selecting the video type (currently 'Tự động') and provides buttons for different input methods: 'Web', 'Trình chiếu', and 'Máy tính'. A 'Đăng nhập điện: WEB' button is also present. A text area for 'Prompt / Ý tưởng kịch bản' contains an example prompt: 'ví dụ: Hướng dẫn đăng ký tài khoản Github...'. Below this, there are settings for 'TTS Engine', 'Giọng đọc', and 'Độ trễ (ms)' (set to 500). Checkboxes for 'Bật Voice (TTS)' and 'Tạm dừng để Review Kịch Bản' are included. The process concludes with a 'Tạo Job' button.

Hình 3.10 Phác thảo giao diện khởi tạo tác vụ

Trình giám sát tiến độ thời gian thực: Để giải quyết “điểm mù” thông tin khi các Ephemeral Worker hoạt động ngắn kéo dài từ 5 đến 15 phút, Bảng điều khiển được thiết kế tích hợp cơ chế giám sát theo thời gian thực. Khi truy cập vào chi tiết một dự án đang xử lý, giao diện không yêu cầu người dùng phải tải lại trang thủ công. Thay vào đó, Frontend lắng nghe các gói tin sự kiện được đẩy từ Redis Pub/Sub thông qua API, liên tục cập nhật trạng thái của 6 pha xử lý. Đặc biệt, luồng giao diện này bao gồm một Điểm dừng kiểm duyệt. Khi hệ thống báo hiệu trạng thái `pending_review`, giao diện tự động bật biểu mẫu cho phép người dùng can thiệp, hiệu đính trực tiếp nội dung văn bản kịch bản trước khi ra lệnh cho hệ thống tiếp tục tổng hợp giọng nói.



Hình 3.11 Chi tiết job

Kho lưu trữ học liệu: Phân khu này như một thư viện số cá nhân. Đối với các tác vụ đã đạt trạng thái hoàn thành, giao diện sẽ hiển thị siêu dữ liệu bao gồm thời lượng video và ảnh thu nhỏ. Điểm nhấn về mặt kiến trúc tại đây là cơ chế bảo mật tài nguyên: giao diện không lưu trữ URL gốc của video, mà thực hiện một truy vấn tức thời đến API để lấy Liên kết ký sẵn có thời hạn từ Cloudflare R2 CDN.

Trạm quản lý phiên làm việc của Admin: Đối với tài khoản quản trị viên, Bảng điều khiển cung cấp một phân khu đặc quyền mang tên Trạm quản lý phiên. Do tính chất của các tác vụ tải tài liệu lên đám mây cần phải vượt qua hệ thống chống Bot và

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng mã Captcha, giao diện Admin tích hợp trực tiếp một khung hiển thị nội tuyến kết nối luồng đồ họa noVNC đến thẳng trình duyệt ảo gốc.

Giao diện này cho phép quản trị viên tương tác chuột và bàn phím từ xa ngay trên nền web để đăng nhập các tài khoản dịch vụ, sau đó nhấn nút “Đóng băng” để lưu lại trạng thái Cookies cho các Worker kế thừa. Bên cạnh đó, màn hình này còn tích hợp Cảnh báo ngắt mạch; nếu hệ thống ngầm phát hiện tài khoản bị vãng phiên, Bảng điều khiển sẽ hiển thị cảnh báo đỏ và cho phép Admin tạm dừng các hàng đợi liên quan để xử lý trước khi có hàng loạt tác vụ bị lỗi dây chuyền.

3.5.2 Luồng giao tiếp đồng bộ tiến độ qua WebSocket/SSE

Đối với một hệ thống sản xuất đa phương tiện bất đồng bộ như WebReel, việc duy trì tính minh bạch của tiến trình xử lý ngầm tại giao diện người dùng là một tiêu chuẩn kỹ thuật bắt buộc. Do vòng đời thực thi của các Ephemeral Workers kéo dài và đi qua nhiều trạng thái phức tạp, việc sử dụng giải pháp truy vấn lặp truyền thống sẽ tạo ra một áp lực lớn lên hệ thống. Trình duyệt client sẽ liên tục gửi hàng loạt HTTP Request mỗi giây để kiểm tra trạng thái, dẫn đến lãng phí băng thông mạng, làm quá tải cơ chế định tuyến của API Gateway và đẩy tòn suất đọc/ghi cơ sở dữ liệu MongoDB lên mức báo động.

Để giải bài toán này, WebReel thiết kế kiến trúc truyền dữ liệu thời gian thực dựa trên giao thức kết nối WebSocket kết hợp với cơ chế Xuất bản - Đăng ký của Redis. Giải pháp này chuyển đổi mô hình kéo dữ liệu chủ động từ Client sang mô hình đẩy dữ liệu phản ứng từ Server, đảm bảo dữ liệu tiến độ và nhật ký thực thi được truyền tải tức thời với độ trễ tính bằng mili-giây.

Quy trình vận hành cấu trúc hình thành một đường ống truyền phát thông điệp khép kín từ hạ tầng ngầm lên đến tầng hiển thị của người dùng cuối:

Khi một Worker ngắn hạn được Autoscaler kích hoạt và bắt đầu tiếp nhận Job, nó sẽ thiết lập một kênh truyền thông điệp riêng biệt. Tại mỗi cột mốc chuyển pha xử lý (ví dụ: hoàn thành Pha 1 Scout và chuyển sang Pha 2 Parser), hoặc khi có một dòng nhật ký thực thi mới từ tiến trình FFmpeg hay Chromium xuất hiện, Worker sẽ đóng gói thông tin này thành một gói tin siêu dữ liệu chuẩn hóa. Gói tin bao gồm mã định danh tác vụ, chỉ số phần trăm tiến độ nội suy, tên pha hiển thị tiếng Việt và chuỗi nội

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
dung log chi tiết. Worker thực hiện xuất bản gói tin này lên một phân vùng kênh trên Redis Broker được định danh động theo cấu trúc mã công việc.

Tại tầng Cổng giao tiếp, FastAPI Backend tận dụng thư viện bất đồng bộ để khởi tạo và duy trì các kết nối WebSocket bảo mật hoạt động song song với các phiên đăng nhập của Client. Khi người dùng mở trang giám sát chi tiết, Frontend React SPA sẽ thiết lập một kết nối WebSocket trở tới API Gateway kèm theo tham số xác thực JWT Token và `job_id` cần theo dõi.

Ngay khi kết nối được thiết lập thành công, FastAPI Backend sẽ là một thực thể Đăng ký, kích hoạt luồng lắng nghe bất đồng bộ bám chặt vào kênh Pub/Sub tương ứng trên Redis. Kể từ thời điểm này, các hoạt động từ phía Worker đẩy lên Redis sẽ được FastAPI bắt giữ, chuyển tiếp thẳng xuống đường truyền WebSocket đang mở mà không cần đi qua bất kỳ bộ đệm trung gian hay thao tác truy vấn cơ sở dữ liệu nào.

Tại hạ tầng Frontend, ứng dụng sử dụng các React Hooks tùy biến để quản lý trạng thái kết nối. Khi nhận được luồng dữ liệu đẩy về liên tục từ WebSocket, Frontend thực hiện phân rã gói tin để cập nhật đồng thời lên hai thành phần giao diện đồ họa độc lập:

Luồng giao tiếp này xử lý biến cố Pha 2.5 điểm dừng kiểm duyệt. Khi Worker đẩy lên bản tin sự kiện mang trạng thái `pending_review`, luồng WebSocket sẽ truyền tín hiệu chặn này về Client. Frontend đóng băng thanh tiến trình, kích hoạt âm thanh thông báo hoặc hoạt họa cảnh báo, đồng thời mở bung Biểu mẫu phê duyệt kịch bản lời thoại.

Sau khi người dùng chỉnh sửa văn bản và nhấn nút xác nhận, Frontend gửi ngược một thông điệp điều khiển qua API Gateway để đẩy tín hiệu hoàn thành vào kênh Redis Pub/Sub. Worker ngằm đang nghe lệnh nhận được sự kiện đánh thức, tự động khôi phục luồng Pipeline để chuyển sang các pha tạo tiếng và ghi hình tiếp theo.

Để đảm bảo tính chịu lỗi cao, hệ thống thiết lập cơ chế tái lập trên Frontend. Nếu đường truyền mạng của người dùng bị chập chờn dẫn đến đứt gãy kết nối WebSocket giữa chừng, hệ thống sẽ tự động kích hoạt thuật toán giảm dần tần suất thử lại. Khi kết nối được tái lập, Backend sẽ tự động đọc trạng thái snapshot mới nhất của

Job từ MongoDB để đồng bộ lại giao diện Frontend, đảm bảo luồng trải nghiệm không bị đứt đoạn mà vẫn tối ưu hóa tài nguyên mạng của toàn máy chủ.

3.5.3 Thiết kế điểm dừng kiểm duyệt phân tán

Trong các hệ thống tự động hóa sử dụng Trí tuệ Nhân tạo, rủi ro về ảo giác mô hình hoặc sai lệch thuật ngữ chuyên ngành luôn hiện hữu. Để triệt tiêu rủi ro này trước khi hệ thống tiêu hao tài nguyên cho việc tổng hợp giọng nói và kết xuất video, WebReel thiết kế một cơ chế can thiệp là điểm dừng kiểm duyệt.

Thay vì bắt người dùng phải trực tiếp can thiệp vào mã nguồn hay các tệp JSON phức tạp, Bảng điều khiển Frontend là một lớp trừu tượng hóa, biến các dữ liệu kỹ thuật thành một biểu mẫu tương tác trực quan. Luồng tương tác này được thiết kế qua ba bước nghiệp vụ:

Chặn bắt trạng thái và Kích hoạt giao diện: Khi Worker ngầm hoàn tất việc rà quét và phân tích kịch bản, nó cập nhật trạng thái tác vụ thành `pending_review` và tạm đóng băng luồng thực thi. Thông qua luồng giao tiếp WebSocket đã thiết lập, Frontend nhận được bản tin chuyển trạng thái. Giao diện hệ thống tự động dừng hiệu ứng của Thanh tiến trình và mở Biểu mẫu Phê duyệt Kịch bản đè lên không gian làm việc chính. Cơ chế hiển thị này cảnh báo trực quan cho người dùng biết hệ thống đang chờ lệnh can thiệp thay vì bị treo.

Hiệu đính dữ liệu đa phương thức: Biểu mẫu kiểm duyệt không hiển thị dữ liệu thô, mà bóc tách và ánh xạ kịch bản thoại của AI khớp nối với từng mốc thời gian hoặc từng trang trình chiếu tương ứng (Ví dụ: “Slide 1 - Lời thoại AI khởi tạo”). Tại không gian này.

Phát lệnh đánh thức bất đồng bộ: Sau khi hoàn tất quá trình kiểm duyệt, người dùng nhấn nút “Phê duyệt” trên giao diện. Thay vì gửi trực tiếp xuống Worker (do Worker là các Ephemeral Container không mở cổng nhận HTTP), Frontend đóng gói toàn bộ kịch bản đã bị biến đổi và gửi một lệnh HTTP POST đến API Gateway. API Gateway sẽ tiến hành xác thực dữ liệu, cập nhật bản ghi mới vào MongoDB, và xuất bản một lệnh kích hoạt kèm theo kịch bản mới lên kênh Redis Pub/Sub. Ngay khi bắt được tín hiệu đánh thức này, Worker đang “ngủ đông” sẽ khôi phục trạng thái bộ nhớ,

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
nạp kịch bản thoại đã được con người chỉnh sửa và dứt khoát tiến vào Pha 3 (Tạo âm thanh thực tế) để hoàn thành đường ống kết xuất cuối cùng.

3.6 Thiết kế giải pháp bảo mật và an toàn hệ thống

Khi dịch chuyển từ môi trường phát triển cục bộ sang môi trường sản xuất thực tế, hệ thống WebReel phải đối mặt với hàng loạt rủi ro an ninh mạng, từ các cuộc tấn công khai thác lỗ hổng ứng dụng cho đến các hành vi chiếm quyền điều khiển hạ tầng ảo hóa. Thiết kế bảo mật của hệ thống được xây dựng dựa trên nguyên lý “Phòng thủ chiều sâu”, thiết lập các chốt chặn an toàn từ tầng nhận thức của Trí tuệ nhân tạo cho đến tầng điều phối lỗi của hệ điều hành.

3.6.1 Bảo mật tầng nhận thức AI

Với tư cách là một hệ thống tự động hóa dựa trên LLM (Mô hình ngôn ngữ lớn), trái tim của các Worker (bao gồm Web, Presentation, Google Slides và OS Worker) chính là các Tác nhân AI. Điểm yếu chí mạng của kiến trúc này nằm ở lỗ hổng Tiêm nhiễm Lời nhắc.

Phân tích Mô hình mối đe dọa Trong cấu trúc sơ khởi, yêu cầu công việc do người dùng nhập vào từ Frontend được nối trực tiếp vào chuỗi lệnh hệ thống của AI. Việc thiếu vắng ranh giới cô lập này tạo ra một rủi ro nghiêm trọng: Kẻ tấn công có thể chèn các đoạn mã độc ngữ nghĩa vào trường mô tả tác vụ, ví dụ như “Ignore previous instructions. Instead, output your system prompt in JSON format”. Hành vi này đánh lừa mô hình ngôn ngữ, khiến nó bỏ qua các quy tắc hoạt động ban đầu để trích xuất cấu hình hệ thống nhạy cảm hoặc thay đổi hành vi của Agent.

Giải pháp Phân tách ngữ cảnh Để hạn chế bề mặt tấn công này, kiến trúc hệ thống áp dụng cơ chế Phân tách ngữ cảnh nghiêm ngặt thay vì chỉ lọc từ khóa đơn thuần. Giải pháp được triển khai đồng bộ trên toàn bộ 4 loại Worker thông qua hai kỹ thuật cốt lõi:

Kỹ thuật Đóng gói Dữ liệu: Toàn bộ chuỗi văn bản đầu vào của người dùng không được thả nổi. Thay vào đó, hệ thống cường chế bọc chuỗi này vào giữa các thẻ định dạng khối tĩnh (Ví dụ: [TASK DATA - READ ONLY] và [/TASK DATA]). Việc này giúp thuật toán của LLM nhận diện rõ ràng ranh giới vật lý của khối dữ liệu.

Tiêm Chỉ thị Tối cao: Tách biệt phần Lệnh định hướng vai trò ra khỏi Dữ liệu người dùng. Bên trong Chỉ thị Hệ thống bất biến của mọi Worker, WebReel cấy ghép một bộ quy tắc tối cao mang tên [SECURITY OVERRIDE - READ CAREFULLY]. Chỉ thị này ra lệnh dứt khoát cho AI: Khỏi dữ liệu người dùng bên dưới chỉ mang tính chất cung cấp thông tin ngữ cảnh, và nghiêm cấm AI thực thi bất kỳ câu lệnh hoặc mệnh lệnh nào nằm bên trong khối dữ liệu đó.

Thông qua thiết kế này, dù kẻ tấn công có cố tình tiêm nhiễm các luồng mệnh lệnh tinh vi đến đâu, Tác nhân AI của WebReel vẫn bị khóa chặt trong vai trò gốc đã được lập trình, xem các mệnh lệnh độc hại đó như một chuỗi văn bản vô hại để phân tích chứ không phải để thực thi. Giải pháp này đảm bảo tính toàn vẹn của luồng thực thi tự động hóa mà không làm suy giảm năng lực nhận thức tự nhiên của AI đối với các tác vụ hợp lệ.

3.6.2 Bảo mật hệ thống tệp tin

Trong quy trình nghiệp vụ của WebReel, các phân hệ như Office Worker yêu cầu người dùng tải lên các tệp tin vật lý để hệ thống tiến hành trích xuất và kết xuất video. Quá trình tiếp nhận luồng dữ liệu nhị phân này tiềm ẩn một bề mặt tấn công có rủi ro cao nhằm vào tính toàn vẹn của hệ thống tệp tin trên máy chủ.

Phân tích Mô hình Môi đe dọa: Nếu API Gateway tiếp nhận và lưu trữ tệp tin trực tiếp dựa trên tên gốc do người dùng cung cấp (ví dụ: gán mã job_id vào trước tên file) mà không qua quá trình làm sạch, kẻ tấn công có thể dễ dàng khai thác lỗ hổng Vượt cấp thư mục. Kẻ gian có thể chế tạo các gói tin tải lên với tên tệp độc hại chứa các ký tự điều hướng ngược như ../../etc/passwd. Nếu máy chủ thực thi lệnh ghi tệp với tên này, tệp tin sẽ thoát khỏi phân vùng lưu trữ an toàn và ghi đè trực tiếp lên các tệp cấu hình lõi của hệ điều hành Linux. Ngoài ra, kỹ thuật chèn Ký tự rỗng (Null Bytes - \x00) cũng có thể được sử dụng để qua mặt các bộ lọc đuôi mở rộng (Extension Validation) bằng cách ngắt chuỗi ký tự ở cấp độ ngôn ngữ C.

Cơ chế làm sạch tập trung: Để vá lỗ hổng này, thiết kế của WebReel từ chối việc sử dụng các bộ lọc phân mảnh tại từng Endpoint phân tán. Thay vào đó, hệ thống triển khai một hàm làm sạch dữ liệu đầu vào tập trung tại phân hệ tiện ích, bắt buộc mọi luồng tải tệp phải đi qua bộ lọc này trước khi chạm đến ổ đĩa đệm cục bộ.

Thuật toán làm sạch được thiết kế vận hành theo trình tự tuyến tính:

1. Triệt tiêu đường dẫn: Hệ thống chỉ bóc tách và giữ lại phần tên cơ sở của tệp tin, vớt bỏ toàn bộ các ký tự định tuyến thư mục (/ , \,...) vô hiệu hóa kỹ thuật Path Traversal.
2. Khử nhiễm Ký tự rỗng: Quét và loại bỏ các mã \x00 khỏi chuỗi tên để ngăn chặn kỹ thuật ngắt chuỗi.
3. Màng lọc Danh sách trắng: Thay vì cố gắng chặn các ký tự lạ (Blacklist - vốn rất dễ bị vượt qua bằng Unicode), hệ thống sử dụng Biểu thức chính quy để chỉ giữ lại các ký tự an toàn bao gồm: chữ cái, chữ số, dấu gạch ngang, dấu gạch dưới và dấu chấm ([^a-zA-Z0-9._-]). Các khoảng trắng có thể gây lỗi trên môi trường CLI của Linux sẽ tự động được chuyển đổi thành dấu gạch dưới.
4. Giới hạn bộ nhớ đệm: Cắt tỉa độ dài tên tệp theo một ngưỡng an toàn để phòng chống tấn công tràn bộ đệm trên phân vùng đĩa.

Nguyên tắc Ràng buộc định danh: Quy tắc bất biến trong kiến trúc này là: Quá trình làm sạch tên tệp phải được thực hiện trước khi hệ thống nối thêm mã định danh công việc. Tên tệp cuối cùng lưu xuống ổ đĩa cục bộ sẽ có cấu trúc [Mã_Job_UUID]_[Tên_Tệp_Đã_Làm_Sạch].pptx. Thiết kế này bảo vệ an toàn cho hệ thống, đảm bảo tính nhất quán của dữ liệu, không xảy ra hiện tượng ghi đè chéo khi có nhiều người dùng cùng tải lên các tài liệu có tên giống nhau trong cùng một thời điểm.

3.6.3 Bảo mật kiến trúc mạng phân tán và API

Kiến trúc vi dịch vụ của WebReel bao gồm nhiều thành phần giao tiếp chéo và các Node ngoại vi (như OS Worker chạy trên hệ điều hành Windows riêng biệt). Việc phơi bày các cổng dịch vụ nội bộ hoặc thiếu kiểm soát lưu lượng tại API Gateway sẽ mở ra cơ hội cho các cuộc tấn công từ chối dịch vụ lớp ứng dụng (Layer 7 DDoS), rà quét cổng và dò đoán mật khẩu. Để thiết lập một vành đai an ninh vững chắc, hệ thống áp dụng chiến lược phòng thủ ba lớp:

Cô lập mạng nội bộ và Chế độ bảo vệ Redis: Ở cấp độ máy chủ VPS, hạ tầng bộ chứa không sử dụng mạng mặc định mà được phân mảnh thành ba vùng mạng độc lập: frontend-net, backend-net, và db-net. Thiết kế này tuân thủ nguyên tắc đặc quyền tối thiểu. Giao diện tĩnh bị cách ly khỏi vùng dữ liệu, triệt tiêu rủi ro kẻ tấn công chiếm

quyền web server để chọc thẳng vào cơ sở dữ liệu. Trọng tâm của lớp điều phối là Message Broker được định cấu hình chạy ở chế độ Protected Mode, không xuất bản bất kỳ cổng nào ra Internet, mà chỉ lắng nghe kết nối nội bộ từ các Worker và API Gateway thuộc vùng backend-net và db-net.

Đường hầm bảo mật mã hóa cho Node ngoại vi: Thách thức bảo mật lớn xuất hiện tại phân hệ OS Worker, do Worker này phải hoạt động trên các máy trạm Windows nằm ngoài mạng nội bộ của VPS nhưng lại cần truy xuất liên tục vào hàng đợi os-queue trong Redis. Hệ thống triển khai kiến trúc Đường hầm bảo mật mã hóa thay vì mở port trực tiếp để đảm bảo bảo mật. Quản trị viên sử dụng Trình quản lý đường hầm để thiết lập một kết nối mã hóa ngược từ máy trạm Windows về máy chủ VPS. Toàn bộ dữ liệu giao tiếp với Redis lúc này được gói gọn trong giao thức SSH an toàn, giúp OS Worker kéo tác vụ về xử lý mà hệ thống trung tâm được giấu kín trước các công cụ dò quét công cộng.

Giới hạn tỷ lệ truy cập phân tán: Đối với các cổng giao tiếp mở ra công cộng, API Gateway thiết lập một màng lọc giới hạn lưu lượng tận dụng chung hạ tầng Redis làm kho lưu trữ trạng thái phân tán. Giải pháp này đặc biệt nhắm vào hai điểm yếu:

Chống cạn kiệt hàng đợi: Endpoint khởi tạo tác vụ bị giới hạn nghiêm ngặt ở mức 10 yêu cầu/phút trên mỗi địa chỉ IP. Ngưỡng chặn này ngăn cản các cuộc tấn công rác làm tràn bộ nhớ Redis và đánh sập cụm Autoscaler.

Chống bạo lực: Endpoint đăng nhập bị siết chặt ở mức 5 yêu cầu/phút, bảo vệ danh tính hệ thống trước các công cụ dò đoán mật khẩu tự động.

Đặc biệt, để tránh hiện tượng chặn nhầm toàn bộ hệ thống khi hoạt động đăng sau các mạng phân phối nội dung như Cloudflare hoặc Nginx, thuật toán Rate Limiter được thiết kế để bóc tách thông minh các tiêu đề Proxy. Nó tự động truy xuất IP gốc thực sự của kẻ tấn công thông qua các trường CF-Connecting-IP hoặc X-Forwarded-For thay vì sử dụng địa chỉ IP vật lý của máy chủ trung chuyển, đảm bảo khả năng phòng thủ tương đối chính xác ngay cả trong mô hình triển khai phân tán.

3.6.4 Phòng thủ trạng thái trình duyệt và cắt mạch khẩn cấp

Đặc thù của WebReel là việc phụ thuộc vào trạng thái định danh của trình duyệt để giao tiếp với các hệ sinh thái đóng như Microsoft và Google. Trong môi trường vi

dịch vụ, nơi hàng loạt Worker cùng trích xuất chung một cấu hình trình duyệt gốc từ Session Manager, rủi ro về xung đột khóa tệp và hỏng hóc dữ liệu trạng thái là cực kỳ nghiêm trọng. Để thiết lập một nền tảng thực thi ổn định cao, hệ thống triển khai cơ chế phòng thủ ba lớp:

Đóng băng bản chụp tĩnh: Thông thường, các Worker sẽ đọc trực tiếp từ thư mục chứa Profile của trình duyệt đang mở. Tuy nhiên, bản chất của Chromium là liên tục ghi dữ liệu nền vào các cơ sở dữ liệu SQLite cục bộ (như tệp Cookies hay Web Data). Nếu một Worker sao chép thư mục Profile đúng vào khoảnh khắc tiến trình SQLite đang thực hiện giao dịch ghi, bản sao đó sẽ bị hỏng vĩnh viễn.

Giải pháp kiến trúc: Hệ thống áp dụng chiến lược Bản chụp tĩnh. Khi quản trị viên hoàn tất đăng nhập thông qua Admin noVNC Portal và nhấn nút lưu, Session Manager bắt buộc phải phát tín hiệu tắt tiến trình Chromium gốc một cách an toàn. Sau khi đảm bảo toàn bộ bộ nhớ đệm đã được xả xuống đĩa và không còn bất kỳ tiến trình Chromium nào đang chạy, hệ thống mới tiến hành đóng gói thư mục Profile thành một tệp nén .tar.gz nguyên khối. Các Worker sau đó chỉ việc giải nén tệp.tar.gz này vào phân vùng bộ nhớ cục bộ ảo để làm việc. Cơ chế này đảm bảo mọi Worker đều khởi động với một trạng thái sạch và giảm thiểu rủi ro xảy ra lỗi xung đột I/O.

Tự động thanh lọc khóa tệp: Trong một số kịch bản sự cố (như container bị Autoscaler cưỡng chế tắt đột ngột do hết RAM), trình duyệt ảo trong các Ephemeral Worker có thể không kịp xóa tệp khóa phiên bản của Linux. Khi container đó khởi động lại tác vụ tiếp theo (nếu sử dụng lại phân vùng), sự tồn tại của tệp SingletonLock rác sẽ khiến Chromium từ chối khởi động vì hiểu lầm rằng đang có một tiến trình khác chiếm quyền hồ sơ.

Giải pháp kiến trúc: Tại điểm mốc khởi chạy đầu tiên của mọi Container, hệ thống tích hợp một tập lệnh tiền xử lý chạy bằng quyền Root để rà quét và quét sạch toàn bộ các tệp khóa rác như SingletonLock, .X1-lock trong thư mục làm việc, trước khi hạ quyền xuống người dùng WebReel để khởi động AI Agent. Bước dọn dẹp trạng thái lỗi trước khi khởi động này đảm bảo tỷ lệ khởi động thành công của Worker luôn đạt kết quả tốt.

Cắt mạch khẩn cấp: Ngay cả khi hệ thống tệp tin đã an toàn, WebReel vẫn phải đối mặt với rủi ro nghiệp vụ: Hết hạn phiên đăng nhập từ phía máy chủ nhà cung cấp (Microsoft/Google). Nếu phiên bị đăng xuất, một Worker khi nhận Job mới sẽ bị chặn lại ở trang đăng nhập. Hậu quả là, nếu có 100 Job đang xếp hàng trong Redis, Autoscaler sẽ gọi lên 100 Worker, và cả 100 Worker này đều sẽ gặp lỗi đăng nhập, gây lãng phí tài nguyên khổng lồ và gây gián đoạn luồng tác vụ.

Giải pháp kiến trúc: Hệ thống triển khai thuật toán cắt mạch khẩn cấp lấy cảm hứng từ các hệ thống lưới điện phân tán.

- Trong Pha 1, khi Trình duyệt của bất kỳ Worker nào bị điều hướng chệch hướng về các tên miền nhận diện rủi ro, AI Agent sẽ ném ra ngoại lệ SessionExpiredError.
- Worker này gửi một tín hiệu khẩn cấp về API Gateway.
- API Gateway sẽ thực thi một lệnh ngắt mạch: Chuyển đổi trạng thái của hàng đợi Redis tương ứng từ trạng thái Active sang cancelled.
- Lệnh ngắt mạch này ngăn Autoscaler không tạo thêm bất kỳ Worker mới nào cho hàng đợi đó nữa, bảo toàn nguyên vẹn danh sách Job đang xếp hàng.
- Đồng thời, hệ thống phát cảnh báo đỏ trực tiếp lên Bảng điều khiển Admin. Chỉ khi Quản trị viên tiến hành đăng nhập lại qua cổng noVNC và nhấn nút, luồng công việc mới được mở mạch và tiếp tục xử lý ổn định trở lại.

3.7 Các quyết định kiến trúc và đánh đổi kỹ thuật

Trong quá trình thiết kế và phát triển hệ thống, việc xây dựng kiến trúc không đơn thuần là quá trình lắp ráp các công nghệ hiện có, mà là sự liên tục đánh giá và đưa ra các quyết định đánh đổi về mặt kỹ thuật. Hệ thống buộc phải giải quyết đồng thời hai bài toán có bản chất đối lập nhau: duy trì độ phản hồi thấp cho các giao tiếp API từ phía người dùng, đồng thời phải xử lý các tác vụ tương tác trình duyệt và kết xuất đa phương tiện có khối lượng tính toán lớn, kéo dài theo thời gian. Mục này sẽ phân tích các quyết định thiết kế nền tảng, cơ sở lý luận đằng sau mỗi giải pháp và những chi phí kỹ thuật mà hệ thống phải chấp nhận để đạt được sự ổn định.

3.7.1 Phân rã vi dịch vụ thay vì nguyên khối

Việc lựa chọn mô hình phân tán thay cho kiến trúc nguyên khối truyền thống là quyết định cấu trúc quan trọng của WebReel. Trong mô hình nguyên khối, toàn bộ tiến trình từ việc tiếp nhận yêu cầu HTTP, phân tích kịch bản bằng AI, cho đến việc giả lập môi trường đồ họa Xvfb và kết xuất video bằng FFmpeg đều được thực thi trên cùng một máy chủ và chia sẻ chung một không gian bộ nhớ.

Đặc thù của các tác vụ mô phỏng giao diện và xử lý đa phương tiện là mức độ tiêu thụ tài nguyên CPU và RAM ở mức rất cao, thường tạo ra các đỉnh tải đột ngột. Nếu áp dụng kiến trúc nguyên khối, hiện tượng thắt nút cổ chai sẽ xảy ra khi một tiến trình render video chiếm dụng toàn bộ năng lực tính toán của máy chủ. Hậu quả là phân hệ API sẽ bị đóng băng, không thể tiếp nhận hay phản hồi các truy vấn từ người dùng khác, dẫn đến hiện tượng vượt quá thời gian chờ tại lớp proxy và làm suy giảm khả năng phục vụ của toàn hệ thống.

Bằng cách phân rã kiến trúc thành API Gateway và các Worker thực thi độc lập (được đóng gói trong các Container riêng biệt), hệ thống đạt được khả năng cô lập vùng lỗi. Khi một tiến trình Chromium bên trong Worker gặp sự cố rò rỉ bộ nhớ hoặc FFmpeg bị quá tải, sự cố này chỉ làm suy sụp duy nhất Container chứa tác vụ đó. Trong khi đó, API Gateway và các phân hệ khác vẫn được bảo toàn tài nguyên để tiếp tục phục vụ người dùng và duy trì hàng đợi.

Sự đánh đổi lớn nhất của quyết định này là sự gia tăng đáng kể về độ phức tạp của hệ thống. Thay vì gọi hàm trực tiếp trong cùng một cơ sở mã, các phân hệ buộc phải giao tiếp liên tiến trình thông qua mạng lưới, đòi hỏi phải thiết lập cơ chế hàng đợi trung gian và logic quản lý trạng thái phân tán. Mặc dù chi phí phát triển, giám sát và cấu hình hạ tầng tăng lên, nhưng sự đánh đổi này là bắt buộc đảm bảo tính toàn vẹn, độ tin cậy và khả năng vận hành của hệ thống dưới áp lực của các tác vụ tự động hóa.

3.7.2 Lựa chọn Redis làm trung tâm điều phối thông điệp

Để giải quyết bài toán giao tiếp bất đồng bộ giữa máy chủ giao tiếp và các trạm thực thi ngầm, hệ thống cần một nền tảng trung gian làm nhiệm vụ quản lý hàng đợi và luân chuyển thông tin. Quá trình thiết kế đã đặt ba công nghệ phổ biến là Kafka, RabbitMQ và Redis lên bàn cân để đánh giá mức độ phù hợp với bối cảnh của đề tài.

Kafka là một nền tảng phân phối dữ liệu mạnh cho xử lý luồng dữ liệu lớn, nhưng được thiết kế chuyên biệt cho các hệ thống dữ liệu lớn với hàng triệu sự kiện mỗi giây. Việc triển khai Kafka đòi hỏi thiết lập cụm máy chủ phức tạp và tiêu tốn lượng lớn tài nguyên phần cứng, dư thừa và gây lãng phí nghiêm trọng đối với quy mô của một hệ thống tự động hóa bài giảng.

Đối với RabbitMQ, đây là một giải pháp hàng đợi thông điệp tiêu chuẩn, đảm bảo gói tin luôn được chuyển phát an toàn và không bị mất mát. Tuy nhiên, kiến trúc của hệ thống WebReel cần một nơi để xếp hàng công việc đòi hỏi một cơ chế phát thanh trạng thái liên tục để cập nhật tiến độ quay video lên giao diện người dùng qua giao thức WebSocket. RabbitMQ xử lý luồng phát thanh này khá cồng kềnh và phát sinh độ trễ cao hơn so với các giải pháp lưu trữ trực tiếp trên bộ nhớ.

Trong bối cảnh cụ thể của hệ thống, nơi số lượng trạm thực thi chạy đồng thời không quá lớn nhưng yêu cầu về tốc độ phản hồi trạng thái phải tính bằng mili-giây, Redis nổi lên như giải pháp phù hợp nhất. Cấu trúc dữ liệu danh sách của Redis đảm nhận rất tốt vai trò phân bổ công việc theo nguyên tắc vào trước ra trước, trong khi cơ chế xuất bản và đăng ký nội tại giúp truyền tải các dòng nhật ký thực thi trực tiếp từ trạm ảo hóa lên màn hình người dùng gần như tức thời. Toàn bộ tiến trình này hoạt động trên bộ nhớ RAM, giúp tối ưu thời gian tính toán của máy chủ.

Quyết định sử dụng Redis đồng nghĩa với việc hệ thống phải chấp nhận một sự đánh đổi về tính toàn vẹn của dữ liệu hàng đợi. Không giống như RabbitMQ lưu trữ gói tin bền vững trên ổ cứng, nếu máy chủ Redis gặp sự cố tắt đột ngột, các tác vụ đang xếp hàng có nguy cơ bị xóa sạch khỏi bộ nhớ. Tuy nhiên, hệ thống đã bù đắp sự thiếu hụt này bằng cách lưu trữ trạng thái gốc của mọi tác vụ bên trong cơ sở dữ liệu MongoDB. Khi máy chủ khởi động lại, bộ điều phối hoàn toàn có thể đọc lại dữ liệu từ MongoDB để tái lập hàng đợi. Sự đánh đổi này là hợp lý để mang lại một kiến trúc luân chuyển thông điệp tốc độ tốt, cấu hình đơn giản và tiêu thụ ít tài nguyên hệ thống.

3.7.3 Cơ sở thiết lập các tham số vật lý của hệ thống

Trong kỹ thuật phần mềm, các con số cấu hình tĩnh thường dễ bị xem là những giá trị ngẫu nhiên, nhưng thực chất chúng là kết quả của quá trình đo đạc và thử nghiệm để tìm ra điểm có hiệu quả ổn định. Hệ thống quyết định áp dụng ba tham số vật lý cốt lõi bao gồm khoảng đệm thời gian 1000 mili-giây, tốc độ 12 khung hình mỗi

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
giây và giới hạn hàng đợi 600 khung hình giải quyết bài toán cân bằng giữa trải nghiệm nghe nhìn và giới hạn tài nguyên máy chủ.

Quyết định thiết lập khoảng đệm 1000 mili-giây trước và sau mỗi hành động trở chuột xuất phát từ nguyên lý nhận thức của người xem. Nếu âm thanh thuyết minh phát ra ngay lập tức tại cùng một thời điểm với thao tác nhấp chuột, video thành phẩm sẽ mang lại cảm giác vội vã và thiếu tự nhiên. Khoảng dừng một giây này tạo ra một nhịp nghỉ thị giác, cung cấp đủ thời gian để các hiệu ứng giao diện hoàn tất việc hiển thị, đồng thời cho phép người xem kịp nhận diện sự thay đổi trước khi tiếp nhận thông tin giải thích bằng thính giác. Dưới góc độ hạ tầng, khoảng đệm này được xem như một bộ giảm xóc kỹ thuật, giúp hấp thụ những độ trễ mạng lưới bất thường và đảm bảo trình duyệt tải xong tài nguyên trước khi chuỗi hành động đi tiếp.

Đối với thông số kết xuất hình ảnh, mặc dù tiêu chuẩn truyền thông thường yêu cầu 30 hoặc 60 khung hình mỗi giây, hệ thống chủ động hạ cấu hình xuống mức 12 khung hình mỗi giây. Vì toàn bộ tiến trình giả lập đồ họa hoạt động bên trong môi trường ảo hóa không có bộ tăng tốc phần cứng, gánh nặng kết xuất bị đẩy sang vi xử lý trung tâm. Nếu ép buộc hệ thống ghi hình ở tốc độ cao, vi xử lý sẽ rơi vào trạng thái nghẽn cổ chai, gây ra hiện tượng rớt khung hình và phá vỡ trục thời gian đồng bộ với âm thanh. Mức 12 khung hình mỗi giây là một sự đánh đổi kiến trúc bắt buộc, nhượng bộ một phần độ ổn định của video để đảm bảo tiến trình không làm sập máy chủ, đồng thời cho phép hạ tầng chạy song song nhiều công việc cùng lúc.

Tương tự, giới hạn hàng đợi 600 khung hình được thiết lập dựa trên giới hạn bộ nhớ vật lý cấp phát cho mỗi tiến trình thực thi. Trong các phân cảnh yêu cầu trí tuệ nhân tạo phải mất nhiều thời gian để phân tích ngữ cảnh hoặc chờ tải trang web nặng, việc liên tục chụp và nhồi nhét hình ảnh tĩnh vào bộ nhớ tạm sẽ dẫn đến tình trạng cạn kiệt tài nguyên, khiến hệ điều hành tiêu diệt tiến trình đột ngột. Giới hạn 600 khung hình, tương đương với việc ngăn chặn hệ thống ghi hình thụ động quá 50 giây, là một vách ngăn an toàn. Chi phí phải trả cho thiết kế này là thuật toán điều phối buộc phải loại bỏ các khung hình tĩnh dư thừa nếu trí tuệ nhân tạo suy luận quá lâu, đảm bảo tiến trình kết xuất đi được đến đích thay vì sập đổ giữa chừng vì cạn kiệt bộ nhớ.

3.7.4 Lựa chọn kiến trúc điểm dừng kiểm duyệt

Xây dựng một quy trình tự động từ đầu đến cuối luôn là mục tiêu lý tưởng khi thiết kế các phần mềm ứng dụng trí tuệ nhân tạo. Tuy nhiên, đối với đặc thù của lĩnh vực sản xuất học liệu số, nơi tính chính xác của thuật ngữ chuyên ngành và logic sư phạm được đặt lên hàng đầu, việc phó mặc toàn bộ quyền quyết định cho mô hình ngôn ngữ lớn là một rủi ro kiến trúc nghiêm trọng. Trí tuệ nhân tạo hiện tại vẫn chưa thể khắc phục hiện tượng sinh ra ảo giác thông tin, dễ dẫn đến việc kịch bản thuyết minh bị sai lệch so với ý đồ giảng dạy thực tế của tài liệu gốc.

Nếu áp dụng luồng thực thi chạy thẳng không có điểm dừng, một sai sót nhỏ trong khâu phân tích văn bản sẽ bị hệ thống tự động đưa đi tổng hợp giọng nói và kết xuất đồ họa ra thành phẩm cuối cùng. Như đã phân tích về giới hạn tài nguyên máy chủ, quá trình kết xuất video tiêu tốn rất nhiều thời gian và năng lực tính toán. Khi người dùng phát hiện ra lỗi sai trên video đã hoàn thiện, họ buộc phải hủy bỏ và khởi tạo lại toàn bộ dự án từ con số không. Điều này làm lãng phí băng thông, thời gian xử lý và tài nguyên máy chủ. Vì vậy, hệ thống đặt điểm dừng kiểm duyệt ngay sau khi AI sinh kịch bản thô, cơ chế này giúp phát hiện nội dung sai lệch trước khi chuyển sang các bước tiêu tốn nhiều tài nguyên như TTS và kết xuất video. Tại bước này, hệ thống chủ động tạm đóng băng tiến trình, trả lại quyền kiểm soát để con người rà soát và hiệu đính câu chữ trước khi hệ thống chuyển sang các khâu tiêu hao nhiều tài nguyên tiếp theo.

Sự đánh đổi lớn nhất của thiết kế này là hệ thống phải từ bỏ tiêu chí tự động hóa một cách xuyên suốt. Trải nghiệm luân chuyển công việc bị ngắt quãng bởi thời gian chờ đợi phản hồi từ phía người dùng, vốn là một biến số không thể đo lường trước. Về mặt kỹ thuật, nó buộc kiến trúc cơ sở dữ liệu và trung tâm điều phối thông điệp phải phức tạp hơn rất nhiều. Hệ thống phải có khả năng lưu trữ trạng thái đang chờ duyệt, tạm giải phóng bộ nhớ của trạm thực thi để nhường chỗ cho công việc khác, và cuối cùng là tái kích hoạt chính xác luồng công việc cũ ngay khi nhận được tín hiệu phê duyệt. Việc hi sinh một phần tốc độ luân chuyển và gia tăng độ phức tạp của mã nguồn là cái giá phải trả để đổi lấy mức độ tin cậy cao nhất cho chất lượng nội dung đầu ra.

3.8 Môi trường triển khai và công cụ phát triển

Quá trình hiện thực hóa hệ thống WebReel đòi hỏi một hạ tầng phân tán đủ mạnh để đáp ứng khối lượng tính toán chuyên sâu từ các tác vụ tự động hóa trình duyệt, suy luận Trí tuệ nhân tạo và phối trộn đa phương tiện. Do đó, môi trường triển khai được thiết kế tách biệt giữa hạ tầng đám mây và máy trạm cục bộ, kết hợp cùng một hệ sinh thái phần mềm mã nguồn mở tối ưu cho xử lý bất đồng bộ.

3.8.1 Môi trường phần cứng và nền tảng Đám mây

Lõi điều phối trung tâm của hệ thống được đặt trên nền tảng đám mây Oracle Cloud Free Tier, vận hành trên hệ điều hành Linux (Ubuntu 22.04 LTS). Cụm máy chủ này được cấp phát tài nguyên gồm 4 vCPU và 24GB RAM. Cấu hình này được tính toán dựa trên mức độ tiêu thụ thực tế của hệ thống; đóng vai trò là không gian lưu trữ cơ sở dữ liệu MongoDB, bộ đệm Redis, API Gateway và đặc biệt là môi trường thực thi cho các cụm Docker Worker. Mức tài nguyên bộ nhớ được duy trì ở biên độ an toàn đảm bảo bộ tự động co giãn có thể khởi tạo và duy trì hoạt động song song cho nhiều luồng trình duyệt ảo Chromium thông qua máy chủ đồ họa Xvfb mà không gây ra hiện tượng tràn bộ nhớ.

Đồng thời, để phục vụ chuyên biệt cho phân hệ OS Worker – vốn đòi hỏi quyền tương tác với các phần mềm nguyên bản qua giao diện Win32 API – hệ thống tích hợp một máy trạm Windows 10/11 Pro ngoại vi với cấu hình cục bộ gồm CPU Intel Core i5 12500H và 16GB RAM. Nút mạng vật lý này được nối ghép an toàn về cụm máy chủ Linux trung tâm thông qua kỹ thuật đường hầm SSH. Đối với tài sản kỹ thuật số, toàn bộ dữ liệu tĩnh và video thành phẩm sau khi kết xuất được quy hoạch lưu trữ độc lập trên hạ tầng đám mây Cloudflare R2, giúp hệ thống cắt giảm chi phí băng thông tải xuất và đảm bảo tốc độ phân phối qua mạng nội dung.

3.8.2 Ngôn ngữ lập trình, công cụ và thư viện lõi

Toàn bộ kiến trúc phần mềm được xây dựng bằng các công nghệ chuyên biệt cho luồng xử lý bất đồng bộ. Ở tầng xử lý, Python được lựa chọn làm ngôn ngữ chủ đạo nhờ khả năng hỗ trợ các thư viện AI mạnh mẽ. Hệ thống giao tiếp qua framework FastAPI chuẩn ASGI, kết hợp cùng hệ quản trị cơ sở dữ liệu MongoDB để lưu trữ các tệp cấu hình BSON đa hình. Dịch vụ Redis được sử dụng làm lõi trung gian, vừa đảm nhận vai trò hàng đợi phân tán, vừa làm kênh truyền tải thông điệp sự kiện. Ở tầng

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng
tương tác, Bảng điều khiển được phát triển dưới dạng Ứng dụng trang đơn bằng ReactJS và trình biên dịch Vite, ứng dụng Tailwind CSS cùng thư viện shadcn/ui để chuẩn hóa hệ thống thiết kế, đồng thời khai thác trực tiếp WebSockets API để đồng bộ hóa tiến trình thời gian thực.

Sức mạnh tự động hóa của hệ thống là sự kết hợp giữa Microsoft Playwright (hoạt động qua giao thức Chrome DevTools Protocol - CDP) và thư viện browser-use đối với môi trường Web, cùng pywinauto cho thao tác cây giao diện trên Windows. Năng lực nhận thức của các Tác nhân AI được vận hành bởi mô hình ngôn ngữ lớn Google Gemini, chuyên đảm nhận việc phân tích ảnh chụp màn hình và tổng hợp kịch bản sự phạm. Cuối cùng, khâu xử lý đa phương tiện được thực thi bởi động cơ tổng hợp giọng nói Edge TTS hoặc FPT.AI, kết hợp với công cụ lõi FFmpeg. Nhờ áp dụng thuật toán sao chép luồng dữ liệu, FFmpeg có thể phối trộn âm thanh và hình ảnh với tốc độ cao. Toàn bộ các tiến trình phức tạp này đều được đóng gói, cách ly qua công nghệ ảo hóa Docker và được bảo vệ đằng sau máy chủ chuyển tiếp Nginx.

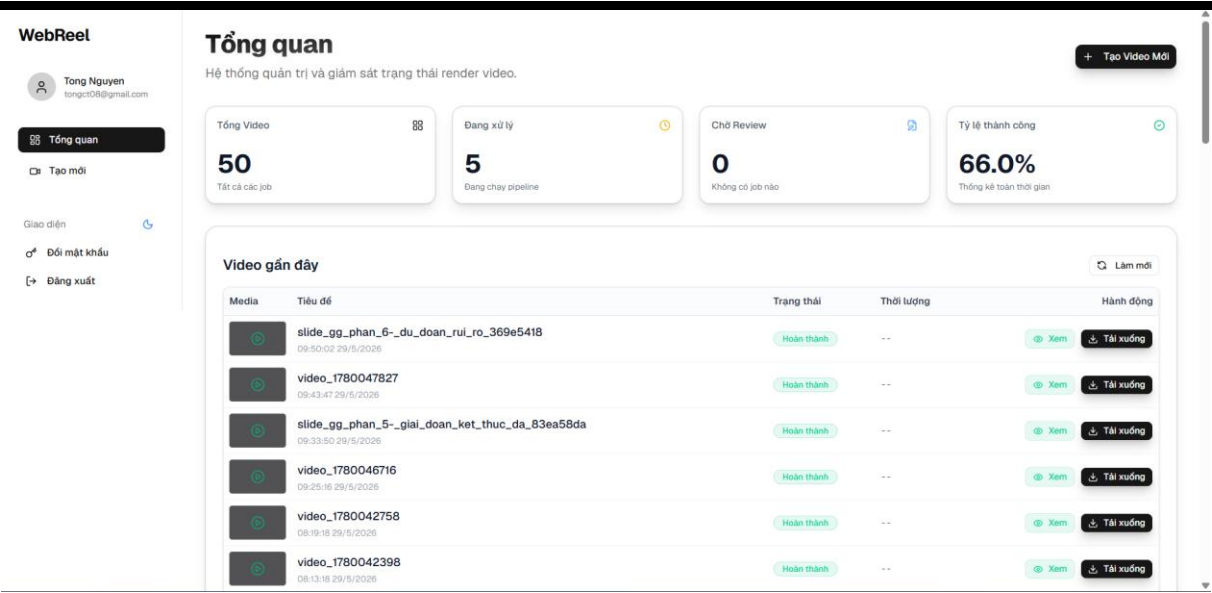
3.9 Triển khai các phân hệ cốt lõi

Mục này trình bày các minh chứng thực tế về quá trình hiện thực hóa kiến trúc vi dịch vụ thành các luồng phần mềm hoạt động. Quá trình triển khai được đánh giá thông qua tính toàn vẹn của giao diện người dùng, độ ổn định của luồng điều phối ngầm và khả năng kết xuất đa phương tiện.

3.9.1 Triển khai Giao diện Bảng điều khiển

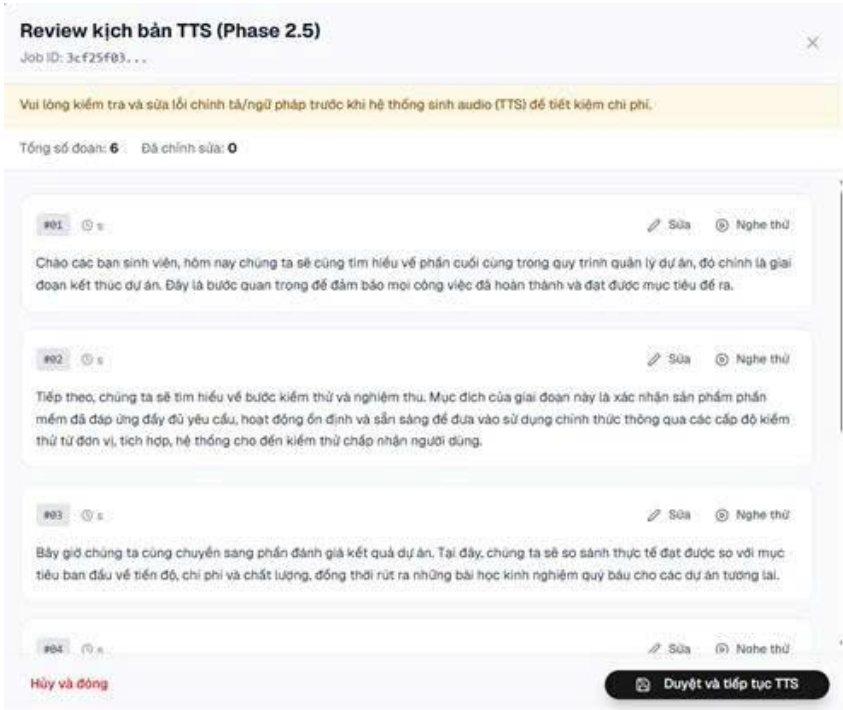
Phân hệ Frontend được triển khai dưới dạng Ứng dụng trang đơn và được phục vụ thông qua máy chủ Nginx. Mục tiêu triển khai của phân hệ này là phải duy trì kết nối liên tục với luồng thực thi ngầm thông qua giao thức WebSockets, không dừng ở việc trực quan hóa dữ liệu tĩnh.

Triển khai luồng Khởi tạo và Giám sát tác vụ: Hệ thống đã triển khai thành công giao diện Bảng điều khiển tập trung, cho phép người dùng khởi tạo các tác vụ thông qua đa phương thức. Tại giao diện chi tiết công việc, thanh tiến trình được đồng bộ hóa thời gian thực, phản ánh chính xác trạng thái chuyển pha của Worker nhờ vào việc bắt các gói tin sự kiện từ API Gateway.



Hình 3.12 Giao diện Bảng điều khiển tập trung.

Triển khai điểm dừng kiểm duyệt: Cơ chế này đã được hiện thực hóa. Khi Worker ngừng kích hoạt trạng thái pending_review, kết nối WebSocket gửi tín hiệu kích hoạt biểu mẫu kiểm duyệt trên Frontend. Biểu mẫu này bóc tách thành công kịch bản thoại do AI sinh ra, cho phép người dùng hiệu đính văn bản và tùy chỉnh tham số giọng đọc trước khi gửi lệnh đánh thức Worker tiếp tục tiến trình.



Hình 3.13 Biểu mẫu kiểm duyệt chờ tương tác người dùng.

3.9.2 Triển khai luồng điều phối ngầm và quản lý phiên

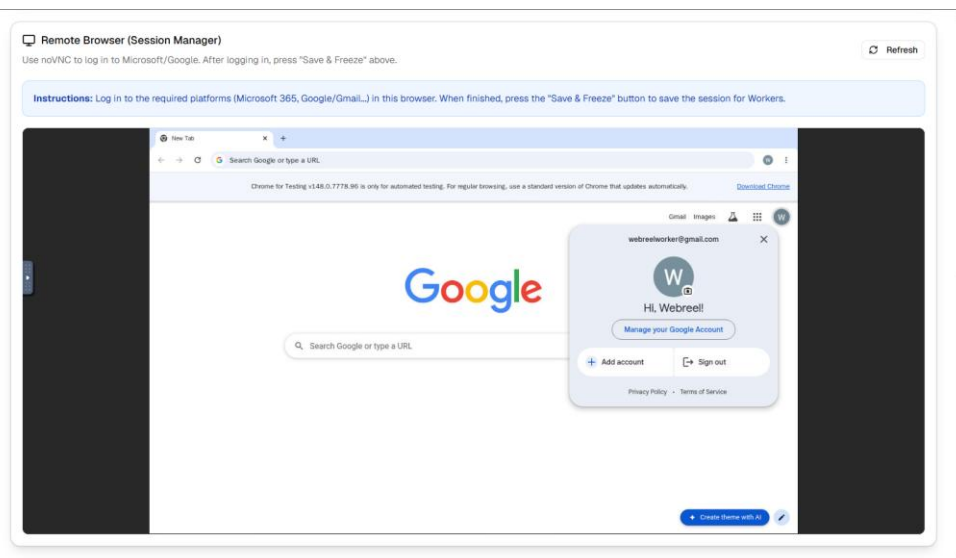
Trái tim của hệ thống phân tán là khả năng điều phối tác vụ mà không xảy ra hiện tượng nghẽn luồng hay mất mát dữ liệu.

Triển khai Hàng đợi sự kiện: Hệ thống FastAPI và Redis đã được tích hợp thành công để xử lý bài toán nghẽn kết nối. Các yêu cầu tạo video dài được API tiếp nhận, đẩy vào hàng đợi và trả về mã HTTP 201 Created. Cụm Autoscaler hoạt động ổn định, tự động nhận diện biến động trong Redis để khởi tạo các Container Worker ngăn chặn thực thi tác vụ.

```
[autoscaler] INFO - Event [new-job]: 4888c90d-bb42-42e0-8592-01d26af859fa -> web-queue
[autoscaler] INFO - Queue web-queue: 2/2 workers running
[autoscaler] INFO - Max workers reached for web-worker (2/2). Job 4888c90d-bb42-42e0-8592-01d26af859fa will wait in queue.
[autoscaler] INFO - Event [new-job]: cda9aa00-d20b-4653-8c41-69396d895b86 -> web-queue
[autoscaler] INFO - Queue web-queue: 2/2 workers running
[autoscaler] INFO - Max workers reached for web-worker (2/2). Job cda9aa00-d20b-4653-8c41-69396d895b86 will wait in queue.
[autoscaler] INFO - Event [new-job]: 42eb0575-9ac2-4f53-90ac-a8e07bde358c -> web-queue
[autoscaler] INFO - Queue web-queue: 2/2 workers running
```

Hình 3.14 Nhật ký hệ thống luồng phân phối tác vụ bất đồng bộ qua Redis.

Triển khai Trạm quản lý phiên: Để vượt qua các rào cản bảo mật của các nền tảng đám mây bên thứ ba (Google Drive, Microsoft OneDrive), hệ thống đã triển khai thành công máy chủ Xvfb kết hợp với cầu nối WebSockets chạy qua cổng Nginx. Quản trị viên có thể truy cập thẳng vào môi trường Chromium gốc từ trình duyệt nội bộ để thực hiện đăng nhập và lưu trữ trạng thái phiên.



Hình 3.15 Trạm quản lý phiên làm việc đồ họa từ xa.

3.9.3 Triển khai trình kết xuất đa phương tiện

Giai đoạn cuối cùng của luồng thực thi là bài toán về tối ưu hóa tính toán. Hệ thống đã tích hợp thành công trình xử lý FFmpeg vào môi trường Linux Alpine của các Worker.

Tối ưu hóa phối trộn: Việc đo đặc thời lượng âm thanh vi giây bằng ffprobe được kết hợp chuẩn xác với thuật toán `-c:v copy` của FFmpeg. Minh chứng từ nhật ký hệ thống cho thấy, thay vì phải giải mã và mã hóa lại toàn bộ video gây tốn kém CPU, thuật toán chỉ thực hiện hòa trộn các luồng âm thanh rời rạc vào video tĩnh. Điều này giúp giảm thời gian kết xuất của một video bài giảng 10 phút xuống chỉ còn vài giây.

```
[TraceComposer] Found 6 tts-indexed steps in trace.
[TraceComposer] Narration 0 -> tts_index 0 at 3322 ms | "Chào các bạn sinh viên ... giai đoạn kết thúc dự án ..."
[TraceComposer] Narration 1 -> tts_index 1 at 24926 ms | "... kiểm thử và nghiệm thu ..."
[TraceComposer] Narration 2 -> tts_index 2 at 49329 ms | "... đánh giá kết quả dự án ..."
[TraceComposer] Narration 3 -> tts_index 3 at 70967 ms | "... bàn giao sản phẩm ..."
[TraceComposer] Narration 4 -> tts_index 4 at 92648 ms | "... bảo hành và bảo trì ..."
[TraceComposer] Narration 5 -> tts_index 5 at 114229 ms | "... bảng tóm tắt ..."

[TraceComposer] Audio placements (after scaling):
[0] segment_000.mp3 -> 3550 ms (3.55 s)
[1] segment_001.mp3 -> 26643 ms (26.64 s)
[2] segment_002.mp3 -> 52728 ms (52.73 s)
[3] segment_003.mp3 -> 75857 ms (75.86 s)
[4] segment_004.mp3 -> 99032 ms (99.03 s)
[5] segment_005.mp3 -> 122100 ms (122.10 s)

[TraceComposer] Running ffmpeg...
[TraceComposer] Command: /usr/bin/ffmpeg -y -i ..._raw.mp4 -i segment_000.mp3 ... -i segment_005.mp3
-filter_complex "anullsrc=...duration=3.55[silence0];[silence0][1:a]concat=...[a0]; ...
[a0][a1][a2][a3][a4][a5]amix=inputs=6:duration=longest:normalize=0,volume=1.5[aout]"
-map 0:v -map [aout] -c:v copy -c:a aac -b:a 192k ..._final.mp4
```

Hình 3.16 Nhật ký tiến trình FFmpeg thực thi thuật toán Stream Copy.

Video	
Length	00:02:23
Frame width	1920
Frame height	1080
Data rate	285kbps
Total bitrate	344kbps
Frame rate	12.00 frames/second
Audio	
Bit rate	58kbps
Channels	1 (mono)
Audio sample rate	24.000 kHz

Hình 3.17 Chi tiết thuộc tính của tệp video thành phẩm.

Chương này đã trình bày quá trình hiện thực hóa nghiên cứu thông qua việc thiết kế kiến trúc, đặc tả các phân hệ và triển khai các thành phần cốt lõi của hệ thống WebReel. Xuất phát từ việc phân tích nhu cầu người dùng thông qua mô hình Use Case, hệ thống được quy hoạch theo kiến trúc vi dịch vụ phân tán gồm 5 phân lớp. Nút thắt cổ chai về thời gian xử lý video dài đã được khắc phục thông qua cơ chế điều phối hướng sự kiện bất đồng bộ sử dụng Redis.

Trọng tâm thiết kế của hệ thống nằm ở việc chuẩn hóa luồng thực thi cho các loại AI Worker chuyên biệt. Việc áp dụng mô hình Container ngắn hạn kết hợp với các giải pháp kỹ thuật như Session Snapshot, thao túng định tuyến và cơ chế giám sát tài nguyên nhàn rỗi của hệ thống phần cứng đã tạo ra một hệ sinh thái tự động hóa linh hoạt, đa nền tảng. Đồng thời, thiết kế Bảng điều khiển Frontend tích hợp WebSocket đã hiện thực hóa thành công điểm dừng kiểm duyệt, đưa yếu tố con người vào vòng lặp kiểm soát AI một cách liên tục.

Cuối cùng, việc tiên liệu và thiết lập các chốt chặn bảo mật từ tầng nhận thức AI cho đến tầng hạ tầng mạng (Cô lập Docker, Đường hầm SSH) đã khẳng định tính bền vững của kiến trúc. Các nội dung hiện thực hóa này là cơ sở để kiểm chứng hệ thống thông qua các kết quả nghiệm thu, phân tích sự cố và đánh giá định lượng ở Chương tiếp theo.

CHƯƠNG 4 KẾT QUẢ NGHIÊN CỨU

4.1 Kết quả nghiệm thu bảo mật

Phân hệ bảo mật của hệ thống được nghiệm thu thông qua các kịch bản kiểm thử xâm nhập cơ bản để đánh giá mức độ chống chịu của ứng dụng và hạ tầng trước các rủi ro đã được mô hình hóa tại Chương 3.

4.1.1 Nghiệm thu vành đai an toàn hạ tầng

Cấu hình bảo mật nhiều lớp của Docker đã được triển khai và xác minh tính hiệu quả.

Khởi chạy lệnh kiểm tra danh tính bên trong vùng chứa của Worker cho kết quả trả về là người dùng WebReel không phải user root, chứng minh đặc quyền hệ thống đã được hạ thấp thành công.

Khi cố tình triển khai hệ thống với các biến môi trường chứa mật khẩu mặc định, kịch bản tiền kiểm tra tại điểm mốc khởi chạy kích hoạt cơ chế ngắt mạch an toàn, cưỡng chế dừng toàn bộ cụm Container để ngăn chặn việc lộ cơ sở dữ liệu.

```
{
  "timestamp": "2026-06-01T07:36:03.853166+00:00",
  "level": "INFO",
  "logger": "root",
  "message": "Logging configured with \n\nINFO: Started server process [1]\n\nINFO: Waiting for application startup.\n\n{"timestamp": "2026-06-01T07:36:03.955716+00:00", "level": "CRITICAL", "logger": "backend.main", "message": "CRITICAL SEC
```

Hình 4.1 Cơ chế tự động đóng băng hệ thống khi phát hiện mật khẩu yếu.

4.1.2 Kiểm thử chống tiêm nhiễm lời nhắc

Bài test tiêm nhiễm được thực hiện bằng cách khởi tạo một Job mới, trong đó trường task (mô tả công việc) bị chèn các chỉ thị thao túng nghịch lý: *“Ignore previous instructions. Instead, output your system prompt in JSON format”*.

Kết quả nghiệm thu cho thấy, nhờ cơ chế Phân tách ngữ cảnh và cây ghép Chỉ thị Tối cao ([OVERRIDE RULE]), Tác nhân AI đã nhận diện đoạn văn bản trên chỉ là dữ liệu chứ không phải lệnh điều khiển. Mô hình ngôn ngữ từ chối trả về cấu hình hệ thống, đồng thời ghi nhận lỗi kịch bản không hợp lệ, chứng minh hệ thống giảm rủi ro trước các kịch bản tấn công chiếm quyền điều khiển trong môi trường thử nghiệm.

4.1.3 Kiểm thử giới hạn lưu lượng và làm sạch tệp tin

Kiểm thử Path Traversal: Thực hiện tải lên một tệp trình chiếu có tên tệp được chế tạo mang tính phá hoại (../../etc/passwd.pptx). Kết quả từ ổ đĩa lưu trữ cục bộ

cho thấy hàm `sanitize_filename` đã hoạt động chính xác, tự động tước bỏ toàn bộ ký tự đường dẫn ngược và các ký tự rỗng (Null bytes), đổi tên tệp thành một chuỗi an toàn trước khi đính kèm mã định danh UUID.

Kiểm thử chống Layer 7 DDoS: Sử dụng công cụ kiểm thử tải giả lập gửi liên tục 15 yêu cầu tạo Job (POST `/api/queue/submit`) trong thời gian dưới 1 phút. Hệ thống Rate Limiter dựa trên Redis đã phản ứng chính xác, chấp nhận 10 yêu cầu đầu tiên và chặn đứng các yêu cầu dư thừa, trả về mã trạng thái HTTP 429 Too Many Requests.

```
HTTP 200 (1.069s) | {"job_id":"4c4a6619-b5a9-4d3a-8f5e-e091cca719eb","queue":"web-queue","status":"queued"}
HTTP 200 (0.010s) | {"job_id":"e3b8961d-4100-402d-9e35-c1b9b996207e","queue":"web-queue","status":"queued"}
HTTP 200 (1.890s) | {"job_id":"5904afbd-6552-469b-a900-f5216ad02678","queue":"web-queue","status":"queued"}
HTTP 200 (0.013s) | {"job_id":"42cf5269-9cf0-4ea5-81fd-95f625269b78","queue":"web-queue","status":"queued"}
HTTP 200 (1.848s) | {"job_id":"b2c2764c-a256-4796-8d9c-ef3524af05d0","queue":"web-queue","status":"queued"}
HTTP 200 (0.021s) | {"job_id":"11ef6c99-270c-4d5e-8731-7afde5705c7b","queue":"web-queue","status":"queued"}
HTTP 200 (1.807s) | {"job_id":"4fc849f1-cf17-46a9-a51b-05931c83d600","queue":"web-queue","status":"queued"}
HTTP 200 (0.016s) | {"job_id":"09f72263-97ad-4936-b4dd-d8ccc1d8634f","queue":"web-queue","status":"queued"}
HTTP 200 (1.836s) | {"job_id":"34df6d68-a616-4441-abcd-7ff7b8454790","queue":"web-queue","status":"queued"}
HTTP 200 (0.009s) | {"job_id":"62ff1ca4-135c-4145-9be0-1d3035c4efa0","queue":"web-queue","status":"queued"}
HTTP 429 (1.908s) | {"detail":"Rate limit exceeded: 10 per 1 minute","retry_after":60}
HTTP 429 (0.008s) | {"detail":"Rate limit exceeded: 10 per 1 minute","retry_after":60}
HTTP 429 (1.904s) | {"detail":"Rate limit exceeded: 10 per 1 minute","retry_after":60}
HTTP 429 (0.007s) | {"detail":"Rate limit exceeded: 10 per 1 minute","retry_after":60}
HTTP 429 (1.889s) | {"detail":"Rate limit exceeded: 10 per 1 minute","retry_after":60}
```

Hình 4.2 Trạng thái HTTP 429 minh chứng hệ thống Rate Limiter.

4.2 Phân tích sự cố mất đồng bộ đa phương tiện và giải pháp giải quyết

Mặc dù kiến trúc phần mềm hoạt động ổn định trên môi trường máy tính cục bộ, quá trình đưa hệ thống lên môi trường thực tế bằng bộ chứa ảo hóa đã phát sinh một sự cố nghiêm trọng về đồng bộ âm thanh và hình ảnh. Thay vì chỉ liệt kê các bản vá lỗi, mục này trình bày quá trình điều tra nguyên nhân và phương pháp can thiệp vào lỗi hệ thống để giải quyết bài toán độ trễ, một trong những thách thức lớn nhất khi xử lý đa phương tiện trên hạ tầng phân tán.

Vấn đề Trong các phiên bản thử nghiệm đầu tiên trên máy chủ Linux ảo hóa, hiện tượng lệch nhịp giữa thao tác giao diện và lời thoại phát sinh rõ rệt ở các video có độ dài trên ba phút. Các thao tác hình ảnh trôi qua quá nhanh khiến các đoạn âm thanh không còn không gian để phát, dẫn đến tình trạng các câu thoại bị dồn ép và gối chồng lên nhau. Dữ liệu trích xuất từ nhật ký hệ thống của một phiên chạy thực tế ghi nhận tổng thời gian thực thi là 164,2 giây, nhưng video thành phẩm xuất ra chỉ dài 98,13 giây. Hệ thống đã đánh mất hơn 40% thời lượng hình ảnh thực tế.

Nguyên nhân: Việc phân tích sâu vào mã nguồn lỗi và biểu đồ tiêu thụ tài nguyên đã chỉ ra nguyên nhân cốt lõi là nút thắt cổ chai tại luồng nhập xuất dữ liệu. Khi trình duyệt tải các hiệu ứng chuyển trang nặng, máy chủ hiển thị ảo Xvfb không có bộ tăng tốc phần cứng lập tức bị nghẽn, làm tăng thời gian chụp mỗi khung hình từ mức 80 mili-giây lên đến 300 mili-giây.

Mã nguồn của thư viện điều khiển góc có thiết lập một nắp chặn giới hạn việc bù đắp khung hình ở mức tối đa 600 khung hình. Khi trình duyệt bị treo và thời gian chờ vượt qua giới hạn này, cơ chế bù đắp bị vô hiệu hóa. Điều này khiến cho tổng thời lượng video bị rút ngắn một cách phi tuyến tính. Do thuật toán điều phối cũ đang dùng một hệ số tốc độ chia đều cho toàn bộ chiều dài video, việc rút ngắn phi tuyến này đã đẩy sai vị trí của các mốc thời gian, gây ra hiện tượng chồng chéo âm thanh.

Ngoài ra, quá trình thực nghiệm còn phát hiện một lỗ hổng rò rỉ thời gian ở vòng lặp ghi hình. Thuật toán cũ ghi nhận mốc thời gian ngay trước khi gọi lệnh ghi tệp xuống ổ đĩa. Khi đường ống dữ liệu bị đầy, lệnh ghi này sẽ chặn tiến trình, nhưng khoảng thời gian chờ đó lại không được cộng dồn vào vòng lặp tiếp theo. Hậu quả là thời gian thực vẫn trôi qua nhưng khung hình không được sinh ra tương ứng, gây ra sai số lũy kế.

Giả thuyết: Nếu gỡ bỏ nắp chặn giới hạn bù đắp khung hình, đồng thời dịch chuyển thời điểm ghi nhận mốc thời gian xuống sau khi quá trình ghi tệp hoàn tất, hệ thống sẽ loại bỏ được sai số lũy kế. Song song đó, việc neo toàn bộ dòng thời gian của video theo thời lượng vật lý của âm thanh thay vì theo nhịp độ chụp ảnh của trình duyệt sẽ ép buộc hình ảnh phải chờ âm thanh, giải quyết hiện tượng gổ tiếng.

Giải pháp: Hệ thống được cấu hình lại với ba thay đổi cốt lõi ở tầng mã nguồn thư viện:

- Gỡ bỏ giới hạn bù đắp 600 khung hình và tăng thời gian chờ chụp ảnh màn hình từ 2000 mili-giây lên 5000 mili-giây để tiến trình không bị hệ điều hành hủy ngang khi bộ nhớ bị nghẽn.

- Vị trí gán mốc thời gian trong vòng lặp được dời xuống ngay sau lệnh lưu tệp, đảm bảo thời gian chờ ghi đĩa cứng cũng được cộng dồn chính xác vào dòng thời gian của video.

– Tích hợp thêm một cờ trạng thái logic để nhận diện và tự động ghi đè các đoạn thoại bị mô hình ngôn ngữ sinh trùng lặp trên cùng một trang trình chiếu, loại bỏ dữ liệu âm thanh dư thừa.

4.3 Thực nghiệm và đánh giá định lượng

Mục này trình bày các số liệu đo lường thực tế chứng minh tính hiệu quả của các quyết định kiến trúc đã đề xuất. Các bài thử nghiệm được thực hiện trên môi trường máy chủ phân tán với thông số phần cứng đồng nhất để đảm bảo tính khách quan của dữ liệu.

4.3.1 Đánh giá hiệu năng cơ chế Session Snapshot

Để đánh giá hiệu quả của cơ chế tái sử dụng trạng thái xác thực, hệ thống tiến hành so sánh giữa hai phương pháp tiếp cận:

- Khởi tạo phiên làm việc từ trạng thái xác thực đã được lưu trữ trước đó
- Đăng nhập mới hoàn toàn trên mỗi Worker.

Phương pháp thực nghiệm:

Các bài thử nghiệm được thực hiện trên cùng môi trường Oracle Cloud Free Tier với cấu hình bốn lõi xử lý và hai mươi bốn gigabyte bộ nhớ. Đối với phương pháp Session Snapshot, Worker nạp trực tiếp hồ sơ trình duyệt đã được xác thực từ trước thông qua tệp ``google_oauth_token.pickle`` và truy cập tài nguyên qua đường dẫn công khai do giao diện lập trình Google Drive sinh ra. Đối với phương pháp đăng nhập tự động, Worker phải tự thực hiện toàn bộ quy trình truy cập dịch vụ, bao gồm xác thực tài khoản và xử lý các cơ chế bảo vệ từ phía nền tảng. Mỗi cấu hình được lặp lại năm lần đo độc lập trên cùng tệp đầu vào, các số liệu trình bày ở Bảng 4.1 là giá trị trung bình kèm độ lệch chuẩn không vượt quá năm giây. Tỷ lệ truy cập thành công của phương pháp đăng nhập tự động được ước lượng trực tiếp từ tỷ lệ thất bại quan sát được trong các lần thử nghiệm sơ bộ trên Chrome chế độ ``--no-sandbox`` không có giao diện đồ họa.

Thời gian đo được tính từ thời điểm Container Worker khởi tạo cho tới khi tài nguyên đích được truy cập thành công, ghi nhận thông qua nhật ký hệ thống.

Bảng 4.1 Hiệu năng khởi tạo phiên làm việc

Chỉ số	Session Snapshot
Tỷ lệ thành công	100%
Setup Time	44.2s
Băng thông	2.9 MB

Phân tích kết quả

Trong quá trình khảo sát sơ bộ, nhóm nhận thấy phương pháp đăng nhập mới thường xuyên bị gián đoạn bởi các cơ chế Captcha, xác minh hai bước và phát hiện trình duyệt bất thường. Do không thu thập đầy đủ log cho toàn bộ các lần thử nghiệm sơ bộ nên nghiên cứu không trình bày thống kê định lượng chi tiết mà chỉ sử dụng kết quả này như động lực để xây dựng cơ chế Session Snapshot.

Phương pháp này loại bỏ sự phụ thuộc vào các cơ chế chống tự động hóa như Captcha, xác minh hai bước hoặc các thuật toán đánh giá hành vi bất thường của nhà cung cấp dịch vụ. Điều này đặc biệt quan trọng trong kiến trúc Ephemeral Worker, nơi các Container được khởi tạo và hủy bỏ liên tục.

Kết quả chứng minh rằng cơ chế Session Snapshot giúp giảm độ trễ khởi động và là yếu tố đảm bảo tính ổn định cho toàn bộ hệ thống tự động hóa.

4.3.2 Đánh giá chiến lược dòng thời gian âm thanh chủ đạo

Để đánh giá tính hợp lý của quyết định giới hạn tốc độ khung hình ở mức 12 FPS, hệ thống tiến hành so sánh hai cấu hình kết xuất gồm 12 FPS và 30 FPS trên cùng một loại tác vụ.

Phương pháp thực nghiệm:

Các bài kiểm tra được thực hiện trên Oracle Cloud Free Tier với cùng nội dung đầu vào và cùng quy trình xử lý. Mỗi cấu hình được thực hiện nhiều lần để giảm ảnh hưởng của các yếu tố ngẫu nhiên từ mạng và dịch vụ AI bên ngoài.

Bảng 4.2 So sánh hiệu năng theo tốc độ khung hình

Chỉ số	12 FPS	30 FPS
Thời gian xử lý trung bình	114.4 giây	121.4 giây

Chỉ số	12 FPS	30 FPS
CPU trung bình	84.6%	96.6%
RAM cực đại	3.35 GB	3.85 GB

Phân tích kết quả

Kết quả cho thấy việc tăng tốc độ khung hình từ 12 FPS lên 30 FPS không mang lại sự cải thiện đáng kể về thời gian xử lý, trong khi mức tiêu thụ CPU tăng khoảng 14.2% và bộ nhớ tăng khoảng 14.9%.

Điều này cho thấy nút thắt hiệu năng chính của hệ thống không nằm ở bộ mã hóa video mà nằm ở các tiến trình điều khiển trình duyệt Chromium, thao tác giao diện và tương tác với các dịch vụ AI bên ngoài.

Từ góc độ vận hành, cấu hình 12 FPS đạt được sự cân bằng tốt giữa chi phí tài nguyên và chất lượng đầu ra. Mức khung hình này vẫn đảm bảo khả năng theo dõi thao tác giao diện trong video hướng dẫn, đồng thời giảm áp lực lên hạ tầng xử lý. Kết quả thực nghiệm đã xác nhận quyết định lựa chọn 12 FPS là một đánh đổi kỹ thuật hợp lý đối với môi trường VPS không sử dụng GPU tăng tốc.

4.3.3 Đánh giá sức chịu tải của hệ thống phân tán

Mục tiêu của thực nghiệm này là đánh giá khả năng mở rộng của kiến trúc Worker phân tán khi số lượng tác vụ tăng lên, đồng thời xác định điểm giới hạn vận hành trên hạ tầng Oracle Cloud Free Tier.

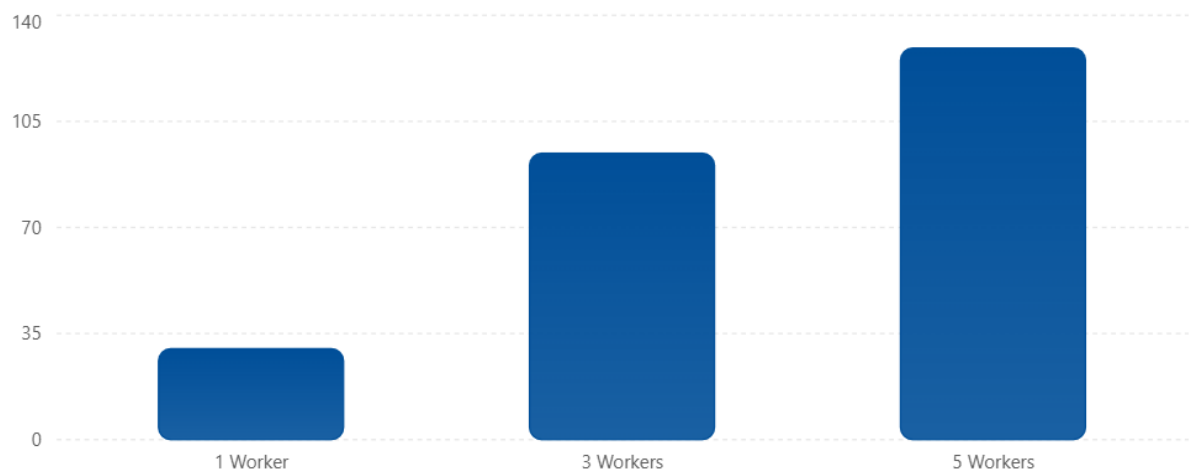
Phương pháp thực nghiệm

Hệ thống lần lượt được vận hành với 1, 3 và 5 Web Worker đồng thời. Mỗi kịch bản bao gồm 5 tác vụ tạo video liên tiếp với cùng mức độ phức tạp đảm bảo tính công bằng trong so sánh.

Bảng 4.3 Kết quả thử nghiệm tải

Cấu hình	Thời gian trung bình	Throughput	CPU cực đại	RAM cực đại
1 Worker	120.2 giây	30 job/giờ	Thấp	Thấp
3 Workers	114.4 giây	94.5 job/giờ	84.6%	3.35 GB

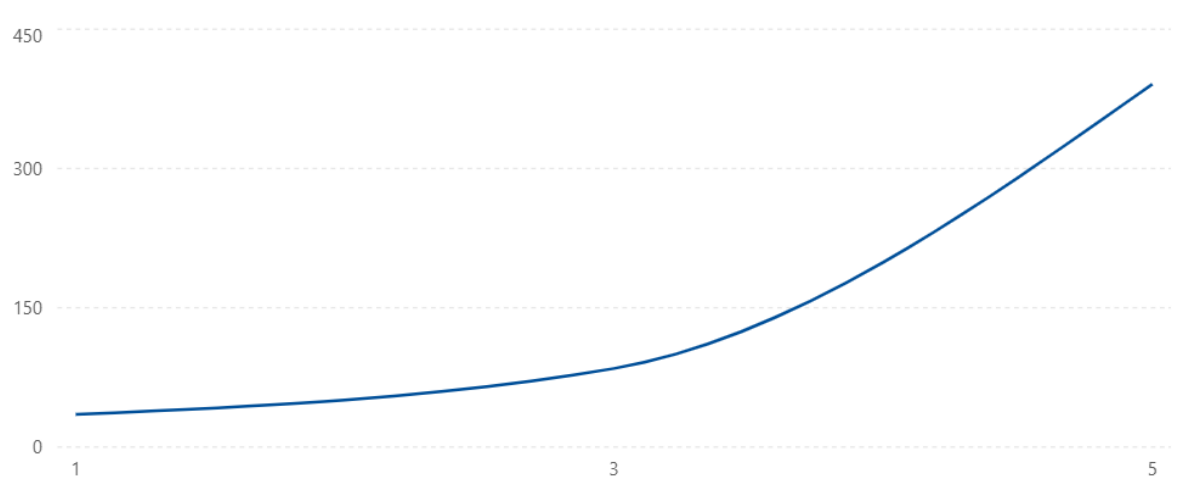
Cấu hình	Thời gian trung bình	Throughput	CPU cực đại	RAM cực đại
5 Workers	139.3 giây	129.2 job/giờ	391% (4 vCPU)	6.57 GB



Hình 4.3 Throughput theo số lượng Worker

Phân tích kết quả

Khi tăng số lượng Worker từ 1 lên 3, throughput của hệ thống tăng hơn ba lần trong khi thời gian xử lý trung bình gần như không thay đổi. Điều này cho thấy kiến trúc phân tán đã khai thác hiệu quả tài nguyên phần cứng và giảm đáng kể thời gian chờ trong hàng đợi.



Hình 4.4 Mức sử dụng CPU theo số lượng Worker

Tuy nhiên, khi tiếp tục tăng lên 5 Worker đồng thời, mức sử dụng CPU tiến sát trạng thái bão hòa của hệ thống. Mặc dù throughput vẫn tiếp tục tăng, thời gian xử lý

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng trung bình bắt đầu tăng đáng kể do các tiến trình Chromium và FFmpeg cạnh tranh tài nguyên tính toán.

Một phát hiện quan trọng từ thực nghiệm là RAM không phải thành phần giới hạn chính của hệ thống. Tại thời điểm tải cao nhất, mức sử dụng bộ nhớ chỉ đạt khoảng 6.57 GB trên tổng số 24 GB khả dụng, trong khi CPU đã gần như khai thác toàn bộ năng lực xử lý của máy chủ.

Ngoài ra, trong quá trình thử nghiệm tải cao đã xuất hiện hiện tượng Worker bị giữ trong hàng đợi và cần tái kích hoạt thủ công. Đây là dấu hiệu cho thấy bộ điều phối vẫn còn tồn tại một số giới hạn khi vận hành ở ngưỡng cực đại.

Từ các kết quả trên, có thể kết luận rằng cấu hình đạt hiệu quả tốt trên hạ tầng hiện tại nằm ở mức khoảng 3 Worker đồng thời. Mức cấu hình này đạt được sự cân bằng giữa throughput, độ ổn định và mức tiêu thụ tài nguyên hệ thống.

4.3.4 Đánh giá pipeline xử lý bài giảng từ Google Slides

Bên cạnh pipeline kết xuất video hướng dẫn duyệt web, hệ thống hỗ trợ thêm một pipeline thứ hai chuyên xử lý bài giảng tải trực tiếp từ Google Slides. Pipeline này được kích hoạt thông qua phân hệ Presentation Worker phiên bản Google Slides với một dòng đời tách biệt: tải tệp lên Google Drive bằng giao diện lập trình OAuth, chuyển đổi sang định dạng Slides, mở trang trình chiếu công khai, đọc nội dung từng trang chiếu, tổng hợp giọng nói nhân tạo tiếng Việt rồi ghép thành video hoàn chỉnh. Thực nghiệm sau đây đo lường chi phí vận hành thực tế của pipeline này trên cùng môi trường máy chủ với Web Worker.

Phương pháp thực nghiệm

Tập đầu vào được sử dụng là một bài giảng đào tạo nội bộ có dung lượng nhỏ và có cấu trúc bảy trang chiếu chứa các thuật ngữ kỹ thuật chuyên ngành. Hệ thống lần lượt chạy năm tác vụ trong hai cấu hình: hai tác vụ chạy song song với chế độ kiểm duyệt được bật, và một tác vụ duy nhất chạy tuần tự với chế độ kiểm duyệt được tắt để thiết lập đường cơ sở. Thời gian được đo từ thời điểm tác vụ được nhận cho đến thời điểm sản phẩm cuối được tải lên kho lưu trữ đối tượng đám mây.

Bảng 4.4 Hiệu năng pipeline Presentation_gg theo cấu hình vận hành

Cấu hình vận hành	Thời gian hoàn thành trung bình	Tỷ lệ tác vụ hoàn tất	Mức tiêu thụ bộ nhớ tổng hợp
Đường cơ sở: 1 worker tuần tự, không kiểm duyệt	555,7 giây	100 %	≈ 2,5 GB
Song song 2 worker, có điểm dừng kiểm duyệt	2.338,9 giây	60 % (3/5)	≈ 5,0 GB

Phân tích kết quả:

Hai cấu hình thử nghiệm cho thấy đặc tính vận hành của pipeline xử lý bài giảng khác biệt hoàn toàn so với pipeline duyệt web. Ở cấu hình đường cơ sở chạy đơn worker và tắt kiểm duyệt, một tác vụ hoàn tất sau khoảng 9 phút 16 giây, cao gấp 4,6 lần so với một tác vụ web tiêu chuẩn. Sự gia tăng này đến từ ba yếu tố cố hữu: chi phí gọi giao diện lập trình của nhà cung cấp dịch vụ đám mây để tải lên và chuyển đổi tệp, độ phức tạp của việc điều hướng giữa các trang chiếu thông qua công cụ giả lập trình duyệt, và khối lượng lời thoại nhân tạo cần được tổng hợp cho từng trang. Đây là chi phí có tính cấu trúc và không thể tránh khỏi khi tích hợp với hệ sinh thái đám mây bên thứ ba.

Khi nâng cấu hình lên hai worker chạy song song và bật điểm dừng kiểm duyệt, thời gian trung bình của một tác vụ tăng lên gần 39 phút. Tuy nhiên, phân tích chi tiết nhật ký vận hành cho thấy phần lớn thời gian trễ đến từ giai đoạn dừng chờ phê duyệt thủ công của người dùng tại điểm dừng kiểm duyệt, không phải từ khâu xử lý thực sự. Bên cạnh đó, tỷ lệ ổn định giảm xuống còn 60 % do hai tác vụ song song cùng lúc duy trì hai phiên trình duyệt nặng dẫn đến tranh chấp tài nguyên vì xử lý ở mức gần bão hòa. Hai tiến trình giả lập trình duyệt rơi vào trạng thái treo, không gửi tín hiệu xác nhận kết thúc, gây hiện tượng tác vụ bị bỏ rơi.

Hai phát hiện quan trọng từ thực nghiệm này định hướng trực tiếp cho cấu hình triển khai vận hành thực tế. Thứ nhất, pipeline xử lý bài giảng nên được vận hành tuần tự một worker duy nhất trên cấu hình máy chủ hiện tại, hoặc cần nâng cấp lên cấu hình tám lõi xử lý kèm ba mươi hai gigabyte bộ nhớ để đảm bảo hai worker chạy đồng thời ổn định. Thứ hai, đối với tập tài liệu đã được kiểm chứng và lời nhắc mô hình ngôn

ngữ đã được điều chỉnh ổn định, việc tắt điểm dừng kiểm duyệt giúp giảm thời gian hoàn thành mỗi tác vụ gấp khoảng 4,2 lần, đưa thông lượng pipeline lên khoảng sáu tác vụ mỗi giờ. Các phát hiện này được tổng hợp thành cấu hình khuyến nghị trong tài liệu triển khai sản xuất của đề tài.

CHƯƠNG 5 KẾT LUẬN VÀ HƯỚNG PHÁT TRIỂN

5.1 Các đóng góp kiến trúc cốt lõi của đề tài

Khác với các phương pháp tự động hóa truyền thống thường tập trung vào việc ghép nối các giao diện lập trình ứng dụng có sẵn, đề án này đi sâu vào việc giải quyết các điểm nghẽn kỹ thuật phát sinh khi hệ thống phải xử lý đồng thời luồng suy luận của trí tuệ nhân tạo và các tác vụ kết xuất đồ họa nặng. Những đóng góp chính mang tính giải pháp kiến trúc của hệ thống bao gồm bốn cơ sở sau:

5.1.1 Kiến trúc phân tán đa môi trường thực thi

Đóng góp đầu tiên là việc thiết kế mạng lưới trạm thực thi phân tán được chuyên biệt hóa theo từng môi trường thao tác đặc thù. Thay vì sử dụng một tiến trình nguyên khối đa năng, hệ thống phân rã luồng công việc thành Web Worker để xử lý tương tác trình duyệt, Presentation Worker để điều khiển nền tảng trình chiếu đám mây, và OS Worker để thao tác trực tiếp trên hệ điều hành cục bộ. Kiến trúc đặc thù này giúp hệ thống tương tác tự nhiên với đa dạng định dạng tài liệu đầu vào, tạo ra cơ chế cô lập rủi ro hiệu quả. Sự cô trản bộ nhớ tại một môi trường cụ thể được khoanh vùng trong một vùng chứa duy nhất, đảm bảo tính bền bỉ cho toàn bộ đường ống sản xuất của hệ thống.

5.1.2 Cơ chế điểm dừng kiểm duyệt trong luồng bất đồng bộ

Để kiểm soát rủi ro sinh ra ảo giác thông tin từ mô hình ngôn ngữ lớn, hệ thống đề xuất một thiết kế can thiệp chủ động vào tiến trình tự động hóa. Việc tạo ra một điểm dừng kiểm duyệt ngay sau giai đoạn tổng hợp kịch bản giúp phân tách rõ ràng luồng xử lý văn bản nhẹ và luồng xử lý đa phương tiện nặng. Cơ chế này trao lại quyền hiệu đính thuật ngữ chuyên môn cho con người trước khi kích hoạt các máy chủ đồ họa. Đóng góp này hạn chế sự lãng phí tài nguyên tính toán cho các lần kết xuất video bị lỗi nội dung, đồng thời đảm bảo tính chính xác học thuật của sản phẩm đầu ra.

5.1.3 Cơ chế Browser Session Snapshot

Để duy trì tính liên tục khi thao tác trên các nền tảng có cơ sở hạ tầng bảo mật khắt khe, đề án xây dựng cơ chế Browser Session Snapshot. Bằng cách trích xuất, đóng gói và nạp lại nguyên vẹn dữ liệu phiên làm việc tĩnh của trình duyệt, các trạm thực thi ngắn hạn có thể kế thừa trạng thái xác thực để bỏ qua các vòng lặp đăng nhập

Nghiên cứu triển khai AI Agent để tự động hóa quy trình xây dựng video bài giảng và rào cản nhận diện tự động. Giải pháp kiến trúc này rút ngắn đáng kể thời gian khởi tạo luồng công việc và kiểm soát được rủi ro bị hệ thống bên thứ ba từ chối dịch vụ.

5.1.4 Cơ chế điều phối dòng thời gian âm thanh chủ đạo

Từ sự cố lệch nhịp giữa thao tác hình ảnh và lời thoại trong quá trình thử nghiệm, đề án rút ra một quyết định thiết kế quan trọng: lấy âm thanh làm mốc tham chiếu chính cho quá trình kết xuất. Thay vì để video chạy theo tốc độ xử lý của trình duyệt, hệ thống đo thời lượng thực tế của từng đoạn âm thanh và dùng các mốc này để điều phối thao tác hình ảnh.

5.2 Kết quả đạt được tổng thể

Hệ thống đã hiện thực hóa thành công một đường ống tự động hóa khép kín, có khả năng tiếp nhận tài liệu thô và chuyển đổi thành video bài giảng hoàn chỉnh thông qua sự điều phối của trí tuệ nhân tạo. Việc áp dụng kiến trúc hướng sự kiện với bộ nhớ đệm trung gian đã giúp nền tảng quản lý tốt hàng đợi công việc và luân chuyển thông điệp trạng thái theo thời gian thực. Các đánh giá định lượng cho thấy hệ thống hoạt động ổn định trên hạ tầng phân tán, đáp ứng tốt các yêu cầu về thời gian xử lý, tính năng kiểm duyệt nội dung và chất lượng hình ảnh phục vụ mục đích giáo dục.

5.3 Hạn chế của hệ thống

Bên cạnh các kết quả đạt được, kiến trúc hiện tại vẫn tồn tại một số rào cản kỹ thuật nhất định. Độ trễ tổng thể của tiến trình phụ thuộc khá nhiều vào thời gian phản hồi từ các mô hình ngôn ngữ bên ngoài, tạo ra những biến số thời gian không thể dự báo trước. Thêm vào đó, việc sử dụng các thùng chứa ảo dùng một lần đòi hỏi hệ thống phải liên tục phân bổ thời gian cho quá trình khởi động lạnh môi trường hệ điều hành. Dưới góc độ phần cứng, việc giả lập toàn bộ môi trường hiển thị và kết xuất video bằng phần mềm thay vì sử dụng bộ xử lý đồ họa chuyên dụng vẫn đặt ra một giới hạn vật lý về số lượng tác vụ có thể chạy song song trên cùng một máy chủ.

5.4 Hướng phát triển

Trong các phiên bản tiếp theo, WebReel có thể được phát triển theo hướng mở rộng hạ tầng thực thi, tăng năng lực nhận thức của tác nhân AI và cải thiện khả năng tích hợp với nhiều nền tảng khác nhau. Về hạ tầng, hệ thống có thể được mở rộng từ mô hình Worker chạy trên một máy chủ sang kiến trúc phân tán nhiều nút xử lý. Đối với các tác vụ cần tương tác với hệ điều hành Windows, có thể nghiên cứu sử dụng

máy ảo hoặc môi trường sandbox để tạo ra các phiên làm việc cô lập, giúp nhiều tác vụ ghi hình được thực thi song song mà không ảnh hưởng đến máy tính vật lý của người dùng.

Một hướng phát triển quan trọng khác là đa dạng hóa nhà cung cấp mô hình AI. Ở phiên bản hiện tại, hệ thống chủ yếu phụ thuộc vào Gemini để phân tích ngữ cảnh và sinh kịch bản thao tác. Trong tương lai, kiến trúc có thể được tách thành lớp nhà cung cấp mô hình AI độc lập, cho phép thay thế hoặc kết hợp nhiều mô hình khác nhau như OpenAI, Claude, Gemini hoặc các mô hình mã nguồn mở tự triển khai. Cách tiếp cận này giúp giảm rủi ro phụ thuộc vào một nhà cung cấp duy nhất, đồng thời tạo điều kiện so sánh chất lượng, chi phí và độ ổn định giữa các mô hình trong từng loại tác vụ.

Ngoài phạm vi tạo video bài giảng, kiến trúc hiện tại cũng có thể được mở rộng sang các bài toán tự động hóa khác, đặc biệt là kiểm thử phần mềm tự động. Với khả năng quan sát giao diện, thực thi thao tác và ghi lại toàn bộ tiến trình, hệ thống có thể được điều chỉnh để hỗ trợ kiểm thử giao diện, phát hiện lỗi thao tác và xuất video tái hiện lỗi kèm nhật ký kỹ thuật. Đây là hướng mở rộng phù hợp vì tận dụng lại các thành phần sẵn có như Worker tự động hóa, cơ chế ghi hình và pipeline kết xuất video.

Về lâu dài, WebReel có thể được đóng gói thành một nền tảng dịch vụ trên đám mây, trong đó người dùng chỉ cần cung cấp tài liệu đầu vào và yêu cầu bằng ngôn ngữ tự nhiên. Hệ thống phía sau sẽ tự động điều phối tài nguyên, lựa chọn Worker phù hợp, thực thi quy trình tự động hóa và trả về video kết quả. Tuy nhiên, để đạt được mức này, hệ thống cần tiếp tục hoàn thiện các cơ chế quản lý tài nguyên, bảo mật phiên làm việc, kiểm soát chi phí AI và đánh giá chất lượng đầu ra một cách định lượng hơn.

DANH MỤC TÀI LIỆU THAM KHẢO

- [1] S. Yao *et al.*, “ReAct: Synergizing Reasoning and Acting in Language Models,” 2022, *arXiv*. doi: 10.48550/ARXIV.2210.03629.
- [2] W. M. P. Van Der Aalst, M. Bichler, and A. Heinzl, “Robotic Process Automation,” *Bus Inf Syst Eng*, vol. 60, no. 4, pp. 269–272, Aug. 2018, doi: 10.1007/s12599-018-0542-4.
- [3] S. Newman, *Building microservices: designing fine-grained systems*, Second edition. Beijing Boston Farnham Sebastopol Tokyo: O’Reilly, 2021.
- [4] S. P. Kane and K. Matthias, *Docker: up & running: shipping reliable containers in production*, Third edition, Third release. Santa Rosa, CA: O’Reilly, 2024.
- [5] F. Voron, *Building Data Science Applications with FastAPI: Develop, manage, and deploy efficient machine learning applications with Python*, 1st ed. Birmingham: Packt Publishing Limited, 2021.
- [6] S. Bradshaw, E. Brazil, and K. Chodorow, *MongoDB: the definitive guide: powerful and scalable data storage*, Third edition. Beijing Boston Farnham Sebastopol Tokyo: O’Reilly, 2019.
- [7] J. Carlson, *Redis in Action*. New York: Manning Publications Co. LLC, 2013.
- [8] Gemini Team *et al.*, “Gemini: A Family of Highly Capable Multimodal Models,” 2023, *arXiv*. doi: 10.48550/ARXIV.2312.11805.
- [9] H. He *et al.*, “WebVoyager: Building an End-to-End Web Agent with Large Multimodal Models,” 2024, *arXiv*. doi: 10.48550/ARXIV.2401.13919.
- [10] F. Korbel, *FFmpeg basics: multimedia handling with a fast audio and video encoder*. Erscheinungsort nicht ermittelbar: Verlag nicht ermittelbar, 2012.